CII EASTERN REGION HACKATHON · BANKING AND FINANCE TECHNOLOGY

Legacy Banking Modernisation Platform

Research dossier, solution design and novelty analysis — round-2 edition

---

Team StudyEdge · Odisha University of Technology and Research, Bhubaneswar
Tamanna Panda · Swayam Subhankar Sahoo · Adyasha Das · Soumyajit Sarkar
Compiled 4 August 2026 · Revised 12 August 2026 for round 2

Every substantive claim carries a source; the bibliography is the final part. Figures that could not be traced to a primary source are marked **[contested]**; team-authored estimates in the round-2 material are marked **[ASSUMPTION]** rather than presented as fact.

CONTENTS

Research Work

Evidence base for the **Legacy Banking Modernisation Platform** submission. Team StudyEdge · Digital Nurture Hackathon 2026 · Banking and Finance Technology.

Compiled 4 August 2026. Every substantive claim here carries a source; see `06 - Sources.md` for the full list. Where a widely-repeated figure could not be traced to a primary source, it is marked **[contested]** rather than quietly used.

Files

File	What it contains
`00 - Round-2 Briefing.md`	**Read first.** The whole submission in one sitting: the sixty-second pitch, the deck slide-by-slide with carry numbers, the ten figures to memorise with caveats, round-2 Q&A answers, a plain-language glossary, and the deliverables checklist.
`01 - Problem Research.md`	Evidence that the problem is real, large, and specifically Indian: COBOL install base, failed migrations and their cost, RBI's regulatory posture, why verification (not translation) is the binding constraint.
`02 - Solution Research.md`	The technical foundations: the test-oracle problem, differential testing, why COBOL decimal arithmetic breaks naive rewrites, how business rules are extracted today, how test inputs are generated.
`03 - Competitive Landscape.md`	Who already does this. **Read this before the next pitch** — the headline finding is uncomfortable and changes how the idea should be positioned.
`04 - Feasibility and Build Plan.md`	What can actually be built: GnuCOBOL, parsers, public COBOL corpora, coverage tooling, and a staged plan sized to a student team.
`05 - Risks and Judge Questions.md`	The hard questions a Cognizant judge will ask, with defensible answers.
`06 - Sources.md`	Bibliography, grouped by topic, with a confidence rating per source.
`07 - Verification Pass.md`	**Read alongside `03`.** Independent re-check of every load-bearing claim against primary sources: 12 confirmed, 6 corrections, one novelty claim partially falsified, and one piece of prior art found that is closer than anything in `03`.
`08 - Business Plan Research.md`	Round-2 material (CII template): stakeholder mapping, market potential, investments, returns, timelines. Team-authored assumptions are marked **[ASSUMPTION]** and separated from sourced claims. Includes the open items to close before the final round.
`Solution Folder/`	Design options, ranked novelty proposals, the recommended architecture, and pitch language.
`Visuals/`	Ten SVG diagrams explaining the problem, the system and the novelty (8–10 added for round 2). Drop straight into the deck.
`deck-renders/`	The thirteen slides of the round-2 deck, rendered to PNG through PowerPoint. Inlined in the dossier as Part 15.
`citations and research paper/`	Every source organised for citation: `references.bib`, annotated bibliographies by category, and verified word-for-word quotes. Metadata was checked individually; unverifiable entries are marked `INCOMPLETE` rather than guessed.

`Legacy_Banking_Modernisation_Research_R2.pdf` — everything above as one continuous document, diagrams and deck slides inlined, including the round-2 material (Part 13 — Business Plan Research; Part 14 — the round-2 written report; Part 15 — the deck itself). Regenerate with `python build-pdf.py` after editing any markdown file; the markdown is the source of truth. `Legacy_Banking_Modernisation_Research.pdf` is the round-1 edition, kept as a historical artifact.

The three findings that matter most

1. The problem statement holds up. The claim of novelty does not.

Differential testing against the legacy program as oracle is exactly what IBM, AWS and at least one well-funded startup already ship. IBM's watsonx Code Assistant for Z has a "Validation Assistant" that passes identical inputs through the COBOL and the Java and flags divergence. AWS ships Blu Age Compare and Mainframe Modernization Application Testing for functional equivalence. Mechanical Orchard's Imogen is built entirely on a "generate-validate loop" proving behavioural equivalence. Cognizant — the organiser — markets accelerators that "extract business rules from a legacy COBOL application, generate specifications and rewrite the application."

This is not fatal. It is *validating*: the team independently identified the same bottleneck that the industry's most serious players converged on, and industry analysts are now calling it the "behavior-first paradigm." But the pitch line "everyone else's submission assumes their AI is right" is true of other *hackathon teams*, not of the market. Say the first, not the second. `03` develops the honest positioning.

2. The defensible contribution is the loop, not the diff.

Every shipping tool treats extraction and verification as separate phases. The written submission's "reciprocal loop" — extracted rules tell the input generator where the boundaries are, and the differential run then issues a per-rule *verified / unproven* verdict — is the part that is genuinely thin in both the products and the literature. The academic work splits cleanly: rule extraction (COBREX, A-COBREX, COBRAIN) and translation validation (SEDCoT, the WCA4Z testing framework) are separate research lines. Closing that loop, and reporting coverage per business rule rather than per line, is the claim worth making.

Verification note (4 August 2026). A falsification pass against primary sources cost this claim some ground: rule-level reporting already exists as a benchmark metric (AgentModernize's Business Rule Preservation Score), and a paper published five days ago — **Locksmith**, arXiv 2607.28271 — builds the same differential spine and runs it off-mainframe. What survived is the *discrimination condition*, the *refuted* verdict, and the regulatory third reference. Full accounting in `07 - Verification Pass.md`.

3. There is a perfect, regulator-backed demo bug sitting in plain sight.

RBI's Master Direction on Interest Rate on Deposits requires savings interest to be computed on a **daily product basis** and every interest payment on a rupee deposit to be **rounded off to the nearest rupee** — while FCNR(B) deposits round to two decimals. That is a real rounding rule, from a real regulator, that a naive Java rewrite using `double` gets wrong in a way that is invisible until audit. Build the demo around it. See `02 §3` and `04 §5`.

Status note

The PPT deadline (4 August 2026, 10:00 AM) coincides with the compilation of this research, so this material is aimed at whatever comes after the deck — a build phase, a demo round, or a Q&A. Nothing here requires changing the submitted problem statement, which the evidence supports as written.

Round-2 update (12 August 2026). The team advanced. Round 2 is the CII Eastern Region Hackathon (submission 13 August; virtual prelims; finals at ICT East, Kolkata, 24 September). CII's template mandates business-plan fields the round-1 material never covered — target users, market potential, investments, returns, timelines. `08 - Business Plan Research.md` documents the evidence and assumptions behind that content, and `Work Done/Written/Legacy_Banking_Modernisation_Platform_R2.md` is the round-2 edition of the written report.

00 — Round-2 Briefing (read this first)

The whole submission in one sitting, for the team. Everything here is a compression of `01` – `08`; if a claim surprises you, the section reference tells you where the evidence lives. Nothing in this file is new research.

The facts of the round: CII Eastern Region Hackathon · deck due **13 August 2026** as `StudyEdge_Banking and Finance Technology.pptx` (.pptx only, named exactly like that) · virtual prelims · finals at ICT East, Kolkata, **24 September 2026** (travel at our own cost).

---

1. The pitch in sixty seconds

> *Indian banks run their core arithmetic on COBOL written in the 1980s. Everyone thinks leaving is hard because translation is hard — it is not. AWS, IBM and TCS all sell translation. Migrations stall because* ***nobody can prove the new program behaves like the old one****, because the business rules were never written down anywhere except the code.*
>
> *Our platform treats that proof as the product. It extracts the rules from the COBOL with line citations, generates boundary-hitting test inputs from those rules, runs* ***both*** *programs on every input — the legacy binary is its own answer key — and diffs the outputs. Every rule ends up* ***verified, unproven, or refuted****. And we add a third reference no vendor has: the RBI's own written arithmetic, so we can catch the legacy itself being non-compliant.*
>
> *Everything runs on free software on a laptop. The deliverables a bank keeps: a specification it never had, a permanent regression suite, an equivalence claim someone can actually sign, and compliance findings.*

If you only remember one line: **the bottleneck is verification, not code generation — so we sell the proof, not the code.**

---

2. The deck, slide by slide

Fourteen slides (as submitted 13 August — V4). The single point each one exists to make, and the number that carries it. Decide who owns each slide when you split the presentation.

#	Slide	The one point it makes	Carry number
1	Title	Proof, not translation	—
2	Problem	Banks run code nobody can safely change	92 of 100 · $1bn · 0 specs
3	Why it's unsolved	Translation is solved; proof is not	62% · 91.2% · <20%
4	Stakeholders *(R2)*	Everyone can generate; nobody can prove	—
5	Our solution	Deterministic pipeline, AI on a short leash	2 bounded LLM calls
6	How it works	The legacy binary is its own oracle	—
7	The finding *(new in V4)*	₹115 vs ₹116, traced to one ROUNDED — illustrative, say so	₹1 per account per accrual
8	Technical details *(R2)*	Every component is free and citable	9,700 / 9,748 NIST
9	What makes it different	Gate · loop · regulator — not the diff	—
10	Market potential *(R2)*	The spend exists; assurance is the gate	~$1bn · 40,000+ complaints
11	Feasibility & scope	Narrow by design; refuses what it can't verify	—
12	Business plan 1/2 *(R2)*	WHY / HOW / WHAT on one page	—
13	Business plan 2/2 *(R2)*	Costs nil to licence; TSB prices the downside	~£400m
14	References	We did the reading	—

Pacing advice from round 1 still holds: slides 6, 7 and 9 are where the pitch is won; do not rush them to spend time on 12–13, which are there to be *seen complying* with the CII template more than to be read aloud.

---

3. Numbers to memorise

The ten figures worth having cold, with the caveat you must attach if challenged. Full provenance: `01`, `04`, `08`.

Figure	Value	Say it as	Caveat
Top-100 banks on IBM mainframes	92 of 100	"widely reported industry figure"	Medium confidence, not a census
Banking systems still on COBOL	> 40%	Pragmatic Coders, 2025	Medium
Indian core-upgrade spend	~$1bn over 5–10 yrs	BCG, 2024	Analyst estimate, single outlet
Indian vs global bank IT spend	5% vs 7–9% of revenue	"industry analysis"	No primary source pinned. **Demoted to a footnote in V4**; the slide-10 stat tile is now the RBI ombudsman figure below
RBI ombudsman complaints	40,000+ (mobile & internet banking), FY22–23	RBI ombudsman data	Solid — regulator's own numbers; on slide 10 since V4
TSB 2018 total cost	~£400m (£48.65m fine + £32.7m redress)	FCA/PRA enforcement	Solid — regulator's own action
HDFC 2020	RBI froze launches & cards; into 2022	RBI supervisory order	Solid
GnuCOBOL conformance	9,700 / 9,748 NIST tests	GnuCOBOL project	Project claims no formal conformance — volunteer that
Best rule extractor	~62% precision / 74% recall	A-COBREX, ICSE 2025	This is our *argument*, not our embarrassment — extraction alone can't ship
LLM translation correctness	2.1%–47.3%	Pan et al., *Lost in Translation*	Range across models/languages
Deterministic vs agentic cost	up to 3.5× cheaper	arXiv 2605.09894	Controlled study

Rounding rule to quote verbatim if asked: RBI Master Direction — interest on rupee deposits **rounds to the nearest rupee; 50 paise and above rounds up** (clause 4(f)). That single sentence is why a naive Java `double` rewrite fails audit.

---

4. The questions round 2 will ask, and the answers

Round 1's hard questions live in `05`. These are the *business-plan* ones this round's template invites, with the honest answer in one breath each.

"What's your ROI number?" We don't claim one — nothing in our research supports a specific multiple, and inventing one would be exactly the confident-wrong-answer failure our product exists to catch. The counterfactual prices the downside instead: TSB spent ~£400m on a failed cutover; our platform's cost is two orders of magnitude below that. (`08 §4`)

"What does it cost to build?" Licence cost is nil — the whole toolchain is open source, no mainframe, no vendor contract. The prototype is four engineers for twelve weeks on laptops; the only variable cost is bounded LLM inference. Those effort figures are our own estimates and we say so. (`08 §3`)

"How big is the market really?" The honest shape: ~$1bn of committed Indian core-upgrade spend is the demand-side proof; our positioning claim — that the assurance step has no product category and is absorbed as consulting hours — is an argument from the competitive landscape, not a statistic, and we label it that way. (`08`

`§2`)

"IBM / AWS already do this." They ship the differential *mechanism* — we say so on the slide. Three things they don't do: refuse unverifiable programs at intake; close the loop so extracted rules direct input generation and get per-rule verified / unproven / **refuted** verdicts; and check both programs against the RBI's written arithmetic, which can catch the *legacy* being wrong. (`03`, `07`, deck slide 8)

"What exists today?" Nothing is built. This is an idea-stage submission with a feasibility-verified build plan: every component named, every licence checked, twelve weeks staged in `04 §7`. Do not imply otherwise — the research folder's whole credibility rests on us never overclaiming.

"Why should a bank trust your AI?" It shouldn't — that is the design. The model is called once per bounded slice, cannot invent line citations, and everything it says is judged by execution. When the extraction layer is wrong, the verdict says **refuted** and shows the counterexample. We are the only team whose demo plans to show our own AI being caught. (`02`, `07`)

---

5. Glossary — plain language

For overviewing across the team; branch-agnostic. Alphabetical.

- **AST (abstract syntax tree)** — the parsed, structured form of source code that tools can walk; what ProLeap produces from COBOL.
- **Boundary value** — an input right at a rule's edge (₹99,999 / ₹1,00,000 / ₹1,00,001). Bugs live at edges; random inputs almost never land on them.
- **CICS / JCL** — IBM mainframe screen-handling and job-control layers. We *refuse* programs using them, because their behaviour depends on state we cannot reproduce.
- **COBOL** — the 1960s-era language most core banking arithmetic still runs on. Verbose, decimal-exact, extremely stable.
- **Delta debugging** — automatically shrinking a pile of failing inputs to one minimal counterexample. Turns "73 failures" into one line a judge can read.
- **Differential testing** — run two implementations on the same input and compare outputs. No specification needed.
- **gcov / coverage** — tooling that records which source lines an execution actually touched. This is how "unproven" is computed rather than guessed.
- **GnuCOBOL** — free, open-source COBOL compiler (compiles via C to a native binary). Passes 9,700/9,748 NIST conformance tests; makes the whole demo possible without a mainframe.
- **Hallucination** — an LLM stating something false with confidence. Our design assumption is that this *will* happen; the harness exists to catch it.
- **Mutation testing** — deliberately break the legacy program (move a threshold, delete a ROUNDED) and check the test corpus notices. If it doesn't, the equivalence claim was hollow — "testing our own tests."
- **Oracle (test oracle)** — the thing that tells you the *correct* output for a given input. Normally a human-written spec; here, the running legacy program itself.
- **PIC clause** — COBOL's field-type declaration (digits, sign, decimal places). Readable mechanically, so input domains can be derived from it.

- **Program slicing** — extracting exactly the source lines that influence one variable. A slice is a set of line numbers — which is what makes our rule citations mechanical rather than model-generated.
- **RBI Master Direction** — the regulator's binding rulebook. The Interest Rate on Deposits Direction fixes daily-product accrual, the ₹1 lakh uniform-rate threshold, and nearest-rupee rounding — our third reference.
- **Reciprocal loop** — our core design: extracted rules tell the input generator where boundaries are; execution then judges the rules. Each layer checks the other.
- **Transpiler** — a program that converts source code between languages (COBOL → Java). Mature, commercial, *not* the bottleneck.
- **Verified / unproven / refuted** — our three per-rule verdicts: confirmed by execution / never exercised, flagged / contradicted by execution (the extraction was wrong).
- **VSAM** — mainframe file storage. A program reading it has external state, so it fails our intake gate.

---

6. Deliverables and where everything lives

Deliverable	File	State
Round-2 deck (submit this)	`Presentation(Work Here)/StudyEdge_Banking and Finance Technology_V2.pptx`	Ready — **rename to drop `_V2` on upload**
Written report, round-2 edition	`Work Done/Written/Legacy_Banking_Modernisation_Platform_R2.md` + `.pdf`	Ready
Research dossier, everything	`Legacy_Banking_Modernisation_Research_R2.pdf` (this document)	Ready
Business-plan evidence	`08 - Business Plan Research.md` (Part 13)	Ready
Deck as images	`deck-renders/` (Part 15)	Ready

Open items before Kolkata (owners needed — see `08`): pin the 5%-vs-7–9% IT-spend figure to a named report or swap in the RBI ombudsman figure; validate the four stakeholder pain statements with one practitioner; decide whether we want a rupee costing, and if so agree the rate assumption out loud.

01 — Problem Research

Evidence for each claim in the submitted problem statement. Structured so that any sentence in the pitch can be traced to a source.

---

1. The install base is real and it is not shrinking

Claim	Figure	Source	Confidence
COBOL still in production	220 billion lines (2017 estimate, still widely cited)	Reuters/industry, repeated in Pragmatic Coders 2025 legacy stats	Medium
Revised estimate	800 billion lines (2022 Micro Focus survey)	Micro Focus / secondary reporting	**[contested]** — a survey extrapolation, not a census. The jump from 220bn to 800bn reflects better counting, not growth. Use "hundreds of billions of lines" if challenged.
Banks running legacy cores	~70% of banks globally	Pragmatic Coders, 2025	Medium
Banking systems using COBOL	>43%	Pragmatic Coders, 2025	Medium
Top-100 banks on IBM mainframes	92 of 100	Widely reported industry figure	Medium
Daily commerce processed	~$3 trillion/day	IBM-sourced figure, widely repeated	Medium
ATM transactions on COBOL	~95% of swipes	Widely repeated	**[contested]** — traces back to a single 2017 press item. Avoid in the deck.

How to use this: one or two figures, attributed, is more credible than a wall of them. The strongest pairing is *"92 of the world's top 100 banks run on mainframes, and over 40% of banking systems still run COBOL"* — both defensible, both make the scale point without inviting a fact-check argument.

---

2. The workforce argument — real, but the numbers are soft

Commonly cited: average COBOL developer age 55–58; roughly 10% of that workforce retiring each year; ~75% expected to retire within a decade; ~24,000 active COBOL developers in the US against hundreds of billions of lines; a Micro Focus survey in which 60% of COBOL-using organisations named finding qualified developers as their single biggest challenge.

Treat all of these as [contested]. They circulate through vendor blogs and recruitment-site content marketing, and the primary sources are old (a Computerworld survey found 46% already noticing a shortage — but that article dates from the 2000s). The *direction* is not in doubt; the precision is.

Better framing for the pitch: the workforce point is not that COBOL programmers are scarce. It is that **the people who wrote the undocumented business rules are gone**, which is a stronger and more specific claim, and it is the one that actually motivates the solution. A bank can hire a COBOL contractor. It cannot hire

back the 1994 engineer who decided how to handle an account opened on 31 January.

---

3. India-specific evidence

This is the part that makes the submission fit the theme rather than being a generic global tech pitch.

The investment gap. BCG estimates Indian lenders must invest **~$1 billion over 5–10 years** to upgrade legacy core banking systems, most of which were set up in the 1990s on monolithic, tightly-coupled architectures unsuited to horizontal scaling.

Chronic underinvestment in IT. Global banks spend 7–9% of revenue on IT; Indian banks up to 5%. That gap compounds: less budget → fewer modernisation attempts → more accumulated undocumented logic.

> ***Citation caution (12 Aug 2026):*** *this 5%-vs-7–9% comparison has no primary source pinned — it circulates through consulting and analyst commentary (likely Gartner / Celent / McKinsey banking-IT benchmarks). It is the weakest citation on the round-2 deck. Either pin it to a named report and year, or swap in the RBI ombudsman complaints figure below, which is regulator-sourced. See* `08 §2` *.*

Failure is already visible to the regulator. RBI's ombudsman recorded over **40,000 mobile and internet banking complaints** across FY22 and FY23.

The regulator has already acted on IT fragility. On 2 December 2020 the RBI ordered HDFC Bank — India's largest private bank — to halt all new digital launches under *Digital 2.0* and stop issuing new credit cards, following repeated outages (penalised for incidents in November 2018 and December 2019, then a data-centre power failure on 21 November 2020). The credit-card restriction was relaxed only in August 2021; the Digital 2.0 restriction ran into 2022. RBI also ordered an external audit of the bank's IT infrastructure.

Why this matters for the pitch: it demonstrates that in India, an IT failure in a bank is not an engineering incident, it is a *supervisory* event that can freeze the bank's product roadmap for over a year. That is exactly the risk calculus that keeps a bank from touching a COBOL accrual engine — and it is a far more relevant example for a CII/Cognizant audience than a UK bank.

---

4. What migration failure actually costs — the TSB case

The single best-documented example, and worth one slide.

- April 2018: TSB migrated from Lloyds' systems to Sabadell's *Proteo4UK* platform. Data migrated; the platform then failed immediately.
- **1.9 million of 5.2 million customers** locked out — this figure comes from press reporting (The Register, IEEE Spectrum), corroborated across several outlets. The FCA's own release says only *"all of TSB's branches and a significant proportion of its 5.2 million customers"*; attribute accordingly. Failures spanned digital banking, telephone banking, branch technology, payments and debit cards. Business-as-usual was not restored until December 2018.
- The FCA and PRA jointly fined TSB **£48.65 million** for operational risk management and governance failures.

- **£32.7 million** paid in customer redress.
- Total direct cost around **£400 million**. The CEO resigned. The former CIO was personally fined **£81,620**.

The point to make with it: TSB did not fail because the new code would not compile. It failed because nobody could prove, before cutover, that the new system would behave like the old one under real conditions. The regulator's finding was about *risk management and assurance*, not about programming. That is a verification failure, and it is exactly the failure this project targets.

---

5. The verification bottleneck — is the thesis actually defensible?

Yes, and it is now the mainstream expert view rather than a contrarian one.

Translation is demonstrably not the hard part any more. Commercial automated refactoring (AWS Blu Age, TCS MasterCraft TransformPlus, Micro Focus/OpenText) has produced COBOL→Java for years. AWS Transform for mainframe went generally available in **May 2025** as an agentic modernisation service. ADP used it to extract business rules from a legacy tax-compliance mainframe, cutting rule-extraction time by 80% and manual effort by over 90%.

But translation output cannot be trusted without checking. The evidence:

- *Lost in Translation* (Pan et al.) translated 1,700 code samples across C, C++, Go, Java and Python. **Correct translations ranged from 2.1% to 47.3%** depending on the model. The authors released 1,748 manually labelled bugs across **15 categories** of translation error. Their conclusion in the abstract is blunt: LLMs are not yet reliable for automated code translation.
- The IBM Research paper on testing WCA4Z states it directly: *"the resulting code cannot be trusted to correctly translate the original code."* That is IBM, about IBM's own product.
- Subtle operator-semantics differences cause **silent** failures — the canonical example being modulo on negative operands, where Python takes the divisor's sign and Java and C take the dividend's. Silent means no crash, no exception, just a wrong number.

And the specification genuinely does not exist. This is the load-bearing claim of the whole submission, and it is supported by the existence of an entire research subfield devoted to recovering it: business-rule extraction from COBOL (see `02 §4`). Tools like IBM ADDI exist precisely because the dependency map and the rules must be *reconstructed* from source. Nobody builds a reconstruction tool for information that was written down.

The honest caveat: the best rule-extraction tools reach roughly **62% precision and 74% recall** (A-COBREX, on fuzzy matching against annotated ground truth). That is the state of the art in 2025. It is also the strongest possible argument for the team's central point — *extraction alone is not safe to ship*.

---

6. Why banks rationally do nothing — the regulatory arithmetic

RBI's IT Governance Master Direction (7 November 2023, effective 1 April 2024) requires regulated entities to maintain a board-approved IT governance framework, including a **board-approved Change Management Policy**. Any change to a core accrual engine is therefore not an engineering decision; it is a board-visible, auditable risk event.

RBI's Interest Rate on Deposits Directions fix the arithmetic itself. Verified against the primary source — Master Direction (Interest Rate on Deposits) Directions, **2016**, updated as on 7 June 2024:

- **Clause 6(a)** — interest on domestic rupee savings deposits is calculated on a **daily product basis**.
- **Clause 6(a)(1)–(2)** — a **uniform rate up to ₹1 lakh**, irrespective of the amount within that limit; **differential rates permitted** on end-of-day balance above ₹1 lakh.
- **Clause 4(f)** — *"All transactions, involving payment of interest on deposits shall be rounded off to the nearest rupee for rupee deposits and to two decimal places for FCNR (B) deposits."*

A related RBI circular states the rounding rule exactly: *fraction of 50 paise and above is rounded to the next higher rupee; less than 50 paise is ignored*. That is **HALF_UP at the rupee**, written down by the regulator.

A 2025 restatement of these Directions exists; its clause numbering has not been verified, so cite the 2016 clauses. See `07 - Verification Pass.md §C1`.

Put these together and the "one paisa across millions of accounts" line in the submission is not rhetoric. Rounding behaviour is a *prescribed* regulatory parameter. A rewrite that rounds differently is not a bug report — it is non-compliance with a Master Direction, discovered at audit, across the entire deposit book, retrospectively.

Conclusion the evidence supports: the bank's refusal to modernise is a correct decision under its own risk framework. The only thing that changes the decision is evidence of equivalence. That is the product.

---

7. Market size — for the "why does this matter commercially" slide

- Mainframe modernisation market: **$8.39bn (2025)** → **$13.34bn (2030)**, CAGR 9.7% (MarketsandMarkets). Other houses put 2025 between $6bn and $10bn with 8–13% CAGR — cite the range, not a single number.
- Named drivers: workforce retirement, stricter regulation, data growth. Banking, insurance and government are the lead verticals.
- The fastest-growing segment is **software** — automated code conversion, testing suites, AI modernisation engines — not services. That is the segment this project sits in.
- India-specific: the ~$1bn BCG figure above is the addressable core-banking upgrade spend for Indian lenders alone.

---

8. One-paragraph version, for the deck

> *India's banks run deposit and loan arithmetic on COBOL written in the 1980s and 1990s. Over 40% of banking systems worldwide still do; 92 of the top 100 banks run on mainframes. BCG puts the bill for upgrading Indian core banking systems at a billion dollars. The blocker is not translation — AWS, IBM and TCS all ship automated COBOL-to-Java conversion today. The blocker is that no specification of the current behaviour exists, so no one can prove the rewrite is equivalent. IBM's own research paper on its translation product says the output "cannot be trusted." When a bank guesses wrong, it costs what TSB's migration cost: 1.9 million customers locked out, a £48.65m regulatory fine, £400m all-in. And RBI writes the rounding rule down — interest on rupee deposits, rounded to the nearest rupee, on a daily product basis — so a rounding drift is not a bug, it is a Master Direction breach across the whole deposit book.*

02 — Solution Research

The technical foundations under each stage of the proposed system, and what the literature says works.

---

1. The idea has a name, and a forty-year literature: the test oracle problem

The submission's central move — *"the legacy program already defines the right answer by running"* — is a known and respected technique. Naming it correctly makes the pitch stronger, not weaker.

The test oracle problem is the difficulty of determining the expected output for a test case, and of deciding whether the actual output is acceptable. It is the reason testing undocumented systems is hard. The canonical survey is Barr, Harman, McMinn, Shahbaz and Yoo, *The Oracle Problem in Software Testing: A Survey* (IEEE TSE, 2015).

There are three standard ways around it:

Technique	How it dodges the oracle	Fit here
Differential testing	Run two implementations on the same input; disagreement is a bug in one of them.	**This is the project.**
Metamorphic testing	Assert relations between outputs of related inputs (e.g. doubling the balance should not decrease interest) without knowing either value.	Useful *secondary* technique — see §6.
Intramorphic testing	Compare against a deliberately modified version of the system itself.	Not applicable.

Differential testing is described in the literature as *"a common practice to alleviate the oracle problem"* — transforming it into a more easily derived form by comparing multiple implementations on the same generated inputs.

The industrial precedent to cite: Diffy. Built at Twitter, later used at Airbnb, Baidu and ByteDance, and open-sourced. Diffy runs new and old service instances side by side, multicasts each request to both, and compares responses to report regressions. Its stated premise: *if two implementations return similar responses across a sufficiently large and diverse set of requests, they can be treated as equivalent.* The Twitter engineering post was titled "Testing services without writing tests."

The same idea appears in banking practice as **dual-run / parallel-run migration**: dual-write (every transaction hits both cores, discrepancies surface immediately), shadow accounting (new core processes real transactions but output is not used for settlement), and blue-green cutover. Practitioners are explicit that *dual-running without automated reconciliation creates false confidence* — which is precisely the gap a rule-level verdict fills.

What this gives the pitch: the mechanism is not speculative. It is what Twitter did to its services and what banks already do at cutover. The project's contribution is applying it *before* the cutover decision, at the level of individual business rules, on a laptop, with no mainframe required.

---

2. Why LLM translation needs this — the quantitative case

Finding	Source
LLM code translation correct in **2.1%–47.3%** of cases across 1,700 samples; 1,748 labelled bugs in **15 categories**	*Lost in Translation*, arXiv 2308.03109
"The resulting code cannot be trusted to correctly translate the original code"	IBM Research, *Automated Testing of COBOL to Java Transformation*, arXiv 2504.10548
COBOL is a **low-resource language** for LLMs, with distinctive logic patterns causing severe accuracy degradation; a symbolic-execution + delta-debugging repair loop improves accuracy by **at least 12%** over state of the art	SEDCoT, arXiv 2607.04092
Subtle operator-semantics differences produce **silent** failures (canonical case: modulo on negative operands differing between Python and Java/C)	arXiv 2605.02195
Code-summarisation hallucination rates measured as high as **66%**, reduced to 59% by mitigation	*Hallucinations in LLM-Based Code Summarization*, PACMSE 2026

That last row is the direct evidence for the submission's line *"a confidently stated wrong rule is more dangerous than no rule at all."* It is not a rhetorical flourish — hallucination in code *explanation* (not generation) is a measured phenomenon with published rates.

One more finding worth building on. A 2026 controlled study comparing deterministic orchestration against fully agentic, LLM-controlled orchestration for COBOL-to-Python modernisation held models, prompts, tools and source programs constant and varied only the control strategy. Deterministic orchestration **matched** agentic accuracy while reducing token consumption by **up to 3.5×** and improving worst-case robustness and run-to-run variance. The authors conclude that structured modernisation workflows with validation stages should use deterministic execution.

Implication for the build: a fixed pipeline (parse → extract → generate inputs → run both → diff → verdict) with LLM calls at specific bounded points is not the naive design; it is the *empirically better* one. Say so — it turns what looks like a simplification into a defended architectural choice.

---

3. The arithmetic is where the bugs actually are

This section is the technical heart of the demo. It is also the most concrete thing the team can show a judge.

COBOL's number model

COBOL financial fields are typically `USAGE PACKED-DECIMAL` (`COMP-3`): each digit stored in 4 bits, with a trailing 4-bit sign nibble. Packed decimal represents decimal numbers **exactly** — no binary-fraction rounding error — which is why it is used for money. A `PIC S9(7)V99 COMP-3` field means precisely seven integer digits and two decimals, and the compiler enforces truncation at that boundary.

COBOL arithmetic statements also carry an explicit `ROUNDED` phrase, and `ON SIZE ERROR` handling for overflow. The rounding behaviour is part of the statement, not a library default.

Where a rewrite goes wrong

- **Using `double` / `float` in Java.** IBM's own COBOL/Java interoperability documentation warns that conversions between COBOL `COMP-1` / `COMP-2` and Java `float` / `double` may lose precision because the underlying representations differ. Binary floating point cannot represent 0.1 exactly; decimal can.
- **The correct target is `BigDecimal`**, with an explicit scale and an explicit `RoundingMode`. `BigDecimal` is both the documented legal partner for zoned and packed decimal items in COBOL/Java interop, and what Micro Focus COBOL uses under the covers when compiling to JVM bytecode.
- **Getting `BigDecimal` but the wrong `RoundingMode`.** `HALF_UP` versus `HALF_EVEN` (banker's rounding) differ only on exact midpoints — a case that arises rarely per account and constantly across millions of accounts. This is the bug class that is invisible in unit tests written by the rewrite team and obvious to a differential harness that generates midpoint inputs deliberately.
- **Truncation points.** COBOL truncates at the `PIC` boundary on every intermediate store. A Java rewrite that keeps full precision through a chain of operations and rounds once at the end produces a *different, arguably more correct*, and therefore non-compliant answer.

The regulatory hook — build the demo on this

RBI's Master Direction on Interest Rate on Deposits specifies:

1. Interest on domestic rupee savings deposits is calculated on a **daily product basis** — "daily product" meaning interest applied on the end-of-day balance.
2. A **uniform rate** on balances up to ₹1 lakh, irrespective of amount within that limit; **differential rates permitted** on end-of-day balance above ₹1 lakh.
3. **All transactions involving payment of interest on deposits shall be rounded off to the nearest rupee** for rupee deposits; **two decimal places** for FCNR(B) deposits.

Item 2 gives a **threshold boundary at ₹1,00,000** — test 99,999 / 1,00,000 / 1,00,001. Item 3 gives a **rounding midpoint** — an accrual landing on ₹x.50. Item 1 gives **date boundaries** — month ends, quarter ends, 29 February, accounts opened on the 31st.

Those are not invented edge cases. They are the boundaries the regulator itself defines, and a demo that finds a divergence at exactly one of them is self-evidently meaningful to a banking judge.

Day-count conventions — the other family of divergence

Interest accrual depends on the day-count basis: 30/360 (every month 30 days, year 360), Actual/365, Actual/360, Actual/Actual. India uses 30/360 for G-Secs. The choice materially changes the accrued amount, and a legacy program's convention is frequently *implicit* in the arithmetic rather than named anywhere. A rewrite that assumes Actual/365 against a legacy 30/360 produces plausible, wrong numbers on every account, every day. This is a textbook case of a rule that exists only in the code.

---

4. Stage 01 — extraction: what already works and how well

Business-rule extraction from COBOL is an established research line. Knowing the numbers protects the team from overclaiming.

Approach	Method	Reported performance
COBREX	Rule-based; control-flow-graph slicing on business variables	F1 ≈ 0.59 against ground truth
A-COBREX (ICSE 2025 demo, IBM Research)	Extends to rules involving multiple business variables	Precision 62.21%, recall 74.12% (fuzzy match, 27 annotated programs)
COBRAIN (EASE 2025)	LLM, few-shot prompting	Precision 1.0 / recall 0.746 vs COBREX; F1 0.73 vs ground truth
Cosentino et al.	Model-based framework: represent COBOL as a model, identify business-concept variables, slice	Foundational, pre-LLM

Two things to take from this table:

1. **Even the best published extractor is wrong roughly a quarter to a third of the time.** The submission's refusal to ship extraction as the deliverable is not modesty; it is the correct reading of the state of the art.
2. **LLM extraction beats rule-based extraction** (COBRAIN F1 0.73 vs COBREX 0.59), so using a model for this stage is defensible — provided the output is verified downstream.

A known structural failure mode, worth quoting: automated rule-extraction techniques that query for code structures and variables are *"high recall, low precision"*, producing lists dense with false positives. That is exactly the condition that makes traceability essential — an engineer needs to confirm or reject each candidate rule by reading three lines, not four thousand.

Slicing is the mechanism to name. Program slicing on business-concept variables is how both COBREX and the model-based frameworks isolate a rule. It also naturally produces the line-level provenance the submission promises, because a slice *is* a set of source lines. This is worth saying explicitly: traceability is not bolted on, it falls out of the extraction method.

---

5. Stage 03 — input generation, the acknowledged hard part

The submission already names this as the limiting factor. The literature agrees and offers four families of technique.

a) Rule-directed boundary value analysis. Derive boundaries from the extracted rules and test at, just below, and just above each. Cheapest to build, highest yield per unit of effort, and it is the mechanism that closes the reciprocal loop. A 2025 paper on LLM-driven boundary-value input generation found prompt-engineered boundary generation effective for fault detection and coverage — relevant because the boundaries here come from natural-language rules.

b) Symbolic / concolic execution. Replace inputs with symbolic values, accumulate path constraints, solve with an SMT solver to obtain inputs reaching each path. KLEE is the reference implementation for C/C++.
Directly relevant precedent: IBM's WCA4Z testing framework uses *symbolic execution to generate unit tests from COBOL programs* (mocking external dependencies) and converts them to JUnit to validate the Java translation. SEDCoT likewise uses symbolic execution with LLM guidance to generate suites that expose semantic discrepancies. The known limitation: treating all inputs as symbolic produces formulae that blow up within a few nested conditions.

c) Grammar-aware random / property-based generation. Cheap volume from the `PIC` clauses themselves — a `PIC S9(7)V99` field defines its own valid domain, sign, and extremes. Good for breadth; will not find a rounding bug alone, as the submission correctly notes.

d) Delta debugging on failures. SEDCoT's third phase: once a divergence is found, shrink the failing input to a minimal counterexample. This is what turns "these 400 inputs diverge" into "the divergence occurs whenever the accrual lands on an exact half-paisa." Cheap to implement, disproportionately valuable for the demo, and it is what makes a finding *actionable* rather than merely reported.

Recommended combination for a hackathon build: (a) as the headline mechanism, (c) for volume, (d) for presentation quality. Mention (b) as the scaling path and cite the IBM precedent — do not attempt to build it.

---

6. Metamorphic relations — a cheap second safety net

Where a rule is extracted but the differential run cannot reach it, metamorphic relations can still assert something. Domain-obvious relations for interest accrual:

- Monotonicity: higher principal at the same rate over the same period must not produce lower interest.
- Additivity across periods: accruing Jan then Feb should equal accruing Jan–Feb, under a stated convention.
- Zero: a zero balance accrues zero.
- Scaling: doubling the principal under a flat-rate tier doubles the interest.

These hold without knowing the correct value, so they apply to inputs the differential harness has not exercised. Worth one line in the deck as evidence of depth; not worth building first.

---

7. Honest assessment of the "reciprocal loop"

The submission's strongest original claim is that extraction and verification strengthen each other. Testing that claim against the literature:

Supported. SEDCoT is a working instance of one direction — symbolic execution generating tests that expose semantic errors, then feeding repair. The WCA4Z framework explicitly generates *feedback to improve the underlying model*. So "verification improves generation" is established.

Thin in the literature. The other direction — *extracted natural-language business rules driving the input generator's choice of boundaries* — is not something the surveyed work does. Rule extraction and translation validation are separate research lines with separate tools and separate papers.

Partially found — corrected after verification. A per-rule verdict is *not* unprecedented. **AgentModernize** (arXiv 2605.17535) defines a **Business Rule Preservation Score**: the percentage of gold-standard rules represented in the modernised output, counted as preserved when the rule appears in the behavioural specification *and* the modernised code enforces it, as verified by targeted tests. That is rule-level reporting.

What remains open, checked against that paper: BRPS scores against **human-annotated gold-standard rules** — presupposing the very artifact a bank does not have — whereas the verdict proposed here applies to rules the system extracted itself with no ground truth. And nothing surveyed requires a rule's boundary to be *behaviourally live* before calling it verified, or issues a **refuted** verdict when the evidence contradicts the rule.

So the contribution is narrower than first claimed: not per-rule reporting, but the discrimination condition and the refutation verdict on self-extracted rules. See `07 - Verification Pass.md §C3`.

Take this pair of numbers from that paper. AgentModernize's specification graph captured **91.2% of gold-standard rules**, while its Behavioural Equivalence Rate reached only **9.4%–19.4%** across models, with baselines at **0.0%**. Extraction works; behavioural equivalence does not follow from it. That is the sharpest single argument in this dossier for why verification, not extraction, is the product.

03 — Competitive Landscape

Read this before the next pitch. The core mechanism proposed in the submission is already shipping from three major vendors and at least one venture-backed startup — including the organiser, Cognizant. This changes what the team should claim, not whether the project is worth building.

---

1. The uncomfortable finding, stated plainly

The submission's central mechanism — *run the legacy program and the rewrite on identical inputs, diff the outputs, treat the legacy as the oracle* — is prior art in commercial products.

Vendor	Product	What it does
IBM	watsonx Code Assistant for Z — **Validation Assistant**	Ensures semantic equivalence between refactored COBOL and transformed Java using automated test generation. Passes the same data inputs through both, compares outputs, flags divergence, helps fix the Java. Backed by an internal testing framework using **symbolic execution** on COBOL to generate JUnit tests, and a separate automated quality-evaluation system combining analytic checkers with LLM-as-a-judge (ASE 2025).
AWS	Mainframe Modernization — **Blu Age Compare** + **Application Testing**	Functional equivalence testing against reference datasets from the mainframe/iSeries. Application Testing (preview Nov 2023) automates functional equivalence testing at scale: test capture, on-demand automated replay, comparison, regression. Quality gates cover completeness (Blu Insights), functional equivalence (Blu Age Compare), performance equivalence, code quality (SonarQube), coverage (JaCoCo).
AWS	**AWS Transform for mainframe** (GA May 2025)	Agentic modernisation: code analysis, business-rule extraction, technical documentation, code refactoring, domain decomposition. ADP cut rule-extraction time 80% and manual effort >90%. Dec 2025: application reimagining.
Mechanical Orchard	**Imogen**	The closest match to the submission's entire thesis. Captures what the system actually does through real production data flows across component interfaces, uses that behavioural specification to guide *and validate* AI code generation, and proves equivalence at every step via a **"generate-validate test loop."** Cuts over only after audit-level behavioural parity is proven. On AWS Marketplace; partnered with Thoughtworks and Leidos; Summer 2026 update integrates AWS Transform business-rule extraction.
Cognizant *(the organiser)*	Modernisation accelerators, Flowsource, Neuro, ZDLC	Marketed capability: *extract business rules from a legacy COBOL application, generate specifications, and rewrite the application into a modern cloud-native Java architecture.* Also markets gen-AI "legacy understanding" — reverse-engineering applications to fill documentation gaps and extract business rules.
TCS	MasterCraft TransformPlus	Automatically extracts application knowledge including business rules, then automates COBOL→Java conversion. v5.0 adds ML-enhanced analysis.
IBM	ADDI	Static analysis: inventory, dependency mapping, business-rule extraction across COBOL, JCL, DB2, IMS.

Added after verification — the closest prior art of all. Locksmith (arXiv 2607.28271, *Agentic Method for Deterministic Validation of Legacy Code Migration*, **30 July 2026**) uses the legacy COBOL as an execution oracle via a *"Parity Gate"*, runs **both targets off-mainframe on commodity hardware** with mocked external calls, generates inputs by *"Witness Search"*, and reports end-state discrepancies. Three case studies, 430–4,114 lines, **91.90% branch coverage** on a production-like program, full parity on all accepted tests.

This is the same spine as the proposed system, published five days before this dossier. It costs the "no mainframe needed" claim its absolute form — that is now true of every *shipping product* but not of the research frontier. Verified against the paper, it reports **no business-rule verdicts and no regulatory checks**, and its mutations are *parity-preserving* for path exploration rather than fault injection for adequacy. Cite it as validation that the architecture works, then say what it does not do.

An industry analyst has already named the category: the **"behavior-first paradigm."** The argument in that analysis is nearly identical to the submission's — direct LLM translation *"introduces compounding logical discrepancies"*, the real problem is *undocumented institutional knowledge*, and pure code conversion just relocates technical debt.

---

2. What this does and does not mean

It does not mean the idea is bad. Independently reaching the same diagnosis as IBM Research, AWS, and a Thoughtworks-partnered startup is a strong signal that the diagnosis is correct. Most hackathon submissions do not survive contact with the market this well.

It does mean two lines in the current write-up must change.

> *"Everyone else's submission assumes their AI is right. Ours assumes it is wrong, and proves the answer anyway."*

True of the other teams in the room. Not true of IBM. If a Cognizant judge who works on mainframe modernisation hears this, the response is *"we ship that."*

> *"Extraction cannot be the product. Verification has to be the product."*

Keep this — it is correct and well-supported. But it is a statement of the *industry's* current direction, not a claim of originality. Framing it as "this is where the serious players have converged, and here is that architecture built on an open stack" is both honest and more impressive than claiming to have invented it.

---

3. The gaps that are genuinely open

Four things the surveyed products and papers do not do. These are where the claim of contribution should sit.

a) Per-rule verification verdict — narrowed after verification

Every surveyed *product* reports equivalence at the level of test cases, and coverage tooling in the AWS stack is JaCoCo — Java line and branch coverage. But in the *literature*, AgentModernize's Business Rule Preservation Score does report at rule level, so the concept is not original (`07 §C3`).

What is still open: BRPS scores against human gold-standard rules, while the verdict proposed here applies to **rules the system extracted itself**; and nothing surveyed requires a boundary to be behaviourally live, or issues a **refuted** verdict. Claim the discrimination condition and the refutation, not per-rule reporting as such. It

requires mapping each extracted rule to source lines (which slicing gives for free) and each execution to the lines it touched (which coverage instrumentation gives), then intersecting them.

b) Rules driving input generation

IBM generates test inputs by symbolic execution on the COBOL. AWS captures inputs by replaying real mainframe traffic. Mechanical Orchard observes live production data flows. **None of them derives test inputs from the natural-language business rules they extracted.** In every case, extraction and test generation are parallel activities on the same source, not a loop.

The submission's reciprocal loop — a rule saying "3.5% below ₹10,000, 4% above" causing the generator to emit 9,999 / 10,000 / 10,001 — is a different mechanism, and it is the one that makes the extraction layer *earn its place* in the verification pipeline rather than being a side deliverable.

c) No mainframe, no licence, no captured production traffic

This is the practical differentiator and it should not be undersold.

- **AWS Blu Age Compare** needs reference datasets *from the mainframe or iSeries*.
- **AWS Application Testing** works by capturing and replaying mainframe traffic.
- **Mechanical Orchard** requires observing *real production data flows*.
- **IBM WCA4Z** requires IBM Z and the associated licensing.

All four require the institution to already have the legacy system running, instrumented, and accessible — plus a commercial engagement. **Say "every shipping product", not "everyone"** — Locksmith runs off-mainframe on commodity hardware, so the absolute form of this claim is false (`07 §C4`). A cooperative bank, a regional rural bank, or a public-sector institution evaluating whether modernisation is even feasible cannot start there.

The proposed system needs only the source code and GnuCOBOL. It generates its own inputs rather than capturing them. That makes it usable at the *decision* stage — before a vendor is selected, before a budget is approved, before anyone has agreed to instrument production. For the Indian market described in `01 §3` — lenders spending under 5% of revenue on IT, facing a $1bn upgrade bill — a zero-licence assessment tool is a genuinely different product from an enterprise migration platform.

d) The output is a specification, not just a pass/fail

The vendor tools produce a migration verdict. The submission produces three retained artifacts: a traceable specification, a permanent regression suite, and an evidence-backed equivalence claim — of which the first two hold value *even if the migration is cancelled*. Nothing surveyed frames the deliverable that way, and for a bank that has been deferring the decision for a decade, "you get documentation and tests whether or not you proceed" removes the reason to defer.

---

4. Academic prior art — and the seam between the papers

Line of work	Papers	Covers
Rule extraction	COBREX, A-COBREX (ICSE 2025), COBRAIN (EASE 2025), Cosentino et al.	Getting rules out of COBOL. Best F1 ≈ 0.73.
Translation validation	*Automated Testing of COBOL to Java Transformation* (arXiv 2504.10548), SEDCoT (arXiv 2607.04092), *Quality Evaluation of COBOL to Java Code Transformation* (arXiv 2507.23356)	Checking a translation is equivalent.
Translation reliability	*Lost in Translation* (arXiv 2308.03109), *Beyond Translation Accuracy* (arXiv 2605.02195)	Measuring how often LLMs get it wrong.
Orchestration	*Deterministic vs. LLM-Controlled Orchestration for COBOL-to-Python Modernization* (arXiv 2605.09894)	How to structure the pipeline.

The seam: the first row and the second row do not cite each other's mechanisms. Extraction papers evaluate against human-annotated ground truth. Validation papers generate inputs from code structure. Nobody uses the extracted rules to drive input generation, and nobody reports validation results back against the extracted rules. That seam is the project.

---

5. Revised positioning — suggested language

For the next round, replacing the current closing line:

> ***The industry has converged on behavioural equivalence as the real deliverable — IBM, AWS and Mechanical Orchard all ship a version of it. Every one of them requires your mainframe, your production traffic, and a commercial engagement before it will tell you anything. We built the same guarantee on an open stack that needs nothing but the source file, and we report it per business rule: of the rules we extracted, these are proven by execution, and these we could not prove. A bank can run it before deciding whether to modernise at all.***

Three claims, all true, none contradicted by the landscape:

1. Same guarantee, no mainframe and no licence.
2. Rule-level verdict, not test-level.
3. Usable at the decision stage, not the migration stage.

04 — Feasibility and Build Plan

What can actually be built, with what, and in what order. Nothing here assumes a mainframe, a paid licence, or a bank's cooperation.

---

1. The COBOL side is solved

GnuCOBOL is the whole reason this project is feasible on a laptop.

- Free software: `cobc` under GPL, `libcob` runtime under LGPL.
- Supports **19 COBOL dialects**, including IBM, MVS, Micro Focus, ACUCOBOL-GT, RM/COBOL, BS2000, GCOS, COBOL85, COBOL2002, COBOL2014.
- Version 3.2 added `-std=ibm-strict` for strict IBM COBOL compatibility, with reserved words updated to **Enterprise COBOL 6.3**.
- Compiles COBOL to C, then to a native binary — so the "legacy" side of the harness is a normal executable that can be run, timed and instrumented.

Conformance evidence, which is worth one line in the deck: GnuCOBOL passes **9,700 of 9,748** tests in the NIST COBOL-85 test suite (CCVS85 — 512 test programs, 8,800+ individual test cases, public domain, mirrored on GitHub). That is a citable answer to "how do you know your COBOL side is behaving like a real mainframe compiler."

State the caveat yourself: the GnuCOBOL project explicitly **does not claim to be a "Standard Conforming" implementation** despite that pass rate. Volunteering this is better than being caught by it.

Honest limitation to state before a judge does: GnuCOBOL is not IBM Enterprise COBOL. Dialect-specific behaviour, especially around `COMP-3` edge cases, `ON SIZE ERROR` semantics and EBCDIC-vs-ASCII collation, can differ. The correct framing is that the *methodology* is compiler-agnostic — swap GnuCOBOL for Enterprise COBOL in a bank's own environment and the harness is unchanged. GnuCOBOL is the free stand-in that makes the demo possible, not a claim that the two compilers are identical.

---

2. Parsing and extraction

Need	Tool	Notes
COBOL AST + semantic graph	**ProLeap COBOL parser** (`uwol/proleap-cobol-parser`)	ANTLR4-based. Produces AST and an Abstract Semantic Graph with data and control flow information (variable access). Passes the NIST suite. Applied to numerous banking and insurance COBOL files. MIT licence. Java. Also maintained as a fork under `openrewrite/`.
Preprocessing	ProLeap preprocessor	Handles `COPY` and `REPLACE`. Extracts `EXEC SQL` / `EXEC CICS` as text — relevant because those are the constructs the project declares out of scope, so they can be *detected and rejected* rather than silently mishandled.
Rule extraction technique	Program slicing on business-concept variables	The method used by COBREX and the model-based frameworks. A slice is a set of source lines, which is exactly the traceability the submission promises.
Rule extraction, practical	LLM few-shot over the sliced unit	COBRAIN's approach; measured F1 0.73 vs COBREX's 0.59. Constrain the model to the slice, not the whole program.

Design decision worth defending: run the LLM over *slices*, not over the whole file. It bounds the context, it makes the line-level citation mechanical rather than model-generated (the model cannot hallucinate a line reference if the slice defines the line set), and it matches the deterministic-orchestration finding — LLM calls at bounded points inside a fixed pipeline.

---

3. Coverage instrumentation — how the "unproven" label gets computed

The verified/unproven verdict needs to know which source lines each execution touched.

- GnuCOBOL compiles via C, so **gcov** (the standard GCC coverage tool, `-fprofile-arcs -ftest-coverage`) is the available path: it produces exact execution counts per statement.
- Caveat found in the GnuCOBOL bug tracker: a coverage build error on Ubuntu 24.04 (bug #997). Budget time for toolchain friction here, and have a fallback.
- **Fallback that is arguably better for the demo anyway:** instrument at the COBOL level rather than the C level — inject a `DISPLAY` trace of paragraph entry, or map the generated C line numbers back to COBOL lines using `cobc -fgen-c-line-directives`. Paragraph-level granularity is sufficient, because extracted rules map to paragraphs and slices, not to individual C statements.
- `GCBLUnit` (`OlegKunitsyn/gcblunit`) exists as a GnuCOBOL unit-testing tool with JUnit-format reporting — useful as a reference for harness plumbing, not as a dependency.

---

4. Test corpus — do not write the COBOL yourselves

Writing your own COBOL and then proving your tool understands it is circular, and a judge will say so. Use public corpora:

Corpus	What it is	Use
`aws-samples/aws-mainframe-modernization-carddemo`	Comprehensive COBOL credit-card management application, Apache 2.0, deliberately written with varied coding styles to exercise analysis and migration tooling. Includes batch programs such as `CBACT04C.cbl` (interest calculation).	**Primary target.** It is AWS's own benchmark for exactly this class of tool, which makes "we ran it on AWS's mainframe modernisation sample" a strong line.
NIST CCVS85 (`newcob.val`)	512 programs, 8,800+ test cases, public domain	Parser robustness testing; conformance claim.
Own minimal programs	A 150–300 line savings-interest accrual program modelled on RBI's rules	Needed for the controlled demo where you *know* the seeded divergence. Present as a demo fixture, not as evidence.

Correction after verification — do not use `CBACT04C.cbl` as the verification demo. It is an interest calculator (652 lines), but it reads **four VSAM files** and calls `FUNCTION CURRENT-DATE` to timestamp records. Database coupling and a clock read are both excluded by this project's own scope rules.

Use it for the intake gate instead, which is a better demo anyway — the tool correctly refusing AWS's own published sample, with a reason, on third-party code. Then lift the interest arithmetic (`(balance × rate) / 1200`) into a pure computational unit for the verification demo and say plainly that this is what was done. Sweep the rest of CardDemo for a genuinely pure program to headline with. See `07 - Verification Pass.md §C2`.

---

5. The demo that should be built

A single, tight narrative beats a broad, shallow one.

1. **Input:** a COBOL savings-interest accrual program (CardDemo's, or a 150-line fixture modelled on RBI's rules).
2. **Extract:** produce ~10–20 plain-language rules, each with a line citation. Include the tier threshold, the rounding step and the day-count handling.
3. **Reimplement:** generate the Java. Do not hand-fix it. The point is that the generated code is *untrusted*.
4. **Generate inputs:** boundaries derived from the extracted rules — ₹99,999 / ₹1,00,000 / ₹1,00,001 (the RBI uniform-rate threshold), accruals landing on exact ₹0.50 (the nearest-rupee rounding rule), 28/29 February, month and quarter ends, zero balance, negative balance, maximum `PIC` value.
5. **Run both. Diff.**
6. **Show the finding:** *"Input: balance ₹1,00,000.00, 12 days at 3.5%. Legacy returns ₹115. Reimplementation returns ₹116. Divergence traced to rule R-07 (line 214, `COMPUTE ... ROUNDED`) — the Java uses `RoundingMode.HALF_UP` where the COBOL truncates."*
7. **Show the coverage table:** 20 rules extracted, 14 verified by execution, 6 unproven — and name which six.

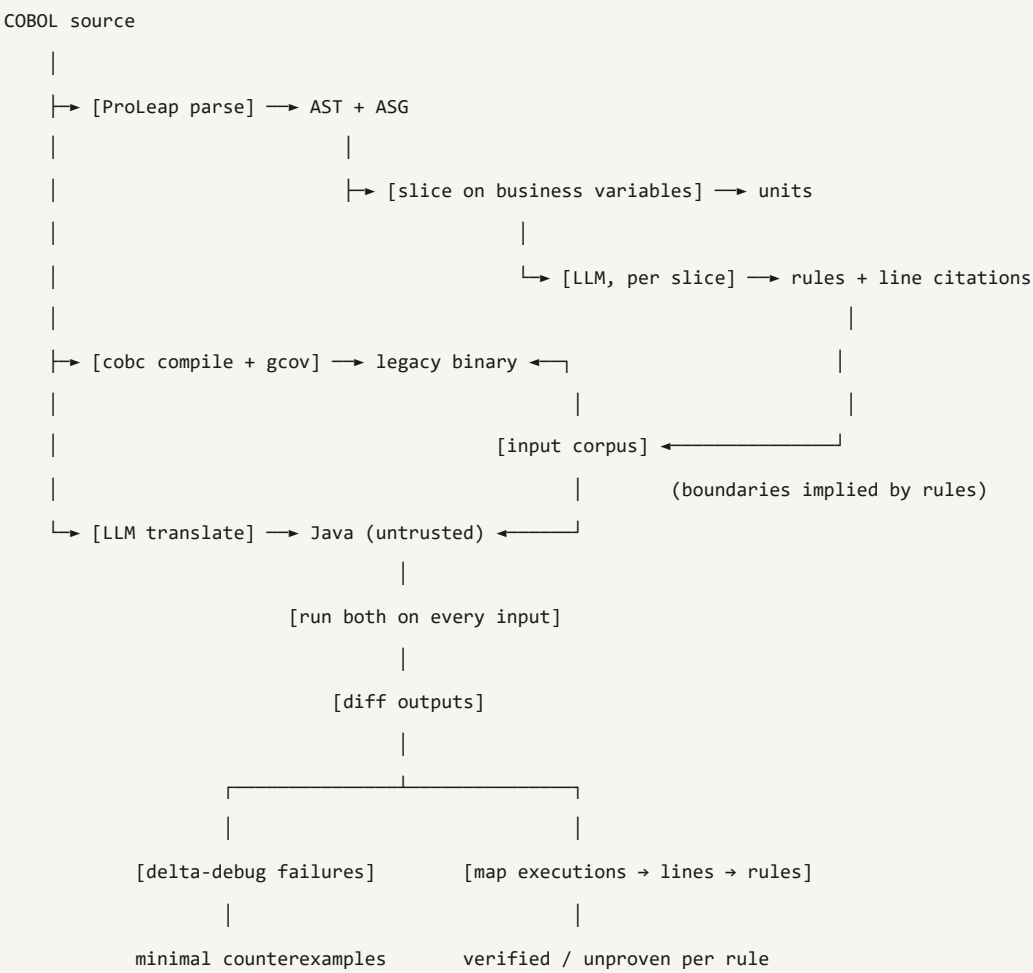
Step 7 is the differentiator. Step 6 is the moment that convinces the room.

Delta debugging (SEDCoT's third phase) is what turns step 6 from "here are 73 failing inputs" into one minimal counterexample. It is a shrinking loop — maybe 40 lines of code — and it improves the demo more than any other equivalent effort.

---

6. Architecture, per the evidence

Deterministic pipeline, bounded LLM calls. Justified by the controlled study showing deterministic orchestration matches agentic accuracy at **up to 3.5× lower token cost** with better worst-case robustness and lower run-to-run variance.



Determinism also matters for a second reason specific to this project: the harness compares two programs for exact equality. A non-deterministic harness undermines its own verdict.

7. Staged plan

Stage	Deliverable	Risk
0	GnuCOBOL installed; a COBOL program compiles and runs from a script; output captured and parsed	Low. Do this first — it de-risks everything else.
1	Differential harness: run two programs on a list of inputs, diff, report	Low. This alone is a working demo.
2	Input generation from `PIC` clauses (domain, sign, extremes) + hand-written boundaries	Low
3	ProLeap parse → slices → LLM rules with line citations	Medium. ProLeap is Java; if the rest of the stack is Python, budget for the boundary or use a simpler regex/heuristic sectioniser for the demo.
4	Rules → boundary inputs (the reciprocal loop)	Medium. **This is the contribution — protect time for it.**
5	Coverage → per-rule verified/unproven verdict	Medium-high. gcov toolchain friction is the known risk; the paragraph-level `DISPLAY` fallback is the mitigation.
6	Delta debugging to minimal counterexample	Low. High demo value per hour.
7	Report generation: specification document + regression suite + equivalence claim	Low. Presentation work, but it is the actual product.

If time runs out: stages 0–2 and 6 produce a demo that works and tells the story. Stages 4 and 5 are what make it *this project* rather than a generic diff tool. Cut stage 3's sophistication before cutting stages 4 and 5.

---

8. Scope boundaries, restated with the technical reason

The submission already scopes these out. The reasons, for when a judge asks:

Excluded	Why, precisely
CICS screens, JCL orchestration	Not pure computation; behaviour depends on terminal state and job scheduling. ProLeap extracts `EXEC CICS` as opaque text, so these are *detectable* — the tool can refuse them explicitly rather than mishandle them.
Database-coupled programs	The oracle requires reproducibility. DB2 state is external and mutable, so the same input does not guarantee the same output.
Non-deterministic logic (clocks, randomness, external state)	Differential testing compares outputs for equality. A program reading `CURRENT-DATE` produces a different answer on each run, so equality is meaningless. Detectable statically by scanning for the relevant intrinsics — again, refuse explicitly.
Whole-system migration	Out of scope by design; this is an assessment and assurance tool, not a migration platform.

Making each exclusion *detectable and explicitly refused* rather than merely documented is worth building. "The tool tells you when it cannot help you" is the same honesty argument as verified/unproven, applied at the intake stage.

05 — Risks and Judge Questions

Cognizant is an organiser. Assume at least one person in the room has worked on mainframe modernisation and knows the vendor landscape. These are the questions that follow from that, with answers grounded in `01` – `04`.

The eight questions that will actually be asked

1. "IBM watsonx Code Assistant for Z already validates COBOL against Java. AWS ships Blu Age Compare. What's new here?"

The one to prepare hardest for. See `03`.

> *Correct — and we'd argue that's the strongest evidence our diagnosis is right, because IBM, AWS and Mechanical Orchard all independently converged on behavioural equivalence as the real deliverable. Three things differ. First, every one of those tools needs your mainframe: Blu Age Compare needs reference datasets from the mainframe, AWS Application Testing works by capturing and replaying mainframe traffic, Mechanical Orchard observes live production data flows, WCA4Z needs IBM Z. We need the source file and nothing else, because we generate our inputs instead of capturing them. Second, they report equivalence per test case; we report it per business rule — of 47 rules extracted, 31 proven, 16 unproven, named. Third, that means we're usable at the decision stage, before a bank has selected a vendor or approved a budget. That's the stage most Indian institutions are actually stuck at.*

Do not claim the mechanism is novel. Claim the deployment point and the unit of reporting are.

2. "Your extraction layer is an LLM. How do you know the rules are right?"

> *We don't, and that's the design. The best published COBOL rule extractor gets about 62% precision and 74% recall; the best LLM approach reports F1 around 0.73. We assume roughly a quarter of our rules are wrong. That's exactly why extraction isn't the deliverable. A rule only gets marked verified when execution confirms it — the differential run is what catches the extraction layer lying. And every rule carries the line numbers it came from, so a reviewer confirms or rejects it by reading three lines instead of four thousand.*

3. "GnuCOBOL isn't IBM Enterprise COBOL. Doesn't that invalidate the whole thing?"

> *For the demo, GnuCOBOL is the free stand-in that lets us run this on a laptop — it supports 19 dialects including an IBM-strict mode aligned to Enterprise COBOL 6.3, and it passes 9,700 of 9,748 NIST COBOL-85 conformance tests. But the methodology is compiler-agnostic. In a bank, you point the harness at the bank's own compiler and the architecture is unchanged, because the harness only needs to compile, execute and capture output. The compiler is a dependency, not an assumption.*

4. "Interest accrual is a toy problem. What about a real core banking system?"

> *Deliberately narrow, for a specific reason: differential testing needs determinism. Same input, same output, every run. Programs that read the system clock, hit DB2, or depend on CICS terminal state break that guarantee, so we exclude them — and the tool detects and refuses them at intake rather than silently producing an unsound verdict. Pure-computation programs are where the method is sound, not where it's easy. Interest accrual is also where the money is: RBI prescribes daily product basis and rounding to the nearest rupee for rupee deposits, so a rounding drift there is a Master Direction breach across the entire deposit book, not a bug report.*

5. "How do you generate inputs good enough to find a real bug? Random inputs won't."

> *Agreed, and we say so in the write-up — input quality caps everything. We don't generate randomly. The extracted rules tell us where the boundaries are. If a rule says a uniform rate applies up to one lakh and a differential rate above it, the generator emits 99,999, 1,00,000 and 1,00,001. If a rule involves rounding, we generate accruals that land on exact midpoints. If it involves dates, we generate 28 and 29 February, month ends, quarter ends. The* `PIC` *clauses give us the domain, sign and extremes for free. And when we find a divergence, we shrink it by delta debugging to a minimal counterexample, so the engineer gets one failing case, not four hundred.*
>
> *The scaling path is symbolic execution — that's what IBM's own testing framework for WCA4Z uses to generate unit tests from COBOL. We're not attempting that at this scale.*

6. "What if the two programs agree on everything? Have you proved anything?"

> *We've proved equivalence over the inputs we ran, and we say exactly that — no more. That's why the coverage table exists. If 16 rules were never exercised, we report 16 unproven rules by name. Practitioners doing dual-run migrations in banks make this point themselves: running two systems in parallel without rigorous reconciliation creates false confidence. Reporting what we didn't prove is what separates evidence from reassurance.*

7. "Who buys this, and why wouldn't they just call Cognizant?"

> *Many of them should, and this doesn't compete with a migration engagement — it precedes one. A regional or cooperative bank deciding whether modernisation is feasible can't start with an enterprise platform engagement. BCG puts the Indian core-banking upgrade bill at about a billion dollars, and Indian banks spend up to 5% of revenue on IT against 7–9% globally. The first artifact those institutions need is a specification and a regression suite for a system they currently can't touch — and they keep both even if they never migrate. That output makes a modernisation programme scopeable, which makes it sellable. It's an on-ramp, not a substitute.*

8. "What's the hardest part, and what have you not solved?"

Answer this one straight; hedging costs more than the admission.

> *Three things. Input generation quality caps everything — we can only verify rules we reach. Coverage instrumentation through GnuCOBOL's C backend is fiddly, and there's an open bug on recent Ubuntu; our fallback is paragraph-level tracing, which is coarser. And the extraction layer is genuinely unreliable — a quarter of its output is probably wrong, which is why it's a suggestion engine for the verifier rather than a deliverable. The thing we have not solved is proving a rule holds for all inputs. We prove it for the inputs we ran. That's differential testing, not formal verification, and we're not going to call it verification of the stronger kind.*

Risks to the project, ranked

Risk	Severity	Mitigation
Judge knows the vendor landscape and reads the pitch as unaware of it	**High**	Lead with the landscape (`03 §5`). Naming IBM and AWS yourselves converts the objection into evidence of research depth.
Nothing is built; the deck promises a system that doesn't exist	High	Stages 0–2 of `04 §7` produce a working differential harness quickly. A running diff beats a perfect diagram.
gcov / GnuCOBOL coverage toolchain friction	Medium	Paragraph-level `DISPLAY` tracing fallback; `cobc -fgen-c-line-directives` for C→COBOL line mapping.
ProLeap is Java, rest of stack likely Python	Medium	Run ProLeap as a subprocess emitting JSON, or use a heuristic sectioniser (division/section/paragraph boundaries) for the demo. Don't let the parser become the project.
Demo COBOL written by the team → circular reasoning	Medium	Use `aws-samples/aws-mainframe-modernization-carddemo` as the primary target. Third-party code the team didn't write.
Contested statistics get challenged	Low-medium	`01` flags every soft figure. Use the defensible ones: 92 of top 100 banks; £48.65m TSB fine; BCG's $1bn; A-COBREX's 62%/74%.
Overclaiming "verification" in the formal-methods sense	Low-medium	Consistently say "evidence of equivalence over the executed input set." Never "proof of correctness." A judge with a formal methods background will pounce on the latter.

Three things to stop saying

1. **"Everyone else's submission assumes their AI is right."** True of the room, false of the market. Replace with the `03 §5` framing.
2. **"95% of ATM swipes run on COBOL."** Traces to a single press item. Use "92 of the world's top 100 banks run on mainframes" instead — same point, defensible.
3. **Anything phrased as "we invented" or "nobody does this."** Three vendors do. The originality is the rule-level verdict and the zero-dependency deployment, and those claims survive scrutiny.

Three things to start saying

1. **"The industry converged on this independently — IBM Research's own paper on their translation product says the output cannot be trusted."** Quoting IBM against IBM's product is the strongest single line available.
2. **"RBI prescribes rounding to the nearest rupee on a daily product basis."** Concrete, Indian, regulatory, and it makes the rounding-bug demo land.
3. **"Of 47 rules extracted, 31 proven, 16 unproven — here are the 16."** The whole thesis in one sentence, and nothing surveyed does it.

Solution Folder

Design options for the Legacy Banking Modernisation Platform, and the specific novelty claims worth defending.

Written against the evidence in `../01` – `../06`. Every "this is novel" statement here has been checked against the vendor landscape in `../03 - Competitive Landscape.md` and the academic prior art in `../02 - Solution Research.md`. Where the check was inconclusive, it says so.

Files

File	Contents
`01 - Solution Options.md`	Nine candidate architectures, from "documentation generator" to "formal equivalence proof". What each is, what it costs to build, what prior art it collides with, and why eight of them are wrong for this team.
`02 - Novelty Proposals.md`	Ten specific novelty claims, ranked by strength. Each with: the claim, the prior art it must survive, how to build it, how to demo it, and how it fails.
`03 - Recommended Architecture.md`	The composite design: components, data model, pipeline, and what each stage outputs.
`04 - Pitch Language.md`	The novelty compressed to slide-length and question-length.

The recommendation in one page

Build the differential harness as the spine. Add three things on top of it. Those three things are the entire novelty claim.

The spine — compile the COBOL with GnuCOBOL, run both implementations on generated inputs, diff — is prior art (IBM, AWS, Mechanical Orchard). It is still the right foundation, because everything else depends on it and it is the part that actually works. Do not pretend it is new.

The three additions, in descending order of strength:

1. Three-way conformance: legacy vs rewrite vs regulation

Every existing tool treats the legacy program as ground truth *by definition*. That is the one assumption in the entire category that nobody has questioned — and it is wrong, because a forty-year-old accrual engine can be non-compliant with a Master Direction issued in 2025.

Add a third reference: a small executable model derived from RBI's stated rules (daily product basis, uniform rate to ₹1 lakh, rounding to nearest rupee for rupee deposits). Now a divergence has four possible readings instead of one, and the most valuable one — *legacy and rewrite agree with each other and both breach the regulation* — is invisible to every two-way tool on the market.

This is the strongest idea in this folder. See `02 §N1`.

2. Three-valued rule verdicts, with a real definition of "proven"

The submitted write-up promises *verified* and *unproven*. Add **refuted**, and define *verified* properly.

A rule is not proven because the lines it came from were executed. It is proven when inputs on both sides of its boundary produced *different* outputs — showing the rule actually governs behaviour — and both implementations agreed on all of them. A rule whose boundary makes no difference to the output is not verified; it is evidence that the extraction layer hallucinated. That rule gets marked **refuted**, and refuted rules are the most valuable output the system produces, because they are the ones an engineer would otherwise have built on.

Verified and narrowed: rule-level reporting itself is *not* original — AgentModernize's Business Rule Preservation Score does it, against human gold-standard rules. What survived falsification is the **discrimination condition** and the **refuted** verdict on rules the system extracted itself. See `02 §N2`, `§N3` and `../07 - Verification Pass.md §C3`.

3. Mutation-certified corpus adequacy

The obvious attack on the whole project: *"your inputs all passed — so what?"*

Answer it by mutating the **legacy** program — move a threshold by one, flip `>` to `>=`, remove a `ROUNDED` — and checking that the input corpus detects each mutant. A corpus that cannot detect a deliberate one-rupee change to the threshold does not prove anything about that threshold. This converts the equivalence claim from a binary pass into a measured one: *"this corpus kills 94% of injected mutations in the accrual logic; the 6% it misses are in these paragraphs."*

Mutation testing is a textbook technique. Using mutants of the **oracle** to certify the adequacy of a **migration equivalence claim** is the novel combination, and it is the answer to the hardest question in `../05`. See `02 §N4`.

If only one novel thing can be built

Build #2 — the three-valued rule verdict. It is the cheapest of the three, it is the direct extension of what the submission already promises, it produces the single best slide (*"of 47 rules, 31 proven, 11 unproven, 5 refuted — here are the 5"*), and it degrades gracefully: even a partial implementation produces a meaningful table.

Build #1 second — it needs one hand-written regulation model, roughly 60 lines, and it transforms the pitch from a testing tool into a compliance instrument, which is what a banking judge is listening for.

Build #3 third — it is the most technically impressive and the least necessary. It wins the Q&A, not the pitch.

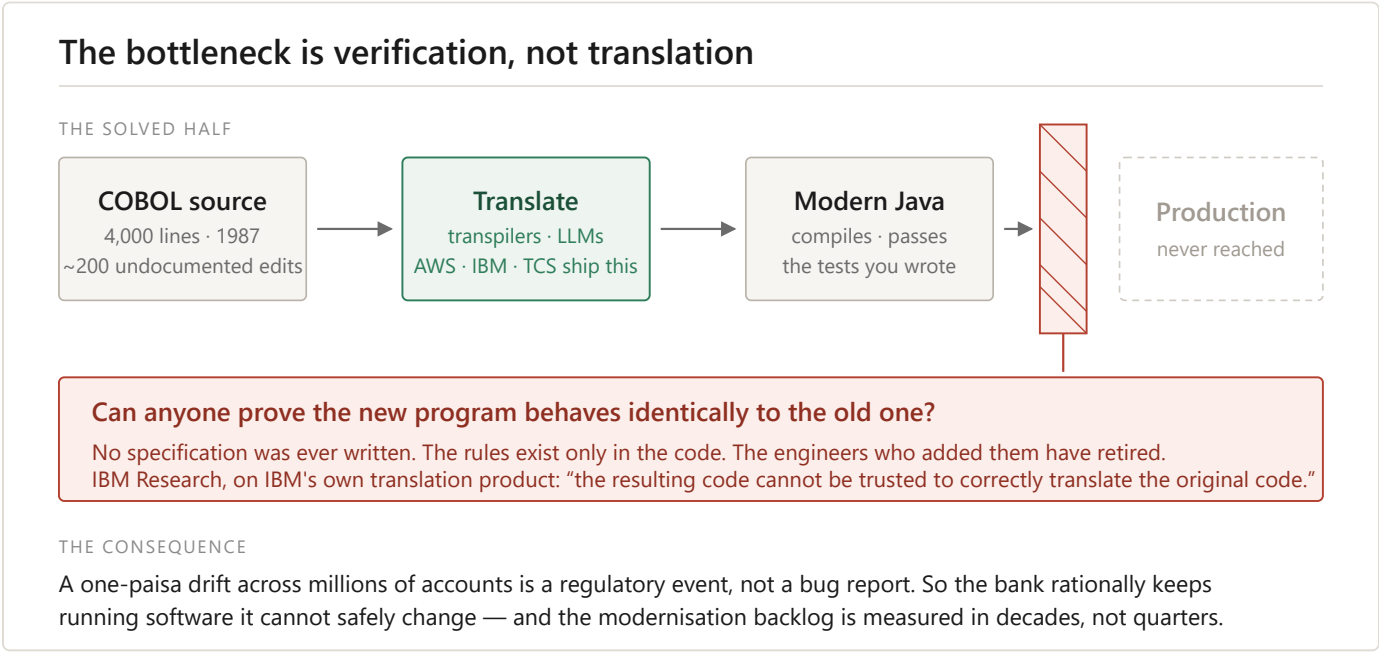
Visuals

Ten diagrams explaining the problem, the system and the novelty. Plain SVG — no dependencies, scales to any size, opens in any browser, and drops straight into the deck. Diagrams 8–10 were added for round 2 and cover the CII business-plan material.

British spelling and the same palette throughout: green = proven, amber = unproven, red = refuted or defective, blue = the regulation, purple = the system's own reasoning.

---

1 · The verification bottleneck



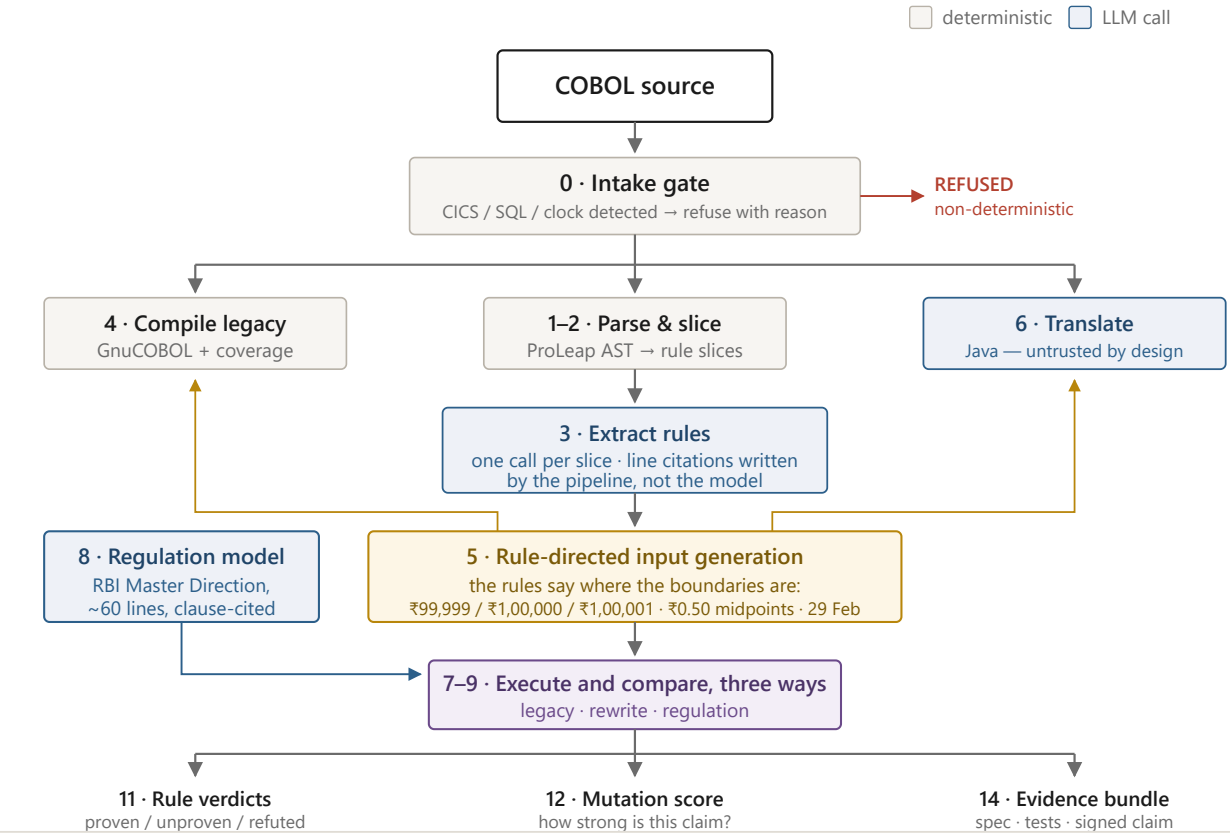
The problem in one frame. Translation is the solved half — transpilers and LLMs do it, and AWS, IBM and TCS all ship it. The barrier sits between the generated code and production: nobody can prove the new program behaves like the old one, because no specification was ever written. Use this as the opening slide.

---

2 · System architecture

System architecture

A deterministic pipeline with bounded LLM calls. Shaded stages carry the novelty.

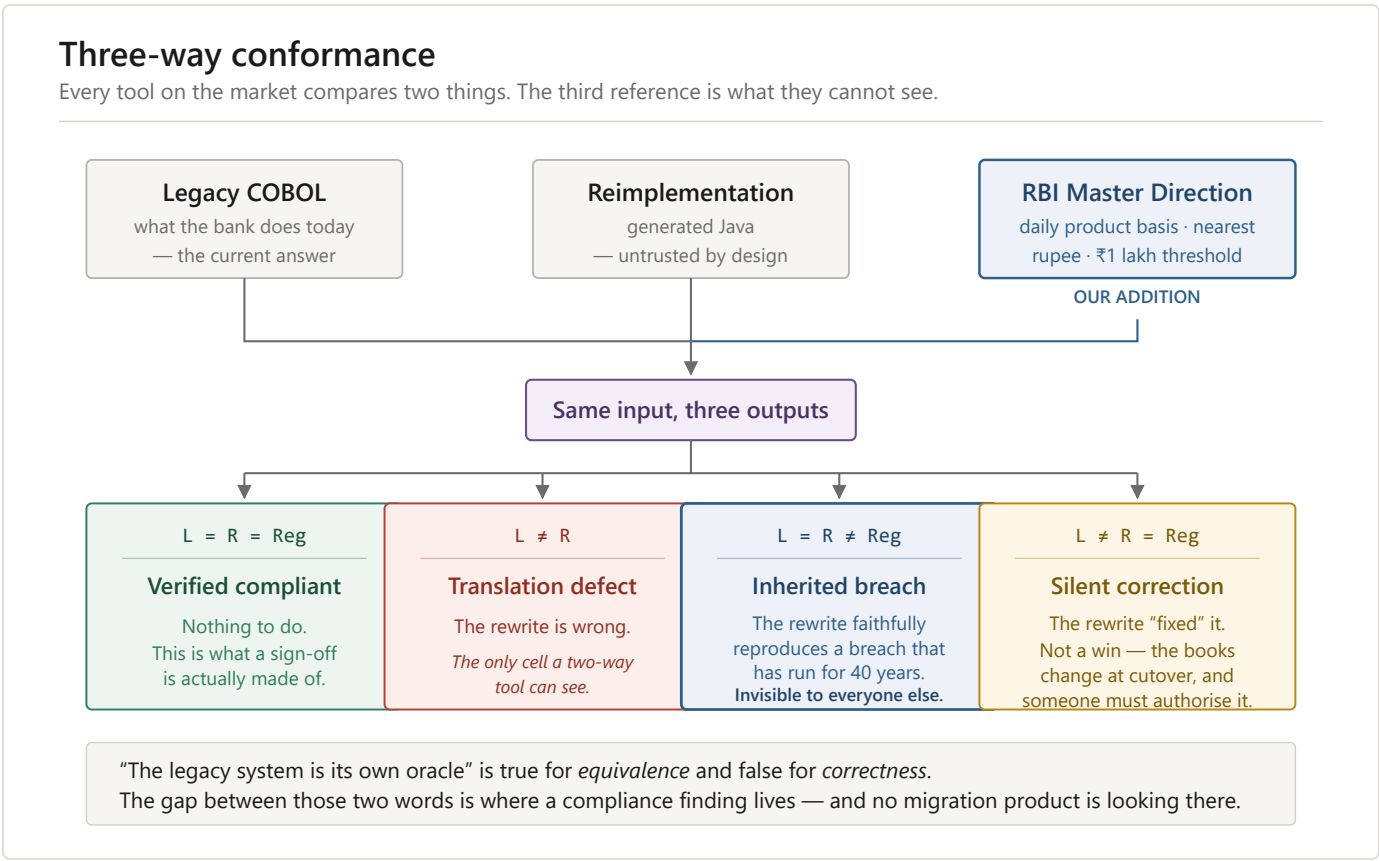


The full pipeline. Deterministic by design, with LLM calls confined to two bounded points — justified by the controlled study showing deterministic orchestration matches agentic accuracy at up to 3.5× lower token cost.

The gold arrows are the reciprocal loop: extracted rules determine which inputs get generated, and those inputs drive both implementations. Stage 0 refuses programs the method cannot handle soundly rather than producing a confident wrong answer.

---

3 · Three-way conformance

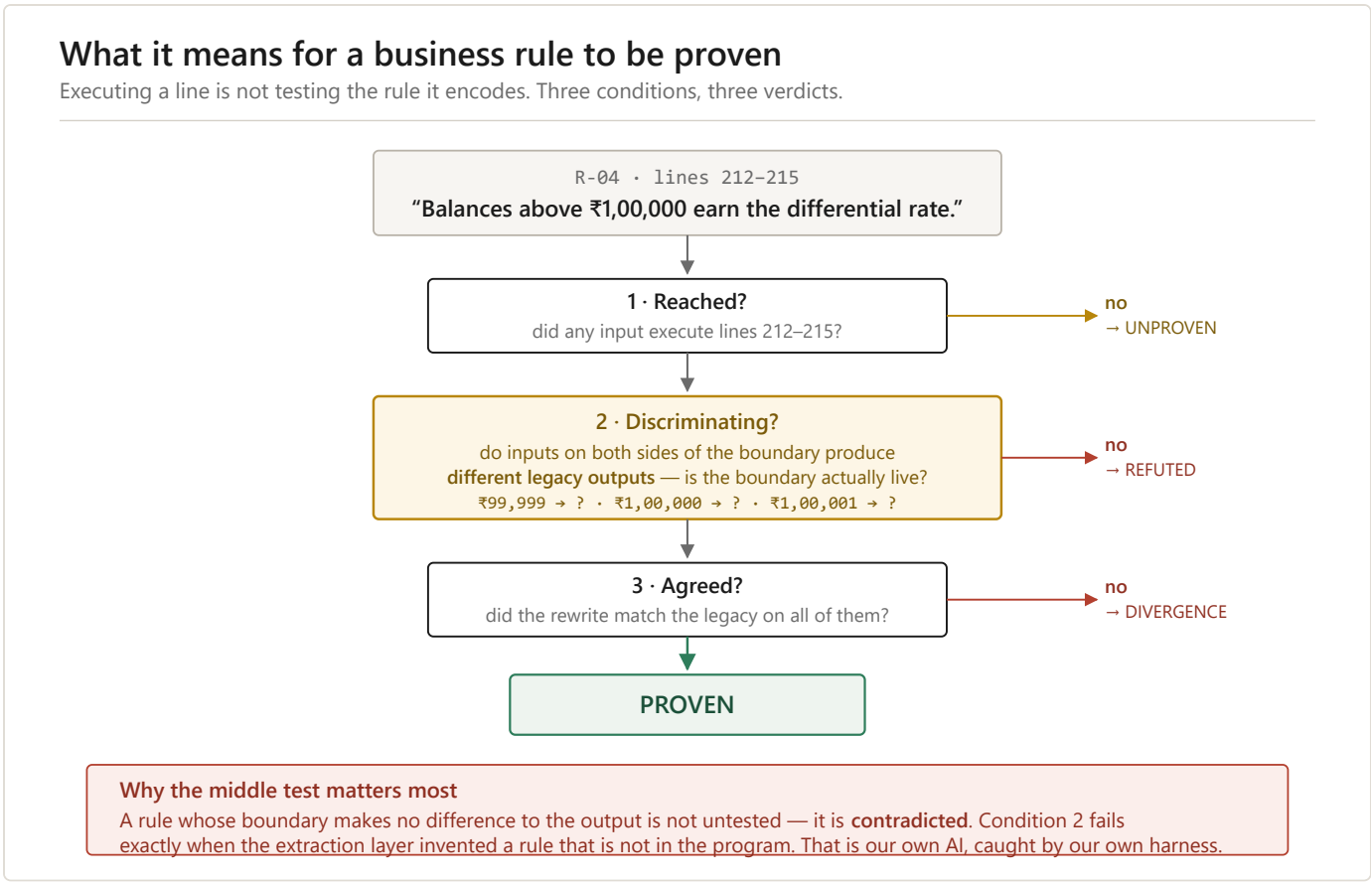


The differentiator. Every tool on the market compares two things and treats the legacy program as correct by definition. Adding the regulator as a third reference produces four outcomes instead of one — and the valuable one, where the legacy and the rewrite agree with each other while both breach the Master Direction, is structurally invisible to a two-way tool.

The line to deliver with this slide: *"the legacy system is its own oracle" is true for equivalence and false for correctness.*

---

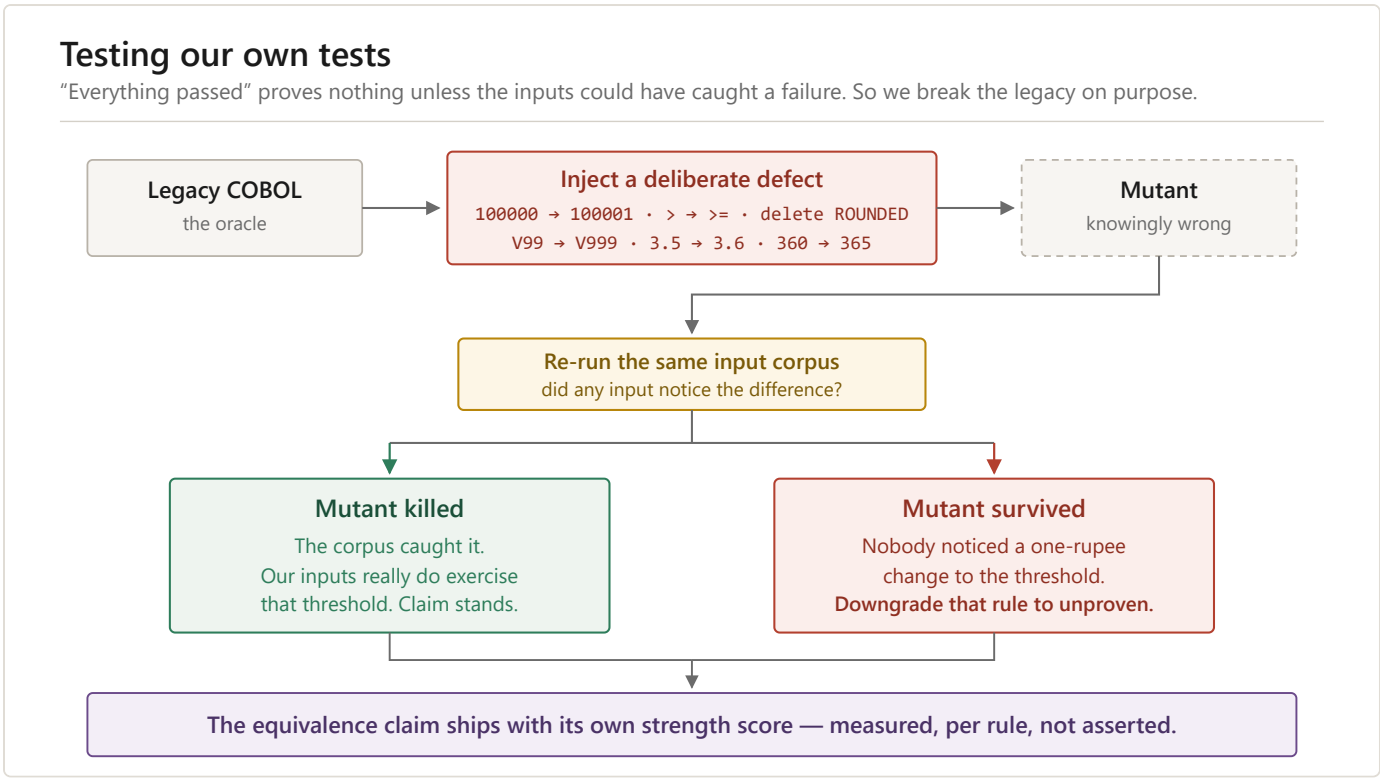
4 · What it means for a rule to be proven



The core technical contribution. Executing a line is not testing the rule it encodes, so *proven* requires three conditions, not one. The middle condition — did inputs on both sides of the boundary actually produce different outputs? — is what catches the extraction layer inventing rules that are not in the program. When it fails, the verdict is **refuted**, not *unproven*.

---

5 · Testing our own tests



The answer to *"everything passed, so what?"* Break the legacy program on purpose — move a threshold by one rupee, delete a `ROUNDED` — and check whether the input corpus notices. If it does not, the equivalence claim over that rule is worth nothing regardless of how many tests passed, and the rule gets downgraded.

This is the Q&A slide. It rarely gets shown; it always gets asked about.

---

6 · Where this sits

Where this sits

The industry agrees verification is the bottleneck. Three vendors ship a version of it. Here is what is left open.

	IBM WCA for Z	AWS Blu Age / Transform	Mech. Orchard Imogen	This project GnuCOBOL, open stack
Proves behavioural equivalence	✓	✓	✓	✓
Runs without a mainframe	✗	✗	✗	✓
Runs without captured production traffic	✓	✗	✗	✓
Usable before a migration is funded	✗	✗	✗	✓
Verdict reported per business rule	per test	per test	per test	✓
Flags rules its own AI got wrong	✗	✗	✗	✓
Measures the strength of its own claim	✗	✗	✗	✓
Checks the legacy against the regulator	✗	✗	✗	✓

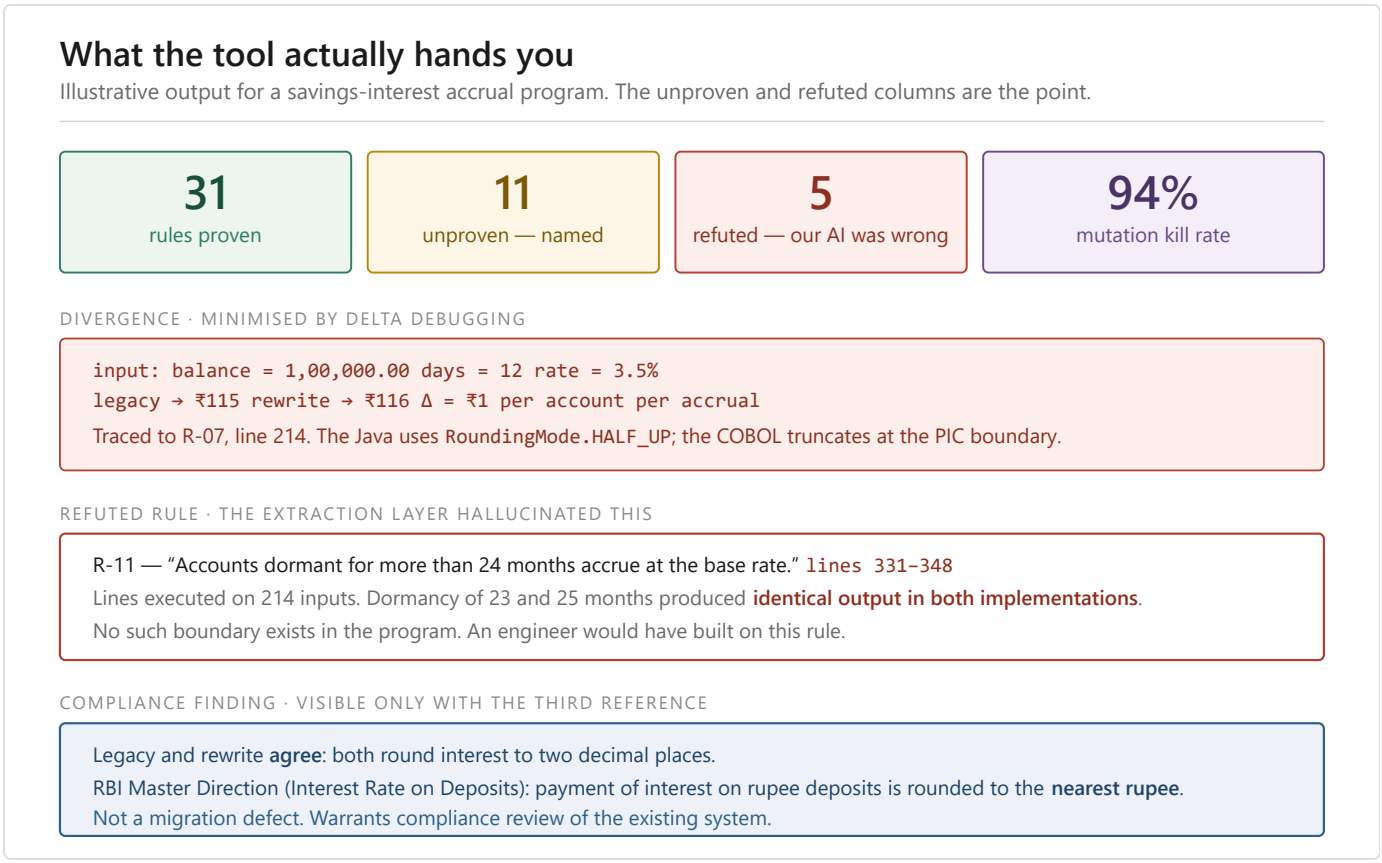
Every product above treats the legacy program as correct by definition. That assumption is the opening.

The honest competitive picture. The first row is deliberately a row of ticks for everyone — claiming to have invented differential verification would not survive a judge who works on mainframe modernisation. The claim is the rows below it.

Keep this in the appendix and bring it out when the "IBM already does this" question arrives.

---

7 · What the tool hands you

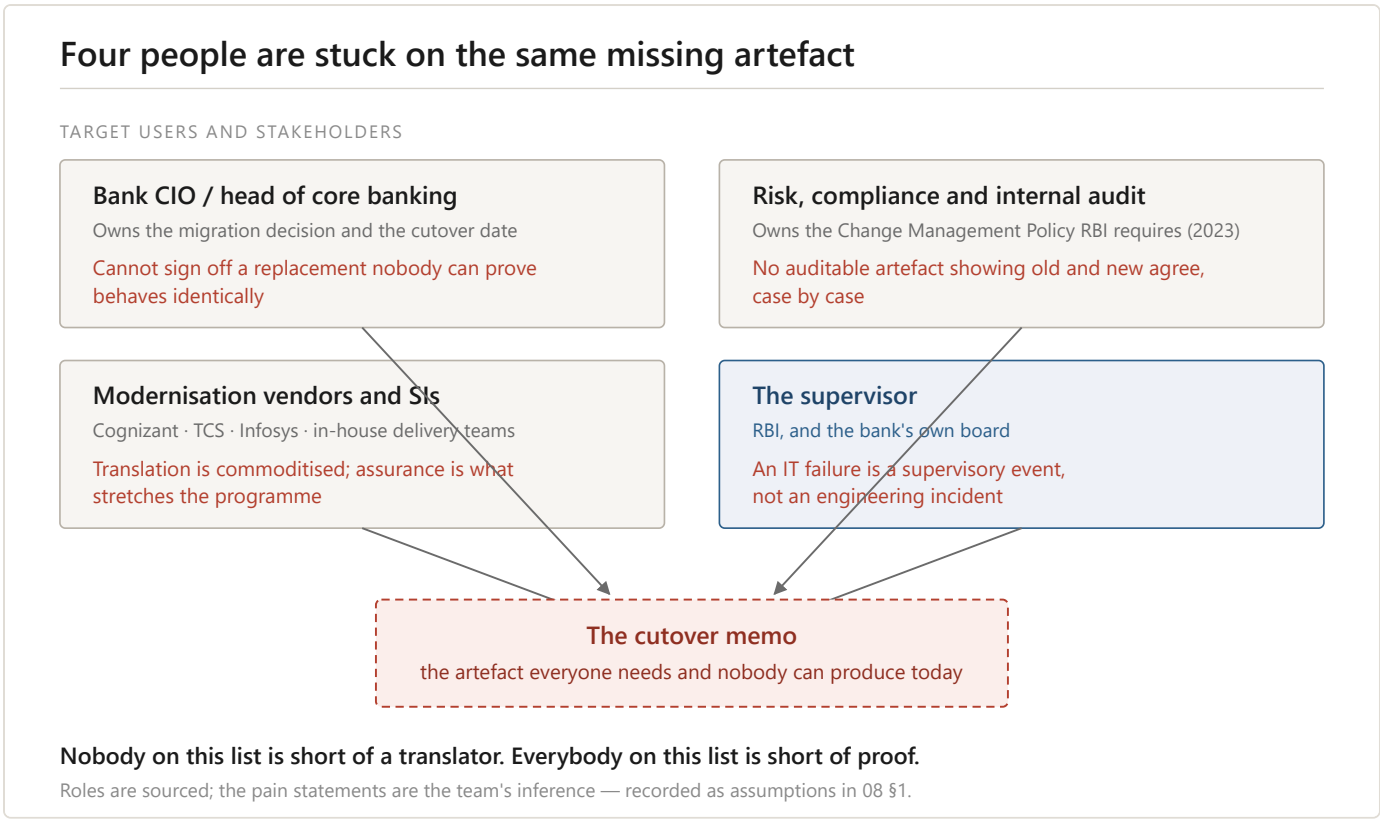


Illustrative output, and the shape of the demo. Four numbers, one divergence minimised to a single line, one refuted rule, one compliance finding.

The refuted rule is the slide that proves the thesis: an AI system catching its own AI lying, with a reproducible counterexample. The compliance finding is the slide a banker starts caring about.

---

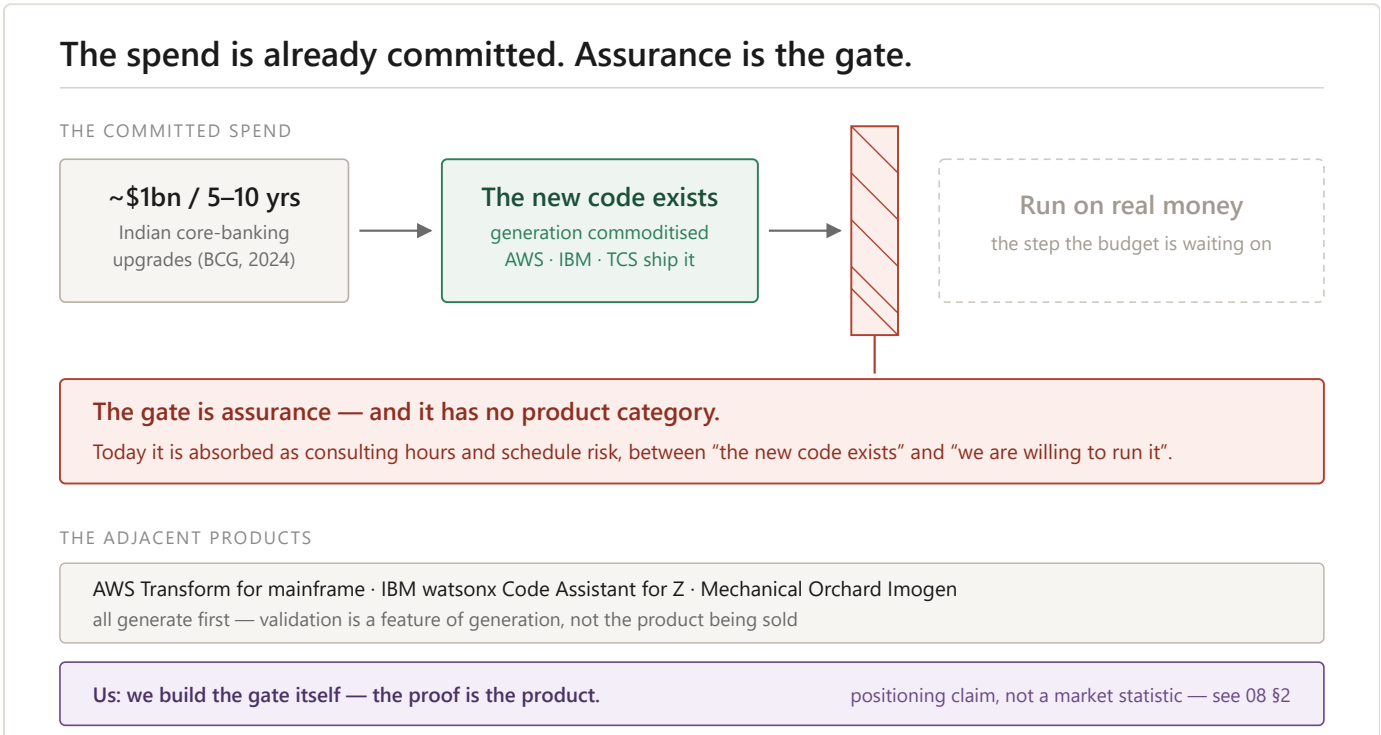
8 · Who is stuck — the stakeholder map



Round-2 addition. Four stakeholders converge on the one artefact none of them can produce: the cutover memo. The supervisor's card is blue because the regulator is the reference, not a user. Pain statements are the team's inference, recorded as assumptions in `08 §1` — the caption says so on the diagram itself.

---

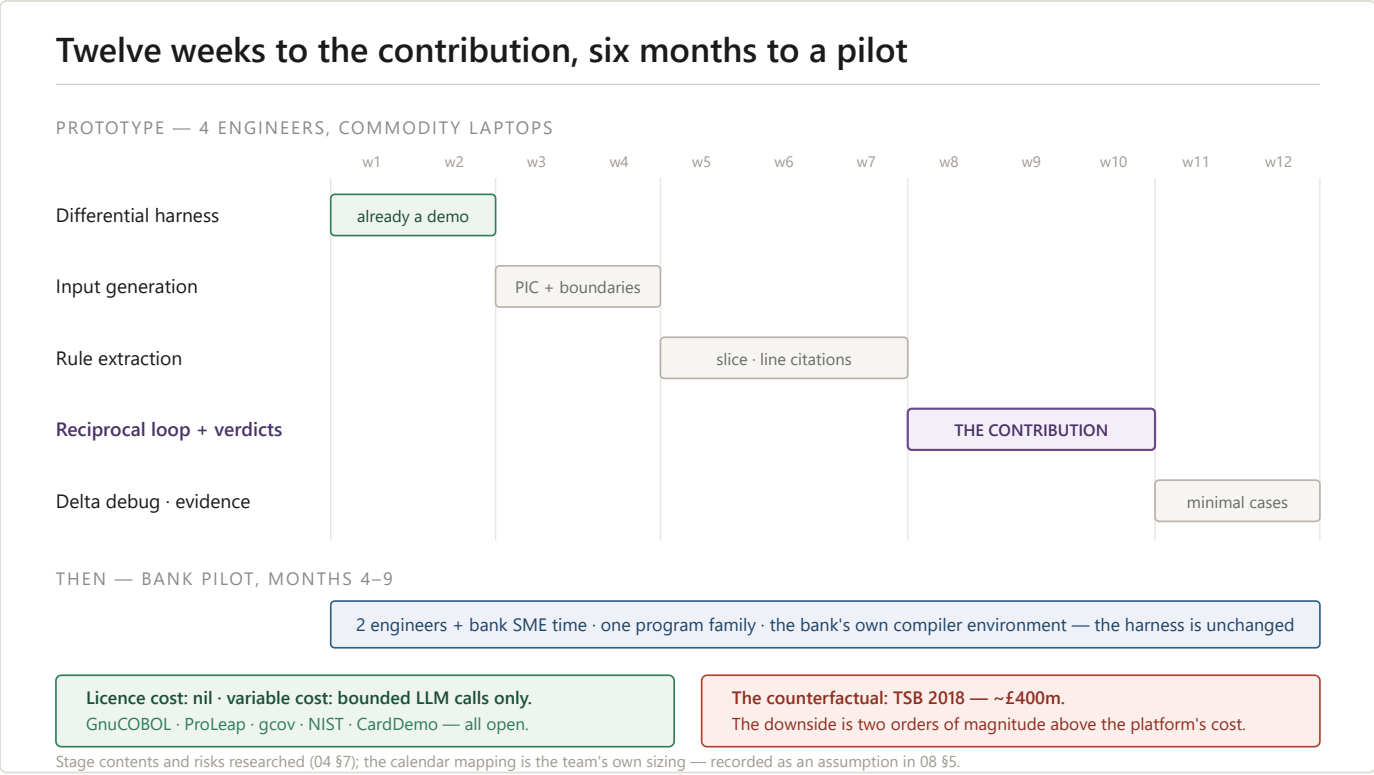
9 · The assurance gate



Round-2 addition; the market-potential argument in one frame, deliberately echoing diagram 1's barrier motif. The committed spend flows through generation (solved, green) and stops at the assurance gate (red). The bottom strips carry the competitive claim honestly: the adjacent products all generate first, and "the proof is the product" is labelled a positioning claim, not a market statistic.

---

10 · Build timeline and the business case



Round-2 addition. The twelve-week prototype plan as a bar schedule — weeks 8–10 in purple because the reciprocal loop and per-rule verdicts are the contribution — then the bank pilot in blue (months 4–9, the bank's own compiler environment). The two footer cards compress investments (licence cost nil) and the counterfactual (TSB, ~£400m). The calendar mapping is team sizing, flagged as an assumption on the diagram.

Using these

- **Editing:** plain text. Change a label by editing the `<text>` element.
- **In the deck:** PowerPoint imports SVG directly (Insert → Pictures). Right- click → Convert to Shape if individual elements need recolouring.
- **In the PDF:** `build-pdf.py` in the parent folder inlines all ten.
- **Sizing:** each has a `viewBox`, so scaling is lossless at any size.

01 — Solution Options

The design space, honestly assessed. Nine candidate architectures for the same problem. Eight are wrong for this team; the reasons are the interesting part.

Scoring: **Build cost** for a four-person student team with a few weeks. **Prior art** — how crowded. **Demo strength** — how well it plays in a room. **Soundness** — whether the claim it makes is actually true.

Option A — Documentation generator

Point an LLM at the COBOL, emit a business-rules document.

Build cost	Very low
Prior art	Extremely crowded — Cognizant, TCS MasterCraft, AWS Transform, IBM ADDI, plus COBREX / A-COBREX / COBRAIN in the literature
Demo strength	Superficially high, and that is the trap
Soundness	**Poor**

Reject. This is the option the submitted write-up already argues against, and the argument is correct. Best published extractors reach ~62% precision / 74% recall (A-COBREX) and F1 ≈ 0.73 (COBRAIN). Measured hallucination rates in code summarisation run as high as 66%. A document that is a quarter wrong, with no marking of which quarter, is a liability generator: an engineer builds on it.

Worth keeping in the deck only as the *strawman* — "this is the obvious approach, and here is the measured reason it fails."

Option B — Pure differential harness

No extraction. Generate inputs from the `PIC` *clauses, run both programs, diff.*

Build cost	**Low** — this is stages 0–2 of `../04 §7`
Prior art	Crowded (Diffy, Blu Age Compare, WCA4Z Validation Assistant) but the mechanism is sound
Demo strength	Medium — it works, but "we ran a diff" is not a pitch
Soundness	**High** — it claims exactly what it proves

Adopt as the spine, do not claim as the contribution. Everything else in this folder is built on top of it. It is also the safety net: if stages 3–5 collapse, this alone still demonstrates a working system.

Its limitation is precisely why the other options exist: a passing diff over inputs nobody chose carefully means very little, and it says nothing about *which business rules* were exercised.

Option C — Extraction plus differential testing, decoupled

Run both A and B. Publish the rules. Publish the diff results. Separately.

Build cost	Medium
Prior art	**This is exactly what the market ships.** IBM WCA4Z: symbolic execution generates tests from the COBOL, separately from any rule extraction. AWS: Transform extracts rules, Application Testing replays captured traffic. Mechanical Orchard: behavioural capture from production flows, rules from AWS Transform
Demo strength	Medium
Soundness	High

Reject as the headline. Two good halves that never touch each other. The extraction output is unverified prose sitting next to a verification result that has no idea the prose exists. It is the current state of the art, and it is where the open seam is (`../03 §4`).

Option D — Reciprocal loop with rule-level verdicts

Extracted rules determine which inputs get generated; execution results determine each rule's verdict.

Build cost	Medium — this is stages 3–5
Prior art	**Open.** No surveyed tool derives test inputs from extracted natural-language rules; none reports coverage per business rule
Demo strength	**High** — produces the coverage table
Soundness	High, if "proven" is defined properly (see `02 §N2`)

Adopt. This is the core. It is the submitted write-up's own "reciprocal loop" made concrete, and it is what makes the extraction layer earn its place in the pipeline rather than being a side deliverable.

Option E — Three-way conformance against the regulation

Add a regulation-derived executable model as a third reference alongside legacy and rewrite.

Build cost	**Low** — one hand-written model of RBI's stated rules, ~60 lines
Prior art	**Open, and structurally so.** Every tool in the category treats the legacy program as ground truth by definition, so none of them can express "the legacy is wrong"
Demo strength	**Very high** for a banking panel
Soundness	High, provided the model's scope is stated narrowly

Adopt as the differentiator. Full development in `02 §N1`. The key insight is that "the legacy system is its own oracle" — the submission's central idea — is true for *equivalence* and false for *correctness*, and the gap between those two is where a compliance finding lives.

Option F — Mutation-certified corpus adequacy

Mutate the legacy program; check the input corpus detects each mutant; report the kill rate as the strength of the equivalence claim.

Build cost	Medium — mutation operators on COBOL source are mostly textual
Prior art	Mutation testing is textbook; **applying it to the oracle to certify a migration equivalence claim was not found in any surveyed tool or paper**
Demo strength	Medium in the pitch, **very high** in Q&A
Soundness	**Very high** — it is the only option that quantifies its own confidence

Adopt if time allows. It is the answer to "your tests all passed, so what?" Full development in `02 §N4`.

Option G — Formal / symbolic equivalence proof

Symbolically execute both implementations; discharge equivalence to an SMT solver.

Build cost	**Very high**
Prior art	Active research — KLEE for C/C++; IBM's WCA4Z framework uses symbolic execution on COBOL to generate JUnit tests; SEDCoT couples symbolic execution with delta debugging
Demo strength	High if it worked
Soundness	Highest of any option — proves *for all inputs*, not just the ones run

Reject for the build; cite as the scaling path. Path explosion is the known killer: treating all inputs as symbolic produces formulae that blow up within a few nested conditions, and COBOL's decimal semantics would need faithful encoding in the solver theory. This is a PhD, not a hackathon.

But say it out loud. "The stronger result is symbolic equivalence — that is what IBM's own testing framework reaches for — and here is why we are not claiming it" is a much better answer than being caught not knowing the distinction. It is also the honest boundary of the word *verification*.

Option H — Shadow run / production traffic replay

Capture real transactions from the bank, replay against both systems.

Build cost	Low in principle
Prior art	Crowded and mature — Diffy, AWS Application Testing, Mechanical Orchard, and standard bank dual-run practice
Demo strength	Zero
Soundness	High

Reject: impossible and undesirable. No student team has a bank's production traffic, and requiring it is precisely the barrier that makes the vendor tools unusable before a migration is funded (`../03 §3c`). Generating inputs instead of capturing them is not a compromise forced by lack of access — it is the reason the tool works at the decision stage.

Option I — Metamorphic assurance for unreachable rules

Assert domain relations that hold without knowing the correct answer: monotonicity, additivity across periods, zero-balance, scaling within a tier.

Build cost	Low
Prior art	Established technique (Segura et al., ACM CSUR); not applied in this setting in anything surveyed
Demo strength	Low on its own
Soundness	Medium — relations are asserted by a human and could be wrong

Adopt as an optional secondary. Its value is narrow but real: it says *something* about rules the differential run never reached, which otherwise sit permanently in the *unproven* column. One line in the deck as evidence of depth. Do not build it before D, E or F.

Option J — Materiality-ranked findings

Rank divergences by projected rupee impact across a portfolio distribution rather than by count.

Build cost	Low
Prior art	Not found, but this is a presentation layer, not a technique
Demo strength	**High** for a banking panel
Soundness	Medium — depends on an assumed portfolio distribution, which must be stated

Adopt as polish. Turns "73 inputs diverged" into *"this divergence affects an estimated 0.3% of accounts and drifts ₹X per crore of deposits per quarter."* Cheap, and it speaks the audience's language. Be explicit that the portfolio distribution is an assumption, not a measurement.

Summary

Option	Verdict	Role
A — Documentation generator	Reject	Strawman in the deck
B — Pure differential harness	**Adopt**	Spine; not a claim
C — Decoupled extraction + testing	Reject	This is the market; the seam is the opportunity
D — Reciprocal loop, rule verdicts	**Adopt**	The core
E — Three-way vs regulation	**Adopt**	The differentiator
F — Mutation-certified adequacy	**Adopt if time**	The credibility layer
G — Formal equivalence	Reject	Name it as the scaling path and the honest boundary
H — Production replay	Reject	Impossible; and its absence is the advantage
I — Metamorphic relations	Optional	Partial coverage of unproven rules
J — Materiality ranking	**Adopt**	Presentation polish

The composite of B + D + E + F + J is the recommended system. `03 - Recommended Architecture.md` assembles it.

02 — Novelty Proposals

Ten claims, ranked by strength. Each states what the claim is, the prior art it must survive, how it is built, how it demos, and how it fails.

Strength combines three things: how open the prior art actually is, how defensible the claim is under a hostile question, and how much of it a student team can build.

N1 — Three-way conformance: legacy vs rewrite vs regulation

Strength: highest. Build cost: low. This is the idea to lead with.

The claim

Every tool in this category — IBM's Validation Assistant, AWS Blu Age Compare, Mechanical Orchard's Imogen — treats the legacy program as ground truth *by definition*. That assumption is load-bearing and unexamined, and it is wrong in one specific, important case: **a program written in 1987 can be non-compliant with a Master Direction issued in 2025.**

The submitted write-up says "the legacy system is its own oracle." That is true for **equivalence** and false for **correctness**. Adding a third reference — a small executable model derived from the regulator's stated rules — makes the gap between those two visible.

The classification this produces

With three references, a given input falls into one of four cells:

Legacy	Rewrite	Regulation	Reading	Who needs to know
=	=	=	Verified compliant	Sign-off
≠	—	—	**Translation defect**	The migration team. This is the only cell two-way tools can see.
=	=	≠	**Inherited compliance defect** — the rewrite faithfully reproduces a breach that has been running for decades	Compliance and the board. Invisible to every other tool.
≠	rewrite = reg	—	**Silent correction** — the rewrite "fixed" something. This is a migration *risk*, not a win: the bank's books change at cutover, and someone has to authorise that	Finance and audit

That third row is the one worth the slide. That fourth row is the one that shows sophistication — most people's instinct is that a rewrite matching the regulation is good news, and in a bank it is an unplanned restatement.

How to build it

Hand-write a small reference model — Python or Java, roughly 60–100 lines — implementing only the rules RBI states explicitly and unambiguously for savings deposits: interest on a daily product basis (applied on end-of-day balance), a uniform rate on balances up to ₹1 lakh, differential rates permitted above, and payment of interest rounded off to the nearest rupee for rupee deposits (two decimal places for FCNR(B)).

Do not attempt to model anything the Direction does not state. The model's value comes entirely from its narrowness — every line of it traces to a clause.

How it demos

> *"The rewrite matches the legacy exactly. Both round to two decimals. RBI says round to the nearest rupee. This bank has been non-compliant since before the Direction was issued, and a faithful migration would have carried the breach forward for another forty years."*

How it fails

- **Over-modelling.** If the model encodes anything the regulator did not actually say, every "compliance finding" is really a finding about the team's own interpretation. Mitigation: every rule in the model carries the clause it came from, and the model refuses to guess.
- **Scope creep.** Bank-specific product terms, board-approved rate cards and legacy grandfathering all legitimately override defaults. The model covers the arithmetic conventions, not the commercial terms. Say this before it is asked.
- **The tool is not a compliance authority.** Frame the output as *"this warrants a compliance review"*, never as *"this is a breach."*

Prior art check

Searched vendor documentation and the extraction/validation literature (`../06`). No tool found that introduces a normative third reference. The reason appears structural rather than accidental: these are migration products, and a migration product that tells you your current system is non-compliant is selling a problem its customer did not ask about. A student project has no such constraint.

N2 — Discriminating rule coverage: a real definition of "proven"

Strength: high. Build cost: medium. This is the core.

The claim

The submitted write-up promises *verified* and *unproven*. The weak version of that is line coverage: the rule's lines were executed, therefore the rule is verified. That version is wrong, and a judge with a testing background will find the hole immediately — a line can be executed a thousand times without the rule it encodes ever mattering to the output.

The proposed definition. A rule *R* with boundary *b* is **proven** when:

1. **Reached** — some input executed the source lines in *R*'s slice; *and*
2. **Discriminating** — inputs exist on both sides of *b* whose *legacy outputs differ*, demonstrating that the boundary actually governs behaviour; *and*
3. **Agreed** — the reimplementation matched the legacy on every one of those inputs.

Condition 2 is the whole idea. It is what separates "we ran this code" from "we tested this rule."

Prior art check — narrowed after verification

Rule-level reporting is not original. AgentModernize (arXiv 2605.17535) defines a **Business Rule Preservation Score**: gold-standard rules counted as preserved when they appear in the behavioural specification *and* the modernised code enforces them, as verified by targeted tests.

What survives. BRPS scores against **human-annotated gold-standard rules** — which presupposes the artifact a bank does not have. The verdict here applies to rules the system extracted itself, with no ground truth. And nothing surveyed requires the boundary to be *behaviourally live* before calling a rule verified.

So claim **the discrimination condition**, not per-rule reporting. Standard coverage tooling in this space is still JaCoCo — Java line and branch coverage. See `../07 - Verification Pass.md §C3`.

It also does real work: condition 2 fails exactly when the extraction layer has invented a rule that is not in the program. Which is N3.

How to build it

Slicing gives each rule a line set for condition 1. gcov (or paragraph-level `DISPLAY` tracing — see `../04 §3`) gives per-input line hits. The rule's stated boundary gives the input triple for condition 2. Condition 3 is the diff you already have.

How it demos

A table. Three columns: rule, verdict, evidence.

> *R-04 · "Balances above ₹1,00,000 earn the differential rate" · **PROVEN** · 1,00,000 → ₹287; 1,00,001 → ₹328; both implementations agreed on 6 inputs spanning the boundary.*

How it fails

Multi-variable and path-dependent rules do not have a single scalar boundary, so the triple generalises awkwardly. A-COBREX exists specifically because rules involving multiple business variables are hard. Mitigation: handle scalar threshold rules properly and mark multi-variable rules as *unproven — boundary not derivable*, which is honest and consistent with the whole design.

N3 — The third verdict: **refuted**

Strength: high. Build cost: very low once N2 exists. Best value per hour.

The claim

Two verdicts are not enough. A rule that is *reached* but *not discriminating* — the lines ran, inputs on both sides of the stated boundary produced identical output — is not merely unproven. It is **evidence against the rule as stated**. Either the boundary is somewhere else, or the branch is dead, or the extraction layer hallucinated it.

Mark it **refuted**, and put refuted rules at the top of the report.

Why it matters

The submitted write-up's own argument is that a confidently stated wrong rule is more dangerous than no rule, because an engineer will build on it. Measured hallucination rates in code summarisation reach 66%; the best COBOL rule extractors sit around F1 0.73. So wrong rules are not a hypothetical — they are the expected case for roughly a quarter of the output.

A system that only distinguishes *proven* from *unproven* dumps every hallucinated rule into the same bucket as every merely-untested one. The refuted bucket is where the extraction layer's lies go, and surfacing them is the strongest possible demonstration that the verification layer is doing its job.

How it demos

This is the moment that proves the whole thesis in one screen:

> *R-11 · "Accounts dormant for more than 24 months accrue at the base rate" ·* ***REFUTED*** *· lines 331–348 executed on 214 inputs; dormancy set to 23 and 25 months produced identical output in both implementations. No such boundary exists in the program. The rule was hallucinated.*

An AI system catching its own AI lying, with a reproducible counterexample, is a better demo than any correct output.

How it fails

A rule can be non-discriminating for a legitimate reason — the input generator never varied the right field, or the boundary is masked by an earlier condition. So *refuted* must mean **"contradicted by the evidence we have"**, not **"false"**. Report the counterexample alongside the verdict so a human can adjudicate. Getting this wording wrong turns a strength into an overclaim.

N4 — Mutation-certified corpus adequacy

Strength: high. Build cost: medium. Wins the Q&A.

The claim

The hardest question in `../05` is: *"everything passed — what have you actually proved?"* Coverage percentages do not answer it. This does.

Mutate the legacy COBOL — the oracle itself — and check that the input corpus detects each mutant. Report the kill rate as the measured strength of the equivalence claim.

If moving a threshold from 10,000 to 10,001 in the legacy program does not cause a single input in the corpus to produce a different answer, then the corpus does not prove anything about that threshold, and the equivalence claim over that rule is worth nothing regardless of how many tests passed.

Mutation operators, all textual and cheap

Operator	Example	Rule class it probes
Threshold shift	`100000` → `100001`	Tier boundaries
Comparison flip	`>` → `>=`	Off-by-one at boundaries
Rounding removal	delete `ROUNDED`	Rounding conventions
Scale change	`PIC S9(7)V99` → `V999`	Truncation points
Constant perturbation	rate `3.5` → `3.6`	Rate application
Day-count swap	`360` → `365`	Accrual convention

Each operator maps to a rule class, so the kill rate can be reported **per rule** rather than as one global number — which composes directly with N2.

How it demos

> *"Our corpus kills 47 of 50 injected mutations in the accrual logic. The three survivors are all in the dormancy paragraph, which is why R-11 and R-14 are marked unproven rather than verified. We are telling you what we did not test."*

How it fails

- **Equivalent mutants** — a mutation that genuinely does not change behaviour cannot be killed by anyone, and inflates the miss rate. This is the classic problem in mutation testing; with textual operators on numeric constants it is rarer than usual, but say so before it is asked.
- **Cost** — every mutant needs a recompile and a full corpus run. Bound the operator count and the corpus size; this is a demo, not a nightly job.

Prior art check

Mutation testing is textbook. **Using mutants of the legacy oracle to certify the adequacy of a migration equivalence claim** was not found in any surveyed product or paper. Claim the combination, not the technique — and say the technique is old, which strengthens rather than weakens the claim.

N5 — Rule-directed input generation (the reciprocal loop, input side)

Strength: medium-high. Build cost: medium. Already in the write-up.

The claim

Existing tools generate test inputs from *code structure* (IBM: symbolic execution over the COBOL) or capture them from *production traffic* (AWS, Mechanical Orchard). None derives them from the **extracted natural-language rules**. Doing so means the extraction layer earns its place in the verification pipeline instead of being a parallel deliverable.

Concretely: a rule stating a uniform rate up to ₹1 lakh causes the generator to emit 99,999 / 1,00,000 / 1,00,001. A rule mentioning rounding causes it to search for accruals landing on an exact ₹0.50. A rule mentioning month-end causes 28 and 29 February, 30 and 31, and quarter ends.

Why it is only medium-high

It is the *mechanism*, and mechanisms are hard to defend as novel — a reviewer can reasonably say "so you prompted for boundaries." Its real value is that it makes N2 and N3 possible at all: without rule-derived boundaries there is nothing to test discrimination against. Present it as plumbing that enables the verdict, not as the headline.

How it fails

Rules stated vaguely produce no boundary. Mitigation: constrain extraction output to a schema with an optional typed `boundary` field, and treat a rule without a derivable boundary as *unproven — boundary not derivable*. Not every rule needs to yield a test.

N6 — Structural provenance: citations that cannot be hallucinated

Strength: medium. Build cost: low. Quiet but genuinely good.

The claim

The write-up promises every rule carries a pointer to the lines that produce it. The naive implementation asks the model to cite line numbers — and models hallucinate line numbers.

Instead: run extraction over **slices**. The slice defines the line set before the model is called, so the citation is a property of the pipeline, not an assertion by the model. The model cannot cite a line it was never shown.

This is an architectural guarantee rather than a mitigation, and it is the kind of detail that signals the team understands where LLM systems actually break. Slicing on business-concept variables is also exactly what COBREX and the model-based frameworks do, so the method has precedent.

How it fails

Slicing quality caps rule quality. A slice that misses a data dependency yields a rule with incomplete provenance. Standard limitation; state it.

N7 — Intake scope gate: the tool refuses work it cannot do soundly

Strength: medium. Build cost: low. Excellent optics.

The claim

The excluded categories — CICS, JCL, database coupling, clock reads, randomness — are not merely documented as out of scope. They are **statically detected and explicitly refused, with a reason.**

ProLeap extracts `EXEC CICS` and `EXEC SQL` as text, so they are trivially detectable. Clock intrinsics are a keyword scan. On finding one:

> `REFUSED — program reads CURRENT-DATE at line 88. Differential testing requires determinism: the same input must produce the same output on every run. This program cannot be verified by this method.`

Why it matters

It is the same honesty argument as verified/unproven, moved to the front door. Every tool that silently produces a confident answer on input it cannot handle is generating exactly the liability this project exists to prevent. A tool that knows its own boundary is a tool a bank can trust with the cases inside it.

How it fails

Detection is incomplete — indirect non-determinism through a called subprogram will slip past a keyword scan. Mitigation: a second signal, running each input twice and flagging any output that differs from itself. Cheap, and it catches what static detection misses.

N8 — The evidence bundle as an audit artifact

Strength: medium. Build cost: low. Product novelty, not technical novelty.

The claim

The output is not a report, it is a **reproducible evidence bundle**: extracted rules with provenance, the complete input corpus, both implementations' outputs for every input, per-rule verdicts, mutation kill rates, the regulation model and its clause citations, plus tool versions and source hashes so the entire run can be reproduced.

RBI's IT Governance Master Direction (effective 1 April 2024) requires regulated entities to maintain a board-approved Change Management Policy. A core-engine rewrite is therefore a board-visible, auditable event that needs evidence, not assurances. This bundle is shaped to be that evidence.

Why it belongs in the pitch

It reframes the deliverable from "a testing tool" to "the artifact someone can sign", which is what the write-up already claims in Part 4 and what a banking audience actually buys. And the specification and regression suite retain their value even if the migration is cancelled — which removes the reason to defer the decision.

How it fails

It is packaging. If N2–N4 are not built, the bundle is a folder of nothing. Build it last.

N9 — Materiality-ranked findings

Strength: low-medium. Build cost: low. Speaks the audience's language.

The claim

Rank divergences by projected rupee impact across an assumed portfolio distribution rather than by count.

> *"This divergence occurs on 0.3% of inputs. Across a ₹500 crore savings book it drifts approximately ₹X per quarter — and because RBI prescribes rounding to the nearest rupee, it is a Master Direction exposure, not a rounding preference."*

How it fails

The portfolio distribution is assumed, not measured. State that on the slide. An unlabelled assumption presented as a number is the exact failure mode this whole project is against.

N10 — Delta-debugged minimal counterexample

Strength: low as novelty, high as demo. Build cost: low.

The claim

When a divergence is found, shrink the failing input to a minimal counterexample. "73 inputs diverge" becomes "the divergence occurs whenever the accrual lands on an exact half-rupee."

Not novel — this is SEDCoT's third phase, and delta debugging is a well-established technique. Build it anyway: it is perhaps forty lines of code and it produces the single most convincing screen in the demo. Present it as good engineering practice, not as a contribution.

Ranking summary

Strengths below are **post-verification** (`../07 - Verification Pass.md §D`).

#	Novelty	Strength	Build cost	Prior art
N1	Three-way conformance vs regulation	**Highest — strengthened**	Low	Open. Locksmith and AgentModernize: no regulatory checks. EvolveWare markets rules for auditors but is static documentation only, with no execution.
N2	Discriminating rule coverage	**Medium-high** (was High)	Medium	Partially taken — BRPS reports at rule level. The discrimination condition is open.
N3	Refuted verdict	**High**	Very low	Open — survived intact
N4	Mutation-certified adequacy	**Medium-high** (was High)	Medium	Locksmith uses *parity-preserving* mutations for path exploration — different purpose. Fault injection for adequacy still open.
N5	Rule-directed input generation	Medium	Medium	Crowded — Locksmith's Witness Search generates inputs, though from structure not rules
N6	Structural provenance	Medium	Low	Method has precedent
N7	Intake scope gate	Medium	Low	Not found; minor
N8	Evidence bundle	Medium	Low	Product framing
N9	Materiality ranking	Low-medium	Low	Presentation
N10	Delta debugging	Low	Low	**Prior art — do not claim**

The defensible novelty claim, after verification:

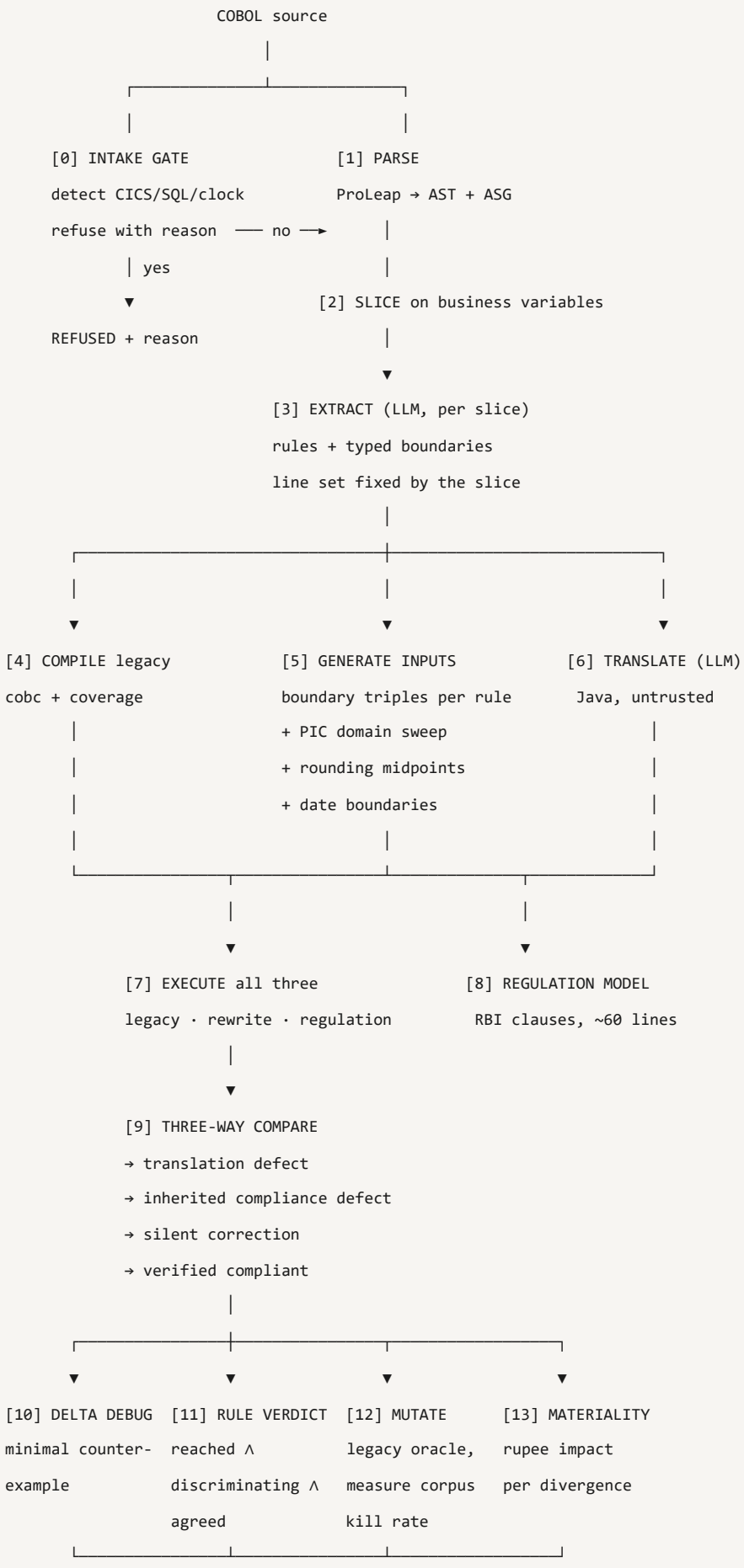
> *A verification harness that issues a runtime verdict on the business rules it **extracted itself** — proven, unproven, or **refuted** — where "proven" requires the rule's boundary to be demonstrably live; that measures the strength of its own claim by **injecting faults into the legacy oracle**; and that checks both implementations against the regulator's stated rules as a third reference, so it can tell a translation defect apart from a compliance breach the rewrite inherited faithfully.*

Every clause of that survived a deliberate falsification attempt. The earlier version of this claim also asserted the differential mechanism and per-rule reporting as novel; **neither is** — the first is prior art in three products, the second in AgentModernize.

03 — Recommended Architecture

The composite of options B + D + E + F + J, carrying novelties N1–N8. Deterministic pipeline, bounded LLM calls — justified by the controlled study showing deterministic orchestration matches agentic accuracy at up to 3.5× lower token cost with better worst-case robustness (`../02 §2`).

Pipeline


COBOL source
[0] INTAKE GATE
detect CICS/SQL/clock
refuse with reason
no
yes
REFUSED + reason
[1] PARSE
ProLeap → AST + ASG
[2] SLICE on business variables
[3] EXTRACT (LLM, per slice)
rules + typed boundaries
line set fixed by the slice
[4] COMPILE legacy
cobc + coverage
[5] GENERATE INPUTS
boundary triples per rule
+ PIC domain sweep
+ rounding midpoints
+ date boundaries
[6] TRANSLATE (LLM)
Java, untrusted
[7] EXECUTE all three
legacy · rewrite · regulation
[8] REGULATION MODEL
RBI clauses, ~60 lines
[9] THREE-WAY COMPARE
→ translation defect
→ inherited compliance defect
→ silent correction
→ verified compliant
[10] DELTA DEBUG
minimal counter-
example
[11] RULE VERDICT
reached ∧
discriminating ∧
agreed
[12] MUTATE
legacy oracle,
measure corpus
kill rate
[13] MATERIALITY
rupee impact
per divergence

```
                                  |
                                  ▼
                      [14] EVIDENCE BUNDLE
           specification · regression suite · equivalence claim
```

Stage notes

[0] Intake gate. Keyword and AST scan for `EXEC CICS`, `EXEC SQL`, `CURRENT-DATE` and other clock intrinsics, `RANDOM`, external file I/O beyond the declared interface. Plus the dynamic check: run every input twice and flag any program whose output differs from itself. Refuse with a reason rather than producing an unsound verdict (N7).

[1]–[2] Parse and slice. ProLeap gives AST plus an Abstract Semantic Graph carrying data and control flow. Slice on business-concept variables — the method COBREX and the model-based frameworks use. If ProLeap's Java/Python boundary costs too much time, fall back to a paragraph-level sectioniser; do not let the parser become the project (`../04 §7`).

[3] Extract. One bounded LLM call per slice. Output constrained to a schema:

```
{
  "id": "R-04",
  "statement": "Balances above ₹1,00,000 earn the differential rate.",
  "lines": [212, 213, 214, 215],
  "boundary": { "variable": "WS-BAL", "op": ">", "value": "100000.00", "type": "decimal" },
  "regulation_ref": "MD Interest Rate on Deposits, savings – differential rate above ₹1 lakh"
}
```

`lines` is written by the pipeline from the slice, never by the model (N6). `boundary` is optional; its absence means the rule cannot be discrimination-tested and is reported as *unproven — boundary not derivable*.

[5] Generate inputs. Three sources, in priority order:

1. **Rule-directed** (N5) — for each rule with a boundary b, emit $b-\varepsilon$, b, $b+\varepsilon$ at the field's declared precision. For rounding rules, search for inputs whose accrual lands on an exact midpoint. For date rules, emit 28/29 Feb, month ends, quarter ends, the 31st.
2. `PIC`**-derived sweep** — the picture clause defines the domain, sign and extremes for free. Zero, negative, maximum, one past maximum.
3. **Random fill** — volume, to catch what the first two missed.

[7]–[9] Execute and compare three ways. The regulation model is a peer, not an authority: it is consulted, and it is scoped only to what RBI states explicitly. The four-cell classification is in `02 §N1`.

[11] Rule verdict. The heart of the system (N2, N3):

```
for each rule R:
    reached        = any input executed R.lines
    discriminating = ∃ inputs on both sides of R.boundary
                       with DIFFERENT legacy outputs
    agreed         = legacy == rewrite on all inputs touching R.lines

    if not reached:                    UNPROVEN  (never exercised)
    elif not R.boundary:               UNPROVEN  (boundary not derivable)
    elif not discriminating:           REFUTED   (boundary not live — evidence against the rule)
    elif not agreed:                   FAILED    (divergence; see counterexample)
    else:                              PROVEN
```

Five outcomes in the implementation, three in the report — FAILED rolls into the divergence list, which is where an engineer wants it.

[12] Mutate. Textual operators on the legacy source (N4): threshold shift, comparison flip, `ROUNDED` removal, scale change, constant perturbation, day-count swap. Recompile, rerun the corpus, record whether any output changed. Report the kill rate **per rule**, so a proven rule with a surviving mutant gets downgraded — that is the corpus admitting it did not really test the rule.

Report structure

Page 1 — Verdict. Rules extracted: 47. Proven 31 · Unproven 11 · Refuted 5. Divergences: 3. Mutation kill rate: 94%. Compliance findings: 1.

Page 2 — Divergences, materiality-ranked (N9), each with a delta-debugged minimal counterexample (N10).

Page 3 — Refuted rules. The extraction layer's errors, with the counterexample that contradicts each. *"Here is where our own AI was wrong, and here is the input that proves it."*

Page 4 — Unproven rules. Named, with the reason: never exercised, or boundary not derivable.

Page 5 — Compliance findings (N1). The three-way classification, with the clause each finding is measured against, framed as *warrants review*.

Appendix — the evidence bundle (N8). Full corpus, all outputs, tool versions, source hashes.

Build order

Follows `../04 §7`, with the novelty stages marked.

Stage	Deliverable	Novelty	Cut priority
0–2	GnuCOBOL running; differential harness; `PIC`-derived inputs	—	Never cut. This is the spine.
3	Slice → LLM rules with pipeline-written citations	N6	Simplify before cutting
4	Rule-directed boundary generation	N5	**Protect**
5	Three-valued rule verdict with discrimination test	**N2, N3**	**Protect — this is the core**
6	Regulation model + three-way compare	**N1**	**Protect — this is the differentiator**
7	Delta debugging	N10	Cheap; keep
8	Mutation certification	N4	First to cut if time runs out
9	Intake gate	N7	Cheap; keep
10	Materiality + evidence bundle	N8, N9	Presentation; do last

Minimum viable novel system: stages 0–2, 4, 5, 6. Everything else is strengthening. A demo with the differential harness, the three-valued rule table and one compliance finding is a complete argument.

04 — Pitch Language

The novelty at four lengths. Use whichever fits the slot.

One sentence

> *Everyone proves the rewrite matches the legacy; we also ask whether the legacy matches the regulator — and we report the answer per business rule, including the rules we could not prove and the ones our own AI got wrong.*

One slide

What everyone builds: legacy in, rewrite in, diff the outputs. IBM, AWS and Mechanical Orchard all ship this. They are right that verification is the bottleneck.

What they all assume: that the legacy program is correct by definition.

Three things we add:

1. **A third reference — the regulation.** RBI prescribes daily product basis and rounding to the nearest rupee. When the legacy and the rewrite agree with each other and both breach the Direction, a two-way diff sees nothing and we see a compliance finding.
2. **Verdicts per business rule, not per test case.** Proven, unproven, refuted. A rule is only *proven* when inputs on both sides of its boundary produced different answers — proving the rule is actually live — and both implementations agreed. *Refuted* means our own extraction hallucinated it, and we show the input that proves it.
3. **We test our own tests.** We mutate the legacy program — move a threshold by one rupee, remove a `ROUNDED` — and measure whether our inputs notice. If they do not, we do not claim that rule is verified. The equivalence claim comes with its own strength score.

And it runs on a laptop. No mainframe, no production traffic, no licence — just the source file and GnuCOBOL. Which means a bank can run it *before* deciding whether to modernise at all.

Thirty seconds, spoken

> *The industry already agrees with us that verification is the bottleneck — IBM Research's own paper on their COBOL translation product says the output cannot be trusted. What nobody questions is the assumption underneath: that the legacy system defines the right answer. It defines the current answer. A program written in 1987 can be non-compliant with a Master Direction issued in 2025, and a perfectly faithful migration carries the breach forward.*
>
> *So we run three things, not two: the legacy, the rewrite, and a small model of what RBI actually prescribes. And we report per business rule — of forty-seven rules we extracted, thirty-one are proven by execution, eleven we could not prove, and five were contradicted by the evidence, which means our own extraction hallucinated them. We show you those five. Then we mutate the legacy program to check our inputs would have caught a one-rupee change to each threshold — because if they would not have, we have not proved anything, and we would rather say so.*

The demo, in the order it should be shown

1. **The COBOL.** A savings-accrual program. Four hundred lines, no comments. *"This is the specification. There is no other one."*
2. **The rules.** Twenty extracted, each with its line numbers. *"That took eleven seconds. Roughly a quarter of it is probably wrong, which is why this is not the product."*
3. **The corpus.** Boundaries derived from the rules themselves — 99,999 / 1,00,000 / 1,00,001, accruals landing on exactly ₹0.50, 29 February. *"The rules tell us where to look."*
4. **The divergence**, delta-debugged to one line. *"Balance ₹1,00,000, twelve days at 3.5%. Legacy: ₹115. Rewrite: ₹116. The Java uses HALF_UP; the COBOL truncates. Across a five-hundred-crore book that is not a bug report, it is a rounding drift the auditor finds first."*
5. **The refuted rule.** *"Rule eleven says dormant accounts drop to the base rate. We tested 23 and 25 months. Identical output, both sides. No such boundary exists — our own extraction invented it. An engineer would have built on that."*
6. **The compliance finding.** *"Here both implementations agree, and both round to two decimals. RBI says nearest rupee. This is not a migration defect. This has been running for forty years."*
7. **The coverage table.** 31 proven, 11 unproven, 5 refuted, 94% mutation kill rate. *"And these eleven, we could not prove. We are telling you which ones."*

Step 5 is the moment the room understands the thesis. Step 6 is the moment a banker starts caring. Step 7 is why they would trust it.

Lines to keep from the existing write-up

- *"The code is the specification."*

- *"A confidently stated wrong rule is more dangerous than no rule at all."*
- *"Coverage is reported rather than assumed."*
- *"Narrow scope is a design decision, not a limitation we hope nobody notices."*

Lines to retire

- *"Everyone else's submission assumes their AI is right. Ours assumes it is wrong, and proves the answer anyway."* — true of the room, false of the market. Replace with the one-sentence version at the top of this file.
- Anything phrased as *"nobody does this"* about differential testing itself. Three vendors do. The rule-level verdict and the regulatory third reference are the claims that survive.

The two hardest questions, answered from this folder

"There's a paper from last week doing exactly this — Locksmith."

> *Yes, arXiv 2607.28271, published 30 July. Same spine: legacy as oracle, both targets off-mainframe, parity gate, 91.90% branch coverage. We'd rather you heard it from us. It's the best evidence we have that the architecture works — an industry team built it independently and got full parity on every accepted test. What it doesn't do, and we checked the paper for this specifically: no business-rule verdicts, and no regulatory checks. Its mutations are parity-preserving, used to reach new paths — the opposite of injecting faults to test whether your corpus would have noticed. That's the space we're in.*

"IBM already ships equivalence validation. What's new?"

> *Three things. They validate per test case; we report per business rule, including the ones we could not prove. They need IBM Z and captured mainframe traffic; we need a source file. And they treat the legacy as correct by definition — so if the legacy has been breaching a Master Direction since 1987, their tool certifies the rewrite as a success and never mentions it. Ours does.*

"Everything passed. What have you actually proved?"

> *Equivalence over the inputs we ran, and nothing more — which is why we mutate the legacy program and measure whether our corpus would have noticed. Ninety- four percent of injected mutations were caught. The six percent that survived are in the dormancy logic, which is exactly why those rules are marked unproven rather than verified. We are not claiming a proof for all inputs; that is symbolic equivalence checking, and it is a different and much harder problem. We are claiming evidence, and we are telling you how strong it is.*

07 — Verification Pass

Independent re-check of every load-bearing claim in this folder, done on 4 August 2026 against primary sources where they exist and two or more independent sources where they do not.

Outcome: 12 claims confirmed, 6 corrections, 1 novelty claim partially falsified, and 1 piece of prior art found that is closer to this project than anything in `03`. Details below. The corrections have been applied to the affected files; this page records what changed and why, so nothing is quietly rewritten.

A. Confirmed against primary sources

#	Claim	Verification	Status
1	IBM says its own translation output cannot be trusted	arXiv 2504.10548 abstract, quoted verbatim: "*the resulting code cannot be trusted to correctly translate the original code*"	**Confirmed — safe to quote**
2	*Lost in Translation* figures	Abstract verbatim: "*correct translations ranging from 2.1% to 47.3% for the studied LLMs*"; 1,700 samples; 1,748 manually labelled bugs; 15 bug categories; 43K+ translations; 1,365 bug-fix pairs	**Confirmed**
3	Deterministic orchestration beats agentic on cost	arXiv 2605.09894 verbatim: "*Deterministic execution also reduces token consumption by up to 3.5x*" and "*achieves comparable computational accuracy to LLM-controlled orchestration while improving worst-case robustness and reducing performance variability across runs*"	**Confirmed**
4	RBI: daily product basis, ₹1 lakh threshold, nearest-rupee rounding	RBI primary source, with clause numbers — see correction **C1** on the year	**Confirmed (clauses), corrected (year)**
5	TSB fine £48,650,000	FCA press release, "*TSB fined £48.65m for operational resilience failings*"; joint FCA/PRA; £32.7m redress	**Confirmed**
6	A-COBREX 62.21% precision / 74.12% recall; COBRAIN F1 0.73 vs COBREX 0.59	Consistent across IBM Research publication page, ICSE 2025 listing, and the EASE 2025 ACM entry	**Confirmed**
7	GnuCOBOL passes 9,700 of 9,748 NIST tests	GnuCOBOL FAQ / Savannah project page	**Confirmed, with a new caveat — see C5**
8	BCG: Indian lenders need ~$1bn over 5–10 years	Business Standard reporting the BCG report *Cloud-based Core Transformations*, August 2024. New corroborating detail: ~80% of Indian bank IT budget goes to "run the bank" vs "change the bank"	**Confirmed**
9	IBM WCA4Z Validation Assistant checks semantic equivalence	Corroborated by IBM documentation summary, Futurum, VTI, and independently by arXiv 2504.10548, which describes the testing framework built for WCA4Z	**Confirmed**
10	Mechanical Orchard requires production data	mechanical-orchard.com/platform verbatim: Imogen "*verifies it against real production data, slice by slice*", and "*reads your source directly*"	**Confirmed**
11	Cognizant markets rule extraction and spec generation	Cognizant's own blog: "*Existing applications can be reverse engineered to fill documentation gaps, interactively explain specific code, and extract business rules*" and generative AI can "*understand existing business logic and generate specifications for parity in new implementations*"	**Confirmed — plus a useful detail: that page describes no validation or equivalence testing at all**
12	TSB: 1.9 million customers locked out	Not in the FCA release, which says only "*all of TSB's branches and a significant proportion of its 5.2 million customers*". The 1.9m figure is corroborated independently by The Register, IEEE Spectrum and The Courier	**Confirmed, but attribute to press reporting, not to the regulator**

B. The significant find: Locksmith

arXiv 2607.28271 — *Agentic Method for Deterministic Validation of Legacy Code Migration*, published 30 July 2026 — five days before this dossier was compiled.

It is closer to this project than anything in `03 - Competitive Landscape.md`. What it does:

- Uses the legacy COBOL program as an **execution oracle**; a *"Parity Gate"* runs both targets on the same inputs and reports end-state discrepancies.
- **Runs both COBOL and Java off-mainframe on commodity hardware**, with external calls mocked.
- Generates inputs via *"Witness Search"* over input mocks to penetrate branches, then applies **parity-preserving mutations** to reach new paths.
- Identifies *"Locked Paragraphs"* — conditions blocking deeper exploration.
- Results: three case studies of 430–4,114 lines; 91.90% branch coverage on a production-like internal COBOL program; all accepted test cases matched.

What this costs us. The claim in `03 §3c` that every equivalent capability requires a mainframe is true of *shipping products* (IBM Z, AWS's captured mainframe traffic, Mechanical Orchard's production data flows) but **not of the research frontier**. Say "every shipping product", not "everyone".

What survives. Verified explicitly against the paper: it reports **no business-rule verdicts and no regulatory compliance checks**. Its mutations are *parity-preserving* — used to reach new execution paths, the opposite purpose from injecting faults to measure corpus adequacy. And it is agentic, which arXiv 2605.09894 gives independent reason to think is the more expensive and less stable choice.

How to handle it in the pitch. Do not hide it. A five-day-old paper from an industry team independently building this architecture, and getting 91.90% branch coverage with full parity, is the strongest possible evidence that the approach is sound. Cite it as validation, then say precisely what it does not do.

C. Corrections applied

C1 · The RBI Master Direction year *(affects `01`, `02`, `06`, Solution Folder, visual 07)*

What was written: *Master Direction — Reserve Bank of India (Interest Rate on Deposits) Directions, 2025, 1 April 2025.*

What is verified: the clauses come from **Master Direction — Reserve Bank of India (Interest Rate on Deposits) Directions, 2016** (3 March 2016, updated as on 7 June 2024), at `rbi.org.in/Scripts/BS_ViewMasDirections.aspx?id=10296`, with these clause numbers:

- **Clause 6(a)** — *"Interest on domestic rupee savings deposits shall be calculated on a daily product basis"*
- **Clause 6(a)(1)** — *"A uniform interest rate shall be set on balance up to Rupees one lakh, irrespective of the amount in the account within this limit."*
- **Clause 6(a)(2)** — *"Differential rates of interest may be provided for any end-of-day savings bank balance exceeding Rupees one lakh."*
- **Clause 4(f)** — *"All transactions, involving payment of interest on deposits shall be rounded off to the nearest rupee for rupee deposits and to two decimal places for FCNR (B) deposits."*

A 2025 restatement does exist — RBI's own FAQ PDF dated 01042025 refers to the *Interest Rate on Deposits Directions, 2025*, and there is a *Commercial Banks — Interest Rate on Deposits Directions, 2025*. **The clause numbering in the 2025 version has not been verified.** Cite the 2016 clause numbers, which are confirmed, and note the 2025 restatement without asserting its numbering.

Bonus find, and it sharpens the demo. A related RBI circular states the rounding rule precisely: *"fraction of 50 paise and above shall be rounded off to the next higher rupee and fraction of less than 50 paise shall be ignored."* That is **HALF_UP at the rupee**, explicitly. It gives the reference model an exact rounding mode instead of an inferred one — and it is the specific behaviour a Java rewrite using `HALF_EVEN` or two-decimal rounding gets wrong.

C2 · CardDemo's `CBACT04C.cbl` is the wrong demo target *(affects `04 §4`, `04 §5`)*

What was written: *"`CBACT04C.cbl` in CardDemo is an interest-calculation batch program — check it first; if it is tractable, it is the ideal headline demo target."*

What the file actually contains: it is an interest calculator (652 lines, ~605 of code) — but it reads **four VSAM files** (transaction category balance, cross-reference, disclosure group, account master) and calls `FUNCTION CURRENT-DATE` to timestamp transaction records.

Both of those are **excluded by this project's own scope rules**: database coupling breaks input reproducibility, and a clock read breaks determinism outright. Recommending it as the headline demo target was wrong.

The fix, which is better than the original plan. Run the intake gate on it and let the tool **refuse it, correctly, with a reason** — on AWS's own published sample. That is a live demonstration of N7 on third-party code:

```
REFUSED — CBACT04C.cbl reads FUNCTION CURRENT-DATE at line N and opens four VSAM files. Differential testing requires determinism. This program cannot be verified by this method.
```

Then extract the interest arithmetic (`(balance × rate) / 1200`) into a pure computational unit for the actual verification demo, and say plainly that this is what was done. Sweep the rest of CardDemo for a genuinely pure program to use as the headline instead.

C3 · "Nothing reports coverage per business rule" is **false** *(affects `02 §7`, `03 §3a`, Solution Folder `02 §N2`)*

arXiv 2605.17535 — *AgentModernize: Preserving Business Logic in Legacy Modernization with Multi-Agent LLMs and Behavioral Specification Graphs* defines a **Business Rule Preservation Score (BRPS)**: the percentage of gold-standard business rules correctly represented in the modernised output, where a rule counts as preserved if it appears in the behavioural specification *and* the modernised code enforces it, as verified by targeted test cases. It also defines a **Behavioural Equivalence Rate (BER)**.

That is rule-level verification reporting. The claim as originally written does not survive.

What does survive, checked against the paper:

- BRPS scores against **human gold-standard rules** — a research benchmark metric. It presupposes someone already wrote down the correct rules, which is exactly the artifact a bank does not have. The proposed verdict applies to rules the system extracted itself, with no ground truth available.
- There is **no discrimination condition** — nothing requiring that inputs on both sides of a boundary produce different outputs.
- There is **no refuted verdict**, and no notion of a rule being contradicted by evidence.
- It does not execute a legacy program as an oracle.

So N2 and N3 are narrower than claimed but not empty. Restate as: *rule-level scoring exists as a benchmark metric against gold-standard rules; no surveyed work issues a runtime refutation verdict on rules the system extracted itself.*

A statistic worth taking from that paper. Full AgentModernize achieved a mean Behavioural Equivalence Rate of **9.4% (GPT-4o-mini), 8.1% (GPT-4o) and 19.4% (codex)** — with baselines at **0.0%**. Its behavioural specification graph captured **91.2% of gold-standard rules**, so extraction works and *code generation* is the bottleneck. That pair of numbers — extraction ~91%, behavioural equivalence under 20% — is the sharpest single argument in this whole dossier for why verification, not extraction, is the product.

C4 · "No mainframe required" — narrow the claim *(affects `03 §3c`, `05`, visual 06)*

True of IBM, AWS and Mechanical Orchard, all verified. Not true of Locksmith, which runs both targets off-mainframe on commodity hardware. Change "every tool" to "every shipping product" throughout.

C5 · GnuCOBOL conformance caveat *(affects `04 §1`)*

New detail from the GnuCOBOL project's own materials: it passes 9,700 of 9,748 NIST tests, **but explicitly does not claim to be a "Standard Conforming" implementation of COBOL.** State both. It strengthens the existing caveat rather than undermining it — and volunteering it is better than being caught by it.

C6 · AWS product naming has moved *(affects `03`, `06`)*

AWS's own documentation now states that both the **Mainframe Modernization self-managed experience** and the **Managed Runtime experience** are *"no longer open to new customers"*, with capabilities folded into **AWS Transform**. The Blu Age Compare and Application Testing descriptions in `03` are accurate as published but describe a product line being restructured. Say "AWS Transform for mainframe (formerly Blu Age / AWS Mainframe Modernization)" to avoid sounding out of date.

D. Novelty claims after verification

Claim	Before	After	Basis
N1 · Three-way conformance vs regulation	Highest	**Highest — strengthened**	Locksmith: no regulatory checks (confirmed). AgentModernize: none. EvolveWare markets rule extraction for auditors but is **static documentation only** — it does not execute or compare behaviour against a regulation. No executable regulatory conformance found anywhere.
N2 · Discriminating rule coverage	High	**Medium-high**	Rule-level scoring exists (BRPS). The discrimination condition does not. Claim the condition, not the concept.
N3 · Refuted verdict	High	**High**	No surveyed work issues a contradiction verdict on self-extracted rules. Survives intact.
N4 · Mutation-certified adequacy	High	**Medium-high**	Locksmith uses *parity-preserving* mutations for path exploration. Fault injection into the oracle to score corpus adequacy is a different purpose and was not found. Narrower than claimed, still open.
N5 · Rule-directed input generation	Medium-high	**Medium**	Locksmith's Witness Search generates inputs from program structure, not from extracted rules — so the specific mechanism holds, but input generation for this purpose is now a crowded area.
N6–N10	unchanged	unchanged	Not contested by anything found.

The revised one-sentence novelty claim:

> *A verification harness that issues a runtime verdict on the business rules it extracted itself — proven, unproven, or **refuted** — where "proven" requires the rule's boundary to be demonstrably live; that measures the strength of its own claim by injecting faults into the legacy oracle; and that checks both implementations against the regulator's stated rules as a third reference, so it can tell a translation defect apart from a compliance breach the rewrite inherited faithfully.*

Every clause of that survived the falsification attempt. The earlier version claimed the differential mechanism and the per-rule reporting as novel; neither is.

E. What is still unverified

Stated plainly, because the point of this page is that you should not have to take my word for it.

1. ~~**The clause numbering of the 2025 RBI Direction.**~~ **Half-resolved.** RBI's own 2025 FAQ (now in `citations and research paper/pdfs/`) cites paragraphs 4.22, 9.1.6, 10.2, 20.2.1, 22, 29.1, 29.5, 29.8 — a **decimal scheme**, unlike the 2016 Direction's *clause 4(f) / 6(a)(1)*. So the 2016 references confirmedly **do not** carry over. Which 2025 paragraphs hold the savings, threshold and rounding rules is still unknown; the FAQ does not cover them. Keep citing 2016.
2. **Whether GnuCOBOL and IBM Enterprise COBOL agree on specific `COMP-3` and `ROUNDED` edge cases.** Needs an Enterprise COBOL installation nobody on the team has. Remains a stated limitation, not a resolved question.
3. **The absence claims are still absence-of-evidence.** I attempted falsification on the two that matter and it cost one claim (C3) and narrowed two more. A more exhaustive search — particularly of patents, where a "rule base coverage" patent surfaced and was not read in full — could narrow them further.
4. **Whether any pure-computation program exists in CardDemo** suitable as a headline demo target. `CBACT04C` is ruled out; the rest of the repository has not been swept.
5. **Judging criteria for the hackathon.** Still unknown. Every positioning recommendation assumes a technically literate Cognizant panel.

F. What this changes about the plan

Nothing structural. The problem statement is unaffected and every figure in it held up. The architecture in `Solution Folder/03` is unchanged — if anything Locksmith is evidence that it works, since an industry team built the same spine and reached 91.90% branch coverage with full parity.

Three concrete actions:

1. **Lead with N1.** The regulatory third reference is now the only novelty claim that survived unchallenged, and it got stronger.
2. **Re-pick the demo target.** `CBACT04C` becomes the intake-gate demonstration instead of the verification demonstration.

3. **Add Locksmith and AgentModernize to the landscape**, and use AgentModernize's 91.2%-extraction / sub-20%-equivalence pair as the headline evidence for the thesis. It is better than anything the dossier had before this pass.

08 — Business Plan Research

Evidence and assumptions behind the round-2 business-plan content (CII Eastern Region Hackathon, submission 13 August 2026). The CII template mandates fields the round-1 material never covered: target users, market potential, technical details, investments, returns and timelines. This file records where each new claim came from, so that any sentence on the round-2 slides can be traced to a source — or to a clearly labelled assumption.

Convention: claims marked **[ASSUMPTION]** are the team's own constructions and have no external source. They are defensible as estimates, but say so if asked. Claims marked **[contested]** follow the convention in `01`.

---

1. Target users and stakeholders

The four-stakeholder mapping used on the deck. Roles are sourced; the pain statements are inference from the sourced material.

Stakeholder	Role	Pain (as pitched)	Status
Bank CIO / head of core banking	Owns the migration decision and the cutover date	Cannot sign off a replacement nobody can prove behaves identically	Inference — reasonable, unsourced **[ASSUMPTION]**
Risk, compliance, internal audit	Owns the board-approved Change Management Policy that RBI's IT Governance Master Direction (Nov 2023, effective Apr 2024) requires	No auditable artefact showing old and new agree, case by case	Role is sourced (`01 §6`); pain is inference **[ASSUMPTION]**
Modernisation vendors and SIs (Cognizant, TCS, Infosys, in-house)	Deliver the migration programmes	Translation is commoditised; assurance is what stretches the programme	Consistent with `03` and `07` findings; the *stretch* claim is inference **[ASSUMPTION]**
The supervisor (RBI, the board)	Oversees	An IT failure is a supervisory event, not an engineering incident	Sourced — HDFC 2020 order, `01 §3`

How to use this: the strongest line is the summary — *nobody on this list is short of a translator; everybody on this list is short of proof*. The named buyer is the person who signs the cutover memo. If a judge with banking experience pushes back on any pain statement, concede it is the team's reading and invite correction — do not defend it as researched fact.

Open item: validate the pain statements with one real practitioner (a core-banking engineer or bank IT auditor) before the final round in Kolkata. One conversation upgrades all four rows from inference to testimony.

---

2. Market potential

Figures used on the deck, and their standing

Figure	Source	Confidence
~$1bn Indian core-banking upgrade spend over 5–10 years	BCG via Business Standard, 2024 (`01 §3`)	Medium — analyst estimate, single outlet
92 of top 100 banks on IBM mainframes	Widely reported industry figure (`01 §1`)	Medium
>40% of banking systems still run COBOL	Pragmatic Coders, 2025 (`01 §1`)	Medium
5% vs 7–9% IT spend as share of revenue, Indian vs global banks	"Industry analysis" (`01 §3`)	**Weakest citation on the deck** — no primary source pinned. See open item below.
40,000+ RBI ombudsman mobile/internet banking complaints, FY22–23	RBI ombudsman data (`01 §3`)	Good — regulator's own numbers. **Currently unused on the deck**; available as a swap-in.

Figures deliberately **not** used, per `01` : the 800bn-lines-of-COBOL estimate and the 95%-of-ATM-swipes claim — both **[contested]**.

The positioning argument

The deck's market slide makes a two-part argument:

1. **The spend is already committed** — modernisation budgets exist (the $1bn figure); the constraint is not demand.
2. **Every rupee of it is gated by one step no one sells as a product** — the assurance step between "the new code exists" and "we run it on real money" has no product category; it is absorbed as consulting hours and schedule risk.

Part 1 is sourced. Part 2 is an **argument, not a fact** — it is the team's synthesis from the competitive landscape (`03` , `07`): AWS Transform, IBM watsonx Code Assistant for Z and Mechanical Orchard's Imogen all *generate* and then validate as a feature of generation; none is sold as a standalone assurance product. Locksmith (arXiv 2607.28271) is research, not product. The claim survives the check in `07` but should be voiced as positioning ("no one *sells* the proof as the product"), never as a market statistic.

Adjacent-product facts used

Claim	Source	Confidence
AWS Transform for mainframe GA May 2025	AWS announcement (`01 §5`)	Good
ADP: rule-extraction time −80%, manual effort −90%	AWS/ADP case material (`01 §5`)	Medium — vendor case study; attribute to AWS when quoting
IBM's own papers: translated code "cannot be trusted" without checking	IBM Research, WCA4Z testing paper (`01 §5` , quotes in `citations …/05`)	Good — it is IBM about IBM

Open item — pin the IT-spend figure

The 5%-vs-7–9% comparison circulates through consulting and analyst commentary (Gartner / Celent / McKinsey banking-IT benchmarks are the likely origins). Before the final round, either pin it to a named report and year, or replace the stat tile with the RBI ombudsman figure, which is regulator-sourced and currently

unused.

---

3. Investments

What the round-2 deck claims it takes to build, and the standing of each claim.

Claim	Basis	Status
Licence cost: nil	GnuCOBOL GPL/LGPL, ProLeap MIT, gcov GPL, NIST CCVS85 public domain, AWS CardDemo Apache 2.0 (`04 §1–4`)	Sourced, per component
No mainframe, no vendor contract required	GnuCOBOL compiles to native binaries on a laptop (`04 §1`)	Sourced
Prototype: 4 engineers × 12 weeks	Sized from the staged plan in `04 §7` — stages 0–7 at student pace	**[ASSUMPTION]** — effort estimate, no external basis
Only variable cost is LLM inference	One bounded call per slice (`04 §2`)	Architecture is sourced; "only" is fair given nil licences
Deterministic orchestration ≈ up to 3.5× lower token cost than agentic	Lwin & Kumar, arXiv 2605.09894 (`04 §6`)	Good — controlled study
Bank pilot: 2 engineers + bank SME time, one program family, ~6 months	Team's own sizing	**[ASSUMPTION]**

Deliberate choice: no rupee totals appear on the deck because no costing research exists. The slide says "indicative and effort-based" in so many words. If a money figure is ever wanted: (engineer-months × assumed loaded monthly rate) is the only honest construction, and the rate assumption must be stated on the slide.

---

4. Returns — and the cost of not solving

The template asks to "quantify the benefits & what if I don't solve?". The evidence, all previously gathered:

Upside (what assurance unlocks)

- The ~$1bn committed spend is gated on assurance (argument, §2 above).
- Comparable automation of one pipeline stage: ADP −80% extraction time, −90% manual effort (`01 §5`).
- Review-cost mechanism: line-cited rules mean an engineer confirms a claim by reading three lines instead of four thousand (architecture property, `02`).

Downside (what failure costs)

- **TSB 2018** (`01 §4`): £48.65m FCA/PRA fine, £32.7m redress, ~£400m total, CEO resigned, CIO personally fined £81,620. The regulator's finding was about *risk management and assurance*, not programming — which is exactly the gap this project fills. Best-documented case; keep it as the centrepiece.
- **HDFC 2020** (`01 §3`): RBI froze Digital 2.0 launches and new card issuance; restrictions ran into 2022. The India-specific proof that an IT failure freezes a product roadmap.

Not claimed: any specific ROI multiple, payback period, or revenue figure. Nothing in the research supports one, so the deck argues risk-avoidance and unlock instead. If a judge demands an ROI number, the honest answer is that the counterfactual (TSB's ~£400m) prices the downside, and the product's cost is two orders of magnitude below it.

---

5. Timelines

The deck's week-by-week plan is a repackaging of the staged build plan in `04 §7`. The stage contents and risk ratings are researched; **the calendar mapping is the team's own [ASSUMPTION]**.

Deck line	`04 §7` stage	Notes
Weeks 1–2 — differential harness runs and diffs	Stages 0–1	Low risk; already a working demo
Weeks 3–4 — input generation from PIC clauses + hand boundaries	Stage 2	Low
Weeks 5–7 — parse, slice, extract rules with line citations	Stage 3	Medium (Java/Python boundary)
Weeks 8–10 — reciprocal loop + per-rule verdicts	Stages 4–5	**The contribution — protected time**
Weeks 11–12 — delta debugging, evidence bundle	Stages 6–7	Low risk, high demo value
Months 4–9 — bank pilot, one program family, bank's own compiler	—	**[ASSUMPTION]** — no counterpart in `04` ; sized by the team

The highlighted weeks 8–10 band carries the novelty claim as qualified by `07` : what survived falsification is the discrimination condition, the *refuted* verdict, and the regulatory third reference — not the differential spine itself, which is prior art (Locksmith, WCA4Z validation, Blu Age Compare).

---

6. Template category chips

The CII template's category tags, and why each is lit or not on the deck:

Chip	Lit?	Rationale
Modernization	Yes	The category of the product
AI	Yes	Bounded LLM calls for extraction and translation
Process Re-engg	Yes	Replaces manual assurance (consulting hours) with a pipeline
Cloud	No	Runs on a laptop; no cloud dependency is claimed. Could deploy to cloud, but claiming it adds nothing and invites questions
ROI	Yes	Risk-avoidance case (TSB counterfactual)
Margin improvement	Yes	Assurance absorbed today as consulting hours and schedule risk
Happy Customer / Revenue Growth	No	No evidence chain supports either; claiming them would dilute the honest ones
Time to solve	Yes	12-week prototype plan
Milestones	Yes	Staged plan with named deliverables
Tools	Yes	GnuCOBOL, ProLeap, gcov, delta debugging

---

7. Summary of what is new versus `01` – `07`

New syntheses introduced for round 2 (nowhere else in the research):

1. The four-stakeholder pain mapping (§1).
2. The "assurance gate" market positioning (§2).
3. Effort-based investment estimates (§3).
4. The calendar mapping of the staged plan (§5).
5. The chip rationale (§6).

Everything else on the round-2 slides is a restatement of `01` – `07` material.

Open items

- [ ] Pin the 5%-vs-7–9% IT-spend figure to a named report, or swap in the RBI ombudsman figure (§2).
- [ ] Validate the stakeholder pain statements with one practitioner (§1).
- [ ] Decide whether a rupee costing is wanted for the final round; if so, agree a loaded monthly rate and label it as an assumption (§3).

Legacy Banking Modernisation Platform

Round 2 edition — proving the rewrite behaves exactly like the original

Team StudyEdge · Odisha University of Technology and Research, Bhubaneswar
CII Eastern Region Hackathon · Theme: Banking and Finance Technology
Submission: 13 August 2026 · Finals: ICT East, Kolkata, 24 September 2026

Supersedes the round-1 report (4 August 2026). New in this edition: target users and stakeholders, technical details, market potential, and the business-plan sections (investments, returns, timelines) required by the CII template. Claims sourced in `Research Work/01–08` *; team-authored estimates are marked as assumptions where they occur.*

The one-line thesis

Legacy modernisation fails on verification, not translation. We make behavioural equivalence something a bank can actually prove — and then check that answer against the regulator.

Part 1 — The problem

The situation

Indian banks, insurers and public-sector institutions run their core operations on COBOL programs written in the 1980s. 92 of the world's top 100 banks run core operations on IBM mainframes; more than 40% of banking systems still run COBOL. Everyone involved knows this is a liability. Almost nobody has managed to leave.

Take one program — the engine that calculates interest on savings accounts. Four thousand lines. Written in 1987. Modified roughly two hundred times since. Every modification was a response to something real: a regulatory change, a leap-year bug, a rounding complaint, a fix for accounts opened on the 31st. None of it was documented. The engineers who made those changes have retired.

The misdiagnosis

The common assumption is that modernisation is hard because translating COBOL into Java is hard. It isn't. Commercial transpilers have existed for decades; AWS Transform for mainframe went generally available in May 2025; refactoring platforms are a mature product category. Translation is the solved half. It was never the half anyone was stuck on.

The real problem

Where is the document that states what the program is supposed to do? There isn't one. It was never written. **The code is the specification.** The only ways to learn a business rule are to read four thousand undocumented lines, to run the program and observe it, or to ask someone who retired in 2009.

Why nobody risks it

The rewrite compiles. It passes the tests the team wrote. Should the bank deploy it? Nobody can answer, because nobody knows what the old program does in full. And the tolerance for error is zero:

Failure mode	Consequence
Rounding drift	Silent, compounding, discovered at audit
Date boundary handling	Loans mature on the wrong day
Threshold logic	Wrong rate applied to an entire tier

A discrepancy of one paisa across millions of accounts is a reportable regulatory event. The cost of getting this wrong is not hypothetical:

- **TSB, United Kingdom, 2018.** A core migration failed at cutover. The FCA and PRA fined TSB £48.65m for operational risk and governance failings; £32.7m went in customer redress; total direct cost was around £400m. The CEO resigned and the former CIO was personally fined £81,620. The regulator's finding was about *risk management and assurance* — not about programming.
- **HDFC Bank, India, 2020.** After repeated outages, RBI halted every Digital 2.0 launch and froze new credit-card issuance. The card freeze lifted only in August 2021; the digital freeze ran into 2022. In India, an IT failure in a bank is a supervisory event that freezes the product roadmap.

So the institution keeps running software it cannot safely change — which is the correct decision under its own risk framework. Multiply it across thousands of institutions and the backlog is measured in decades.

The bottleneck is verification, not code generation.

Part 2 — Target users and stakeholders

Stakeholder	Role	Pain the platform addresses
Bank CIO / head of core banking	Owns the migration decision and the cutover date	Cannot sign off a replacement nobody can prove behaves identically
Risk, compliance and internal audit	Owns the board-approved Change Management Policy required by RBI's IT Governance Master Direction (2023)	No auditable artefact showing old and new agree, case by case
Modernisation vendors and system integrators	Cognizant, TCS, Infosys and in-house delivery teams	Translation is commoditised; assurance is what stretches the programme
The supervisor	RBI, and the bank's own board	An IT failure is a supervisory event, not an engineering incident

Nobody on this list is short of a translator. Everybody on this list is short of proof. The buyer is the person who has to sign the cutover memo — and today has nothing to attach to it.

(Roles are sourced; the pain statements are the team's inference and are recorded as such in `08 §1`*.)*

Part 3 — The solution

Why the obvious approach fails

Point a language model at the COBOL and ask it to explain the business rules, and the output looks excellent — clean, confident, well organised. And some percentage of it is wrong. A confidently stated wrong rule is more dangerous than no rule at all, because an engineer will build on it. Extraction cannot be the product.
Verification has to be the product.

The pipeline

A deterministic pipeline with the AI on a short leash — the LLM is called at two bounded points inside a fixed sequence, never as a free agent:

0. **Intake gate.** Programs that read clocks, databases or CICS screens are detected and *refused with a reason* — differential testing is meaningless on non-deterministic programs, so the tool says no rather than mishandling them.
1. **Parse and slice.** ProLeap builds the AST and semantic graph; program slicing on business-concept variables produces bounded units.
2. **Extract rules** — one bounded LLM call per slice, with line citations. The slice defines the line set, so the model cannot invent a citation.
3. **Rule-directed input generation.** The extracted rules say where the boundaries are: ₹99,999 / ₹1,00,000 / ₹1,00,001 (RBI's uniform-rate threshold), accruals landing on exact ₹0.50 (the nearest-rupee rounding midpoint), 28/29 February, month and quarter ends, zero, negative and maximum values.
4. **Execute and compare — three ways.** The same input goes through the legacy COBOL (compiled via GnuCOBOL — the oracle), the generated Java (untrusted by design), and a clause-cited model of the RBI Master Direction. Outputs are diffed exactly.

The key idea: the legacy system is its own oracle

Testing normally requires knowing the correct answer in advance. Here that requirement disappears, because the program being replaced already defines the correct answer — by running. Every mismatch becomes a concrete, reproducible finding rather than a confidence score:

> *Given a balance of 9,999.995 on a leap-year accrual, the legacy program returns 1042.50 and the reimplementation returns 1042.51.*

The reciprocal loop

Extraction and verification strengthen each other. If the rules say balances below ₹1 lakh earn a uniform rate, the input generator now knows where the boundary is and tests either side of it. The comprehension layer makes the testing sharper; the testing layer catches the comprehension layer lying. Neither works alone.

Honesty as a feature

Every extracted rule ends in one of three verdicts:

- **Verified** — the corpus exercised it and all sides agreed on every case.
- **Unproven** — extracted but never exercised; flagged, not silently trusted.

- **Refuted** — the execution contradicted the stated rule; the extraction layer was caught being wrong.

Coverage is reported rather than assumed. The same honesty applies at intake: the tool tells you when it cannot help you.

The regulatory third reference

The RBI Master Direction (Interest Rate on Deposits), 2016 — updated 2024 — fixes the arithmetic in writing: daily product basis (clause 6(a)); a uniform rate up to ₹1 lakh (6(a)(1)–(2)); interest rounded to the nearest rupee for rupee deposits (4(f)), with 50 paise and above rounding up. Checking both programs against the regulator's own arithmetic means the platform can catch the case where the legacy itself is non-compliant — something a pure old-vs-new diff can never see.

Part 4 — Technical details

Layer	Component	Why it is there
Legacy execution	**GnuCOBOL** (cobc/libcob, GPL/LGPL)	19 COBOL dialects; `-std=ibm-strict` against Enterprise COBOL 6.3; compiles to native binaries that can be run, timed and instrumented. Passes 9,700 of 9,748 NIST CCVS85 tests — while the project itself claims no formal conformance, a caveat we state ourselves.
Parsing	**ProLeap COBOL parser** (ANTLR4, MIT)	AST plus abstract semantic graph with data and control flow; handles COPY/REPLACE; extracts `EXEC SQL` / `EXEC CICS` as text, which is what makes the intake gate's refusals detectable rather than accidental.
Extraction	Program slicing + bounded LLM calls	A slice is a set of source lines — exactly the traceability promised. One call per slice bounds context and makes line citations mechanical.
Coverage	**gcov** (via GnuCOBOL's C output)	Exact statement execution counts; fallback is a paragraph-level trace via `-fgen-c-line-directives`. This is how the verified/unproven verdict is computed.
Comparison	Differential runner + **delta debugging**	Same input, three outputs, exact diff; delta debugging shrinks a pile of failing inputs to one minimal counterexample.
Corpora	**AWS CardDemo** (Apache 2.0), **NIST CCVS85** (public domain), own RBI-modelled fixture	Third-party COBOL avoids the circularity of testing our tool on code we wrote. CardDemo's interest calculator reads VSAM files and the clock, so the intake gate correctly refuses it — which is itself a demo.

Integration: CLI plus a JSON evidence bundle. The methodology is compiler-agnostic — swap GnuCOBOL for IBM Enterprise COBOL inside the bank and the harness is unchanged. No mainframe access is required for the prototype.

Why these technologies: everything is free and citable, so the prototype runs end-to-end on a laptop. Deterministic orchestration with bounded LLM calls matches agentic accuracy at up to 3.5× lower token cost, with lower run-to-run variance (arXiv 2605.09894) — and a harness that compares programs for exact equality cannot itself be non-deterministic.

Part 5 — Market potential

The spend is already committed. BCG estimates Indian lenders must invest roughly **$1bn over 5–10 years** to upgrade legacy core banking systems. Indian banks spend up to 5% of revenue on IT against 7–9% for global peers — a gap that compounds into more accumulated undocumented logic. RBI's ombudsman recorded over 40,000 mobile and internet banking complaints across FY22–23; the supervisor is already acting on IT fragility.

Every rupee of that spend is gated by one step. What stalls approved modernisation budgets is the assurance step between "the new code exists" and "we are willing to run it on real money". That step has no product category today — it is absorbed as consulting hours and schedule risk.

The category is forming, and it is forming around generation. AWS Transform for mainframe (GA May 2025; ADP cut rule-extraction time by 80% and manual effort by over 90%), IBM watsonx Code Assistant for Z (whose validation assistant exists because IBM's own papers say translated code cannot be trusted unchecked), and Mechanical Orchard's Imogen (a generate-validate loop) all *generate first* and validate as a feature. None of them sells the proof as the product.

(The gating claim is positioning — the team's synthesis from the competitive landscape — not a market statistic; recorded as such in `08 §2`.)

Part 6 — What makes it different

Stated with the verification pass (`07`) in view, so nothing here overclaims:

1. **The intake gate refuses what it cannot verify soundly** — with a reason, on detection. The honesty argument applied before any analysis begins.
2. **The reciprocal loop with per-rule verdicts.** Shipping tools treat extraction and verification as separate phases; the academic literature splits the same way. Closing the loop — rules direct the inputs, execution then judges the rules, coverage is reported per business rule as verified / unproven / **refuted** — is the part that is genuinely thin in both products and papers.
3. **The regulatory third reference.** Diffing old against new proves equivalence; diffing both against the RBI's written arithmetic can prove the legacy itself wrong. No adjacent product does this.

The differential mechanism itself is prior art — IBM, AWS and Mechanical Orchard ship it, and Locksmith (arXiv 2607.28271) builds the same spine off-mainframe. That convergence validates the problem; the three items above are the contribution.

Part 7 — Business plan

Investments — what it takes

Item	Detail
Licence cost	**Nil.** GnuCOBOL (GPL/LGPL), ProLeap (MIT), gcov, NIST CCVS85 and AWS CardDemo are all open. No mainframe, no vendor contract.
Prototype	4 engineers × 12 weeks on commodity laptops *(effort-based estimate)*
Variable cost	LLM inference only — one bounded call per slice; deterministic orchestration at up to 3.5× lower token cost than agentic
Bank pilot	2 engineers plus bank SME time, one program family, ~6 months *(estimate)*

Returns — and what if I don't solve?

- **Unlocks:** the ~$1bn gated modernisation spend; ADP-comparable automation (−80% extraction time, −90% manual effort) on the extraction stage; review cost per rule drops from four thousand lines to three.
- **The counterfactual:** TSB 2018 — ~£400m, a resigned CEO and a personally fined CIO — for want of pre-cutover assurance. In India, HDFC 2020 shows the same failure freezing a roadmap for over a year.
- No specific ROI multiple is claimed; the counterfactual prices the downside at two orders of magnitude above the platform's cost.

Timelines — time to realise benefit

When	Deliverable
Weeks 1–2	Differential harness runs two programs on an input list and diffs — already a working demo
Weeks 3–4	Input generation from PIC clauses plus hand-written boundaries
Weeks 5–7	Parse, slice, extract rules with line citations
Weeks 8–10	**The reciprocal loop and per-rule coverage verdicts — the contribution**
Weeks 11–12	Delta debugging to minimal counterexamples; evidence bundle
Months 4–9	Bank pilot on one program family, inside the bank's own compiler environment

(Stage contents and risks researched in `04 §7`; the calendar mapping is the team's own sizing.)

Part 8 — Scope and feasibility

In scope: COBOL compiled and executed via GnuCOBOL; retail banking interest accrual; pure computation — input in, output out; third-party COBOL including AWS's own CardDemo sample.

Out of scope, detected rather than ignored: whole-system migration; CICS screens and JCL orchestration; database-coupled programs; anything non-deterministic (clocks, randomness, external state). Differential testing compares outputs for equality, so non-determinism makes the verdict meaningless — the tool refuses these at intake, with a reason.

Known limitation: input-generation quality caps everything. Naive random inputs will not find a rounding bug. That is precisely why the reciprocal loop is where the build effort goes.

Part 9 — Outcome

The institution keeps four artifacts:

1. **A specification** — readable, traceable documentation for a system that never had any.
2. **A regression suite** — thousands of input–output pairs, retained permanently.
3. **An equivalence claim** — evidence-backed, with coverage stated honestly, and the artifact someone can actually sign.
4. **Compliance findings** — the legacy checked against RBI's written arithmetic, not only against itself.

The first two hold their value even if the migration is never approved.

The pitch in one sentence

The industry's most serious players converged on the same bottleneck we did — verification — and every one of them sells generation with validation bolted on. We build the proof as the product, report the coverage we do not have, and check both programs against the regulator.

Sources: `Research Work/06 - Sources.md` *and* `Research Work/citations and research paper/` *(references.bib, annotated bibliographies, verified quotes). Evidence and assumptions for the business-plan sections:* `Research Work/08 - Business Plan Research.md`.

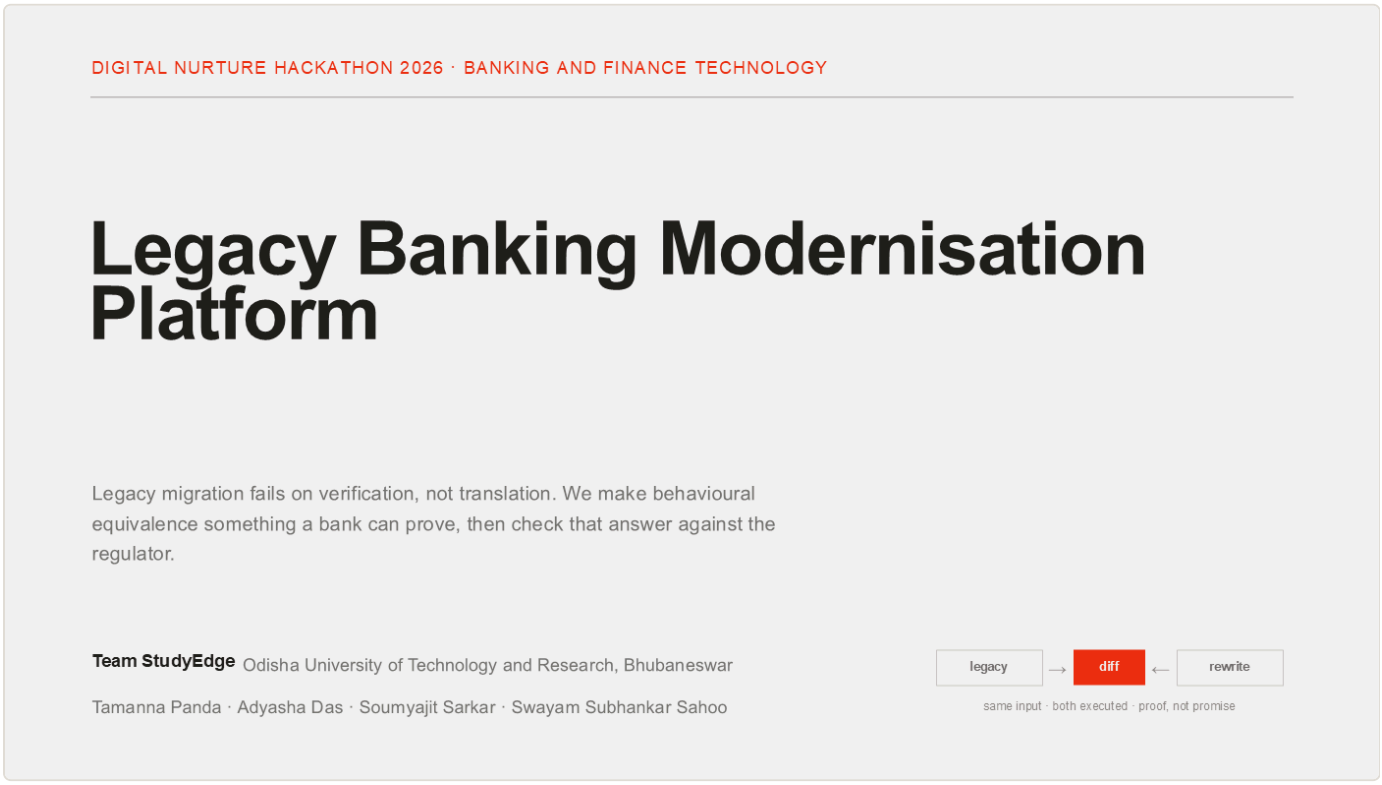
The Round-2 Deck

The thirteen slides of `StudyEdge_Banking and Finance Technology_V2.pptx` as submitted for round 2, rendered to PNG through PowerPoint itself — so these are the slides exactly as an evaluator's machine will draw them.

Slides 4, 7, 9, 11 and 12 are the round-2 additions mandated by the CII template (stakeholders, technical details, market potential, and the two business-plan pages). Every figure on them traces to `01` – `08`; the two business-plan pages carry the template's WHY / HOW / WHAT and Investments / Returns / Timelines structure.

---

1 · Title



2 · Problem

PROBLEM

India’s banks run on code nobody can safely change

92
of the top 100 banks worldwide run on IBM mainframes

40%+
of banking systems still run COBOL written decades ago

$1bn
over 5–10 years to upgrade Indian core banking systems (BCG)

0
specifications state what those programs should do

When a bank guesses wrong, it is a supervisory event, not a bug report

TSB, United Kingdom — 2018

A core migration failed at cutover. The FCA and PRA fined TSB £48.65m for operational risk and governance failings, and £32.7m went in customer redress. Business as usual took eight months to return.

HDFC Bank, India — 2020

After repeated outages the RBI halted every Digital 2.0 launch and froze new credit-card issuance. The card freeze lifted only in August 2021; the digital freeze ran into 2022.

Why the backlog is decades long

A one-paisa divergence across millions of accounts is a reportable regulatory event. Refusing to touch the system is the correct decision under the bank’s own risk framework.

1987
core interest engine written

2018
TSB — £400m failed cutover

2020
HDFC — RBI freezes the roadmap

2026
migration still deferred

02

3 · Why it's unsolved

WHY IT’S UNSOLVED

Translation is solved. Proof is not.

COBOL source → **Translate** → **Modern Java** → **Verification** → **Production**

4,000 lines · 1987 ~200 undocumented edits

transpilers · LLMs AWS · IBM · TCS ship this

compiles · passes the tests you wrote

Can anyone prove the new program behaves identically to the old one?

never reached

No specification was ever written — the rules live only inside the code. Asking an AI to explain them looks excellent, and a meaningful share of the output is wrong:

62%
precision of the best published COBOL rule extractor
A-COBREX, ICSE 2025

91.2%
of business rules successfully extracted by a 2026 system
AgentModernize, arXiv 2605.17535

<20%
behavioural equivalence reached by that same system
baselines scored 0.0%

Extraction works. Behavioural equivalence does not follow from it. Verification has to be the product.

03

4 · Target users & stakeholders — round-2 addition

TARGET USERS & STAKEHOLDERS

Who is stuck, and what they are stuck on

Bank CIO / core banking head	Owns the migration decision and the cutover date.	**Pain:** Cannot sign off a replacement nobody can prove behaves identically.
Risk, compliance and internal audit	Owns the board-approved Change Management Policy required by RBI's IT Governance Direction (2023).	**Pain:** No auditable artefact showing old and new agree, case by case.
Modernisation vendors and system integrators	Cognizant, TCS, Infosys and in-house delivery teams.	**Pain:** Translation is already commoditised; assurance is what stretches the programme.
The supervisor	RBI, and the bank's own board.	**Pain:** An IT failure is a supervisory event, not an engineering incident.

Nobody on this list is short of a translator. Everybody on this list is short of proof.

Our buyer is the person who has to sign the cutover memo — and today has nothing to attach to it.

04

5 · Our solution

OUR SOLUTION

A deterministic pipeline, with the AI on a short leash

0	**Intake gate**	clock / DB / CICS detected? REFUSED — non-deterministic, cannot be verified soundly	DETERMINISTIC
1	**Parse & slice**	ProLeap AST → rule slices	DETERMINISTIC
2	**Extract rules**	with line citations from the slice	BOUNDED LLM CALL
3	**Rule-directed input generation**	the extracted rules say where the boundaries are — ₹99,999 / ₹1,00,000 / ₹1,00,001 · ₹0.50 rounding midpoints · 29 February · month and quarter ends · zero and maximum values	DETERMINISTIC
4	**Execute and compare — three ways**	same input, three outputs: legacy COBOL (GnuCOBOL — the oracle) · reimplementation (generated Java, untrusted) · RBI Direction (the third reference)	DETERMINISTIC

Rule verdicts
proven · unproven · refuted

Mutation score
how strong is this claim?

Compliance findings
legacy vs the regulator

Evidence bundle
spec · tests · signed claim

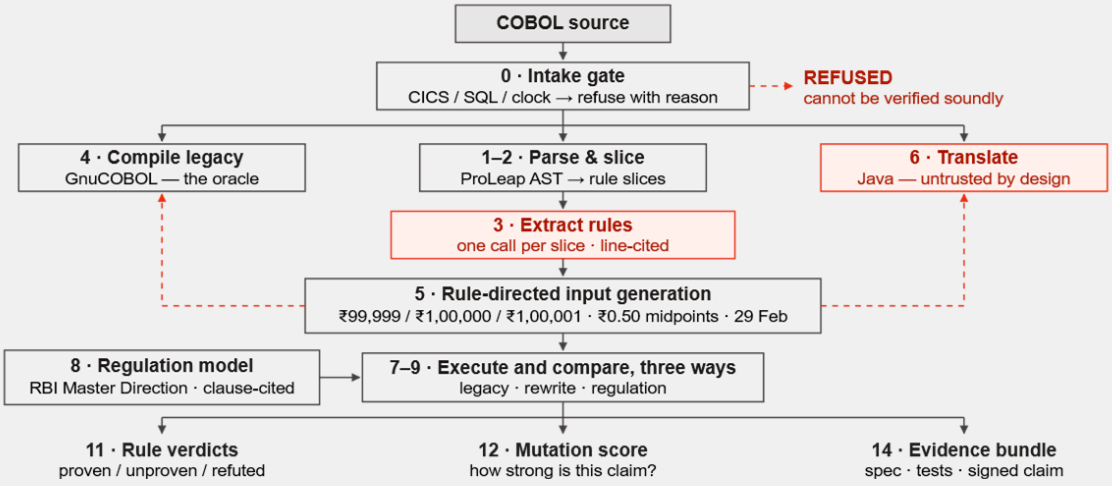
05

6 · How it works

HOW IT WORKS

The pipeline, end to end

DETERMINISTIC · BOUNDED LLM CALL · dashed — the reciprocal loop: rules pick the inputs, the run checks the rules



06

7 · Technical details — round-2 addition

TECHNICAL DETAILS

The stack, and why each piece is there

Legacy execution

GnuCOBOL
cobc / libcob, GPL and LGPL. 19 COBOL dialects, -std=ibm-strict against Enterprise COBOL 6.3.

Conformance
passes 9,700 of 9,748 NIST CCVS85 tests. The project does not claim standard conformance — we state that ourselves.

Why
compiles COBOL to C to a native binary, so the legacy side is a real executable we can run, time and instrument.

Analysis and extraction

ProLeap COBOL parser
ANTLR4, MIT licence. Produces AST plus an abstract semantic graph with data and control flow.

Program slicing
on business-concept variables. A slice is a set of source lines — which is exactly the traceability we promise.

Bounded LLM call
one call per slice, never over the whole file. The model cannot invent a line reference the slice does not contain.

Verification and coverage

gcov
statement execution counts, via -fprofile-arcs. Fallback: paragraph-level DISPLAY trace with -fgen-c-line-directives.

Differential runner
same input, three outputs — legacy COBOL, generated Java, and the RBI Direction as a third reference.

Delta debugging
shrinks 73 failing inputs to one minimal counterexample.

Data sources

AWS mainframe-modernisation CardDemo corpus (Apache 2.0) · NIST CCVS85, 512 programs and 8,800+ cases, public domain · RBI Master Direction clauses encoded as the third reference.

Integration and interoperability

CLI plus a JSON evidence bundle. The methodology is compiler-agnostic — swap GnuCOBOL for IBM Enterprise COBOL inside the bank and the harness is unchanged. No mainframe access required.

07

8 · What makes it different

WHAT MAKES IT DIFFERENT

Three things nobody else does

The differential mechanism itself is prior art. IBM, AWS, Mechanical Orchard and a paper published five days ago all ship a version of it, and we say so. These three are ours.

01 We check the legacy against the regulator

Every tool treats the legacy as correct by definition. A program written in 1987 can breach a Direction issued decades later, so we add a third reference.

L = R = Reg	verified compliant
L ≠ R	translation defect
L = R ≠ Reg	inherited breach
L ≠ R = Reg	silent correction

Only the third row needs a regulator. No other tool can see it.

02 A result for every rule, not every test case

Executing a line is not testing the rule it encodes. A rule counts as confirmed only when all three checks hold.

1. **Reached** — the rule's lines ran
2. **Discriminating** — the boundary changed the output
3. **Agreed** — the rewrite matched every time

Check 2 is the one that catches our own extraction inventing a rule that is not in the program.

03 We test our own tests

"Everything passed" proves nothing unless the inputs could have caught a failure. So we break the legacy on purpose.

Inject a deliberate defect 100000 → 100001 · > → >= · delete ROUNDED · 360 → 365

Re-run the same input corpus

Caught the claim stands

Missed we do not claim it

The equivalence claim ships with its own strength score.

9 · Market potential — round-2 addition

MARKET POTENTIAL

The spend is already committed. Assurance is the gate.

~$1 bn

what Indian lenders must invest over 5–10 years to upgrade legacy core banking systems

Boston Consulting Group, 2024

92 of 100

of the world's largest banks run core operations on IBM mainframes

widely reported industry figure

> 40%

of banking systems still run COBOL in production today

Pragmatic Coders, 2025

5% vs 7–9%

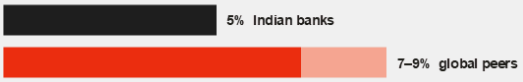
Indian banks' IT spend as a share of revenue, against global peers

industry analysis

Every rupee of it is gated by one step

Modernisation budgets are approved. What stalls them is the assurance step between "the new code exists" and "we are willing to run it on real money". That step has no product category yet — it is absorbed as consulting hours and schedule risk.

IT spend as a share of revenue



The gap compounds: less budget → fewer modernisation attempts → more undocumented logic.

The category is forming around generation

- **AWS Transform for mainframe** generally available May 2025. ADP used it to cut rule-extraction time by 80% and manual effort by over 90%.
- **IBM watsonx Code Assistant for Z** ships with a validation assistant — IBM's own papers say the translated code cannot be trusted without checking.
- **Mechanical Orchard, Imogen** a generate-then-validate loop.

All of them generate. None of them sells the proof as the product.

10 · Feasibility & scope

FEASIBILITY & SCOPE

Narrow by design, and buildable now

In scope

COBOL compiled and executed via GnuCOBOL — free, open source, runs on a laptop Retail banking interest accrual — pure computation: input in, output out Third-party COBOL, including AWS's own CardDemo sample

Out of scope — detected, not ignored

CICS screens and JCL orchestration Database-coupled programs Anything non-deterministic — system clocks, randomness, external state

Differential testing compares outputs for equality, so non-determinism makes the verdict meaningless. The tool refuses these at intake, with a reason.

Build stages — the spine first, the novelty protected

01 · Differential harness

GnuCOBOL compiles and runs; diff two implementations across an input list.

02 · Rules and boundaries

Slice, extract with line citations, derive boundary inputs from the rules.

03 · Per-rule results

Coverage mapped back to rules, including the ones we could not confirm.

04 · Regulation and mutation

The RBI reference model, then fault injection to score the corpus.

11 · Business plan (1/2) — round-2 addition

BUSINESS PLAN (1/2)

Legacy Banking Modernisation Platform

WHY

Explain the problem

Problem & business scenario
92 of the top 100 banks run on mainframes and over 40% of banking systems still run COBOL. Indian lenders face roughly $1bn of core upgrades over 5–10 years (BCG). The business rules were never documented anywhere except the code.

Problem scope
COBOL, pure-computation programs, retail interest accrual. Whole-system migration, CICS and JCL, database-coupled programs and clock or random reads are excluded — and detected and refused at intake rather than silently mishandled.

Target users & stakeholders
Bank CIO and core-banking head, risk and internal audit, modernisation SIs, and the supervisor. All of them can already generate the new code. None of them can prove it is safe.

→ HOW

Explain the solve

Solution overview
Intake gate → parse and slice → bounded LLM rule extraction with line citations → rule-directed input generation → execute three ways → diff. Outputs: rule verdicts, mutation score, compliance findings, evidence bundle.

Technical details
GnuCOBOL (19 dialects, 9,700/9,748 NIST), ProLeap ANTLR4 parser, program slicing, gcov coverage, delta debugging. Deterministic orchestration with the LLM confined to bounded calls.

Innovation
The legacy binary is its own oracle. The reciprocal loop — rules imply boundaries; execution then judges the rules. A third reference, the RBI Direction, checks the legacy against the regulator too. Verdicts are verified, unproven or refuted — never a confidence score.

Why these technologies
GnuCOBOL is free and NIST-validated, so both sides execute for real on a laptop with no mainframe. Deterministic orchestration matches agentic accuracy at up to 3.5× lower token cost.

→ WHAT

Value proposition

A specification that never existed
Plain-language rules, each traced to the exact lines that produce it. Review cost drops from reading four thousand lines to reading three.

A permanent regression suite
Thousands of input–output pairs retained, reusable on every future change to the same program.

An equivalence claim someone can sign
Evidence-backed, with the coverage stated honestly — verified, unproven, or refuted per rule.

Compliance findings
The legacy checked against RBI's written arithmetic, not only against itself.

Value even if the migration is cancelled
The spec and the test suite are retained regardless. The bank ends up with documentation for a system it previously could not touch.

12 · Business plan (2/2) — round-2 addition

BUSINESS PLAN (2/2)

Financials & timelines

INVESTMENTS

What it takes, and what it costs

- **Licence cost: nil**
  GnuCOBOL (GPL/LGPL), ProLeap (MIT), gcov, NIST CCVS85 and AWS CardDemo are all open. No mainframe, no vendor contract.
- **Prototype**
  4 engineers × 12 weeks. Runs on commodity laptops.
- **Only variable cost is inference**
  One bounded LLM call per slice. Deterministic orchestration measured at up to 3.5× lower token cost than an agentic loop.
- **Bank pilot**
  2 engineers plus SME time from the bank, one program family, about 6 months.

Indicative and effort-based. No licence or infrastructure spend is assumed because none is required.

RETURNS

Quantified — and the cost of not solving

What it unlocks

The ~$1bn of Indian core-banking upgrade spend is gated on assurance. Comparable automation (ADP via AWS Transform) cut rule-extraction time 80% and manual effort over 90%.

What if I don't solve — TSB, 2018

1.9m of 5.2m customers hit. £48.65m FCA/PRA fine. £32.7m redress. ~£400m total cost. CEO resigned; the CIO was personally fined £81,620. The regulator's finding was about risk management and assurance — not about programming.

And in India

RBI halted HDFC Bank's Digital 2.0 launches and new card issuance in December 2020; the restrictions ran into 2022. An IT failure here is a supervisory event that freezes the product roadmap.

TIMELINES

Time to realise the benefit

- **Weeks 1–2**
  Harness runs two programs on an input list and diffs. Already a working demo.
- **Weeks 3–4**
  Input generation from PIC clauses plus hand-written boundaries.
- **Weeks 5–7**
  Parse, slice, and extract rules with line citations.
- **Weeks 8–10**
  The reciprocal loop and per-rule coverage verdicts. This is the contribution.
- **Weeks 11–12**
  Delta debugging to minimal counterexamples; evidence bundle generated.
- **Months 4–9**
  Bank pilot on one program family, inside the bank's own compiler environment.

Modernization · AI · Process Re-engg · Cloud · ROI · Margin improvement · Happy Customer · Revenue Growth · Time to solve · Milestones · Tools

12

13 · References

REFERENCES

Sources

PEER-REVIEWED AND PREPRINTS

Hans, S. et al. (2025). Automated Testing of COBOL to Java Transformation. arXiv:2504.10548.

Pan, R. et al. (2024). Lost in Translation: A Study of Bugs Introduced by LLMs while Translating Code. ICSE 2024, arXiv:2308.03109.

Ahmed, S. N. & Galib, M. (2026). AgentModernize: Preserving Business Logic in Legacy Modernization. arXiv:2605.17535.

Ferenczi, A. et al. (2026). Agentic Method for Deterministic Validation of Legacy Code Migration. arXiv:2607.28271.

Shah, S. et al. (2025). A-COBREX: Identifying Business Rules in COBOL Programs. ICSE 2025, Demonstrations.

B S, C. & Chimalakonda, S. (2025). LLM vs Rule-Based: the COBRAIN Tool. EASE 2025, doi:10.1145/3756681.3756982.

Lwin, N. O. & Kumar, R. (2026). Deterministic vs LLM-Controlled Orchestration for COBOL-to-Python Modernization. arXiv:2605.09894.

Barr, E. T. et al. (2015). The Oracle Problem in Software Testing: A Survey. IEEE TSE 41(5), 507–525.

REGULATORY AND SUPERVISORY

Reserve Bank of India (2016, upd. 2024). Master Direction — Interest Rate on Deposits. Clauses 4(f), 6(a), 6(a)(1)–(2).

Reserve Bank of India (2023). Master Direction on IT Governance, Risk, Controls and Assurance Practices.

Financial Conduct Authority (2022). TSB fined £48.65m for operational resilience failings.

INDUSTRY AND MARKET

Boston Consulting Group (2024). Cloud-based Core Transformations, via Business Standard.

IBM. watsonx Code Assistant for Z — Validation Assistant documentation.

Amazon Web Services (2025). AWS Transform for mainframe, generally available. **Mechanical Orchard.** Imogen platform — generate-validate loop.

TOOLS AND STANDARDS

GnuCOBOL Project. GnuCOBOL compiler, GPL/LGPL. NIST COBOL-85: 9,700 of 9,748. **Wolf, U.** ProLeap COBOL Parser, MIT licence. **NIST (1992).** CCVS85 validation system.

13

06 — Sources

Confidence key: **A** — primary source (regulator, standards body, peer-reviewed paper, vendor documentation about its own product). **B** — reputable secondary reporting. **C** — vendor marketing or content-marketing blog; directionally useful, numerically soft.

Regulation (India)

Source	Confidence	Used for
RBI, *Master Direction — Reserve Bank of India (Interest Rate on Deposits) Directions, 2025* (1 April 2025) — https://www.rbi.org.in/ · mirror: https://www.gujfed.com/circular/2025-master-direction-circular/1.4.2025%20Master%20Direction%20-%20Reserve%20Bank%20of%20India%20(Interest%20Rate%20on%20Deposits)%20Directions,%202025.pdf	**A**	Daily product basis; uniform rate to ₹1 lakh; differential rates above; rounding to nearest rupee for rupee deposits, two decimals for FCNR(B).
RBI, *Master Direction on Information Technology Governance, Risk, Controls and Assurance Practices* (7 Nov 2023, effective 1 Apr 2024) — https://vinodkothari.com/wp-content/uploads/2023/11/Comprehensive-IT-Directions-2023_v1-for-upload.pdf	**A**	Board-approved IT governance framework and Change Management Policy requirement.
RBI action on HDFC Bank, Dec 2020 — https://theprint.in/economy/why-rbi-has-restricted-hdfc-bank-from-launching-new-digital-facilities-issuing-credit-cards/556721/ · https://www.business-standard.com/article/finance/hdfc-bank-submits-action-plan-to-rbi-hopes-to-fix-outage-issue-in-3-months-121012200778_1.html	**B**	Digital 2.0 halt, credit-card issuance freeze, external IT audit, prior penalties for Nov 2018 / Dec 2019 outages, Aug 2021 partial relief.

Regulation and failure (UK — the TSB case)

Source	Confidence	Used for
Bank of England / PRA news release, Dec 2022 — https://www.bankofengland.co.uk/news/2022/december/tsb-fined-for-operational-resilience-failings	**A**	£48.65m joint FCA/PRA fine for operational risk management and governance failings.
Computer Weekly — https://www.computerweekly.com/news/252528519/TSB-hit-with-huge-fine-after-IT-migration-disaster	**B**	Migration timeline, Proteo4UK, customer impact.
Futurum Group — https://futurumgroup.com/insights/tsb-bank-fined-62m-for-a-failed-mainframe-migration-a-cautionary-tale-we-can-learn-from/	**B**	~£400m total cost; £32.7m redress; former CIO fined £81,620.
Celent — https://www.celent.com/en/insights/771585276	**B**	Migration lessons.

Indian banking modernisation economics

Source	Confidence	Used for
BCG via Business Standard, Aug 2024 — https://www.business-standard.com/industry/banking/indian-lenders-need-1-billion-upgrade-core-bank-systems-says-bcg-124080101587_1.html	**B** (reporting an **A** study)	~$1bn over 5–10 years; 1990s monolithic cores; IT spend 5% vs 7–9% globally; 40,000+ RBI ombudsman digital-banking complaints FY22–FY23.
Dvara Research, *Modernisation of India's Banking Sector* — https://dvararesearch.com/wp-content/uploads/2024/01/Modernisation-of-Indias-Banking-Sector.pdf	**A**	Indian banking modernisation context.

Scale of the COBOL install base

Source	Confidence	Used for
Pragmatic Coders, *2025 Legacy Code Stats* — https://www.pragmaticcoders.com/resources/legacy-code-stats	**C**	70% of banks on legacy; >43% of banking systems on COBOL; 220bn lines; 92 of top 100 banks on mainframes; $3tn/day. Aggregator — verify any figure before putting it on a slide.
CAST Software — https://www.castsoftware.com/pulse/why-cobol-still-dominates-banking-and-how-to-modernize	**C**	Context.
DXC — https://dxc.com/insights/knowledge-base/blogs/why-banks-still-rely-on-cobol-driven-mainframe-systems	**C**	Context.
MarketsandMarkets, mainframe modernisation market — https://www.marketsandmarkets.com/PressReleases/mainframe-modernization.asp	**B**	$8.39bn (2025) → $13.34bn (2030), CAGR 9.7%.
360iResearch — https://www.360iresearch.com/library/intelligence/mainframe-modernization	**C**	Alternative range: $6–10bn 2025, 8–13% CAGR. Cite as a range.

Workforce figures (average age 55–58, 10%/yr retirement, ~24,000 US COBOL developers, 60% cite hiring as biggest challenge) come from Computerworld (dated), AFCEA, DistantJob, Metaintro and DataField.Dev — all C, several recycling the same unsourced numbers. Treated as [contested] throughout `01`.

Peer-reviewed / arXiv — translation reliability

Source	Confidence	Used for
Pan et al., *Lost in Translation: A Study of Bugs Introduced by LLMs while Translating Code* — https://arxiv.org/abs/2308.03109	**A**	1,700 samples; 2.1%–47.3% correct; 43K+ translations, 1,748 labelled bugs, 1,365 bug-fix pairs, 15 bug categories; prompt crafting +5.5%.
Beyond Translation Accuracy: Addressing False Failures in LLM-Based Code Translation — https://arxiv.org/html/2605.02195v3	**A**	Silent failures from operator-semantics differences (modulo on negatives).
Hallucinations in LLM-Based Code Summarization: Unveiling, Detection, and Mitigation, PACMSE — https://dl.acm.org/doi/10.1145/3808139	**A**	Hallucination rates 66% → 59%; Hallu-Det F1 0.95. Supports "a confident wrong rule is worse than no rule."
Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code — https://arxiv.org/abs/2404.00971	**A**	Hallucination taxonomy: 3 primary, 12 specific categories.

Peer-reviewed / arXiv — COBOL translation validation

Source	Confidence	Used for
IBM Research, *Automated Testing of COBOL to Java Transformation* — https://arxiv.org/abs/2504.10548	**A**	*"the resulting code cannot be trusted to correctly translate the original code"*; symbolic execution generating unit tests from COBOL, mocked dependencies, converted to JUnit; feedback loop into the model.
Quality Evaluation of COBOL to Java Code Transformation (ASE 2025) — https://arxiv.org/abs/2507.23356	**A**	Analytic checkers + LLM-as-a-judge; CI integration; large-scale benchmarking.
SEDCoT: Enhancing LLM-Based COBOL Code Translation via Symbolic Execution and Delta Debugging — https://arxiv.org/abs/2607.04092	**A**	COBOL as a low-resource language; three phases (translate / symbolic-execution repair / delta debugging); ≥12% over SOTA.
Deterministic vs. LLM-Controlled Orchestration for COBOL-to-Python Modernization — https://arxiv.org/abs/2605.09894	**A**	Deterministic matches agentic accuracy; up to 3.5× fewer tokens; better worst-case robustness and lower variance.

Added by the verification pass (see `07 - Verification Pass.md`)

Source	Confidence	Used for
Agentic Method for Deterministic Validation of Legacy Code Migration (**Locksmith**), arXiv 2607.28271, 30 July 2026 — https://arxiv.org/abs/2607.28271	**A**	Closest prior art. Legacy as execution oracle via "Parity Gate"; both targets run **off-mainframe on commodity hardware** with mocks; "Witness Search" input generation; parity-preserving mutations; "Locked Paragraphs". 3 case studies, 430–4,114 LOC, **91.90% branch coverage**, full parity on accepted tests. Confirmed to have **no business-rule verdicts and no regulatory checks**.
AgentModernize: Preserving Business Logic in Legacy Modernization with Multi-Agent LLMs and Behavioral Specification Graphs, arXiv 2605.17535 — https://arxiv.org/pdf/2605.17535	**A**	**Business Rule Preservation Score** and **Behavioural Equivalence Rate** — falsifies the original "nobody reports per-rule" claim. BSG captured **91.2% of gold-standard rules**; BER **9.4% / 8.1% / 19.4%** across models, baselines **0.0%**.
RBI, *Master Direction — Reserve Bank of India (Interest Rate on Deposits) Directions, 2016* (3 Mar 2016, updated 7 Jun 2024) — https://www.rbi.org.in/Scripts/BS_ViewMasDirections.aspx?id=10296	**A**	**Primary source, replaces the 2025 citation.** Clause 6(a) daily product basis; 6(a)(1)–(2) uniform rate to ₹1 lakh and differential above; 4(f) rounding to nearest rupee / two decimals FCNR(B).
FCA, *TSB fined £48.65m for operational resilience failings* — https://www.fca.org.uk/news/press-releases/tsb-fined-48m-operational-resilience-failings	**A**	Fine figure and £32.7m redress, from the regulator. Note the FCA says "*a significant proportion of its 5.2 million customers*" — the 1.9m figure is press reporting.
The Register (24 Apr 2018) — https://www.theregister.com/2018/04/24/gov_demands_urgent_answers_to_tsb_it_meltdown/ · IEEE Spectrum — https://spectrum.ieee.org/new-software-system-snags-tsbs-online-and-mobile-banking-customers-in-uk	**B**	Independent corroboration of the 1.9 million figure.
EvolveWare, *Modernization use cases for business rules* — https://evolveware.com/6-modernization-use-cases-for-business-rules-in-system-analysis-and-design/	**C**	Markets rule extraction for regulatory audit —

Source	Confidence	Used for
		but **static documentation only**, no execution and no behavioural comparison against a regulation. Supports the N1 gap.
Automated Validation of COBOL to Java Transformation, arXiv 2506.10999 — https://arxiv.org/abs/2506.10999	**A**	Further translation-validation work; surfaced during verification, not read in full.

Peer-reviewed — business rule extraction

Source	Confidence	Used for
A-COBREX: A Tool for Identifying Business Rules in COBOL Programs, ICSE 2025 (IBM Research) — https://research.ibm.com/publications/a-cobrex-a-tool-for-identifying-business-rules-in-cobol-programs	**A**	Precision 62.21%, recall 74.12% (fuzzy match, 27 annotated programs).
LLM Vs Rule-Based — The COBRAIN Tool and An Empirical Study on Extracting Business Rules from COBOL, EASE 2025 — https://dl.acm.org/doi/10.1145/3756681.3756982	**A**	COBRAIN precision 1.0 / recall 0.746 vs COBREX; F1 0.73 vs COBREX 0.59.
Cosentino et al., *Extracting Business Rules from COBOL: A Model-Based Framework* — https://inria.hal.science/hal-00869235/document	**A**	Model-based representation, business-concept variable identification, code slicing.
Business as Rulesual: A Benchmark and Framework for Business Rule Flow Modeling with LLMs — https://arxiv.org/abs/2505.18542	**A**	BREX benchmark; 13 SOTA LLMs; expert annotation with ICC/Kappa agreement.

Testing theory

Source	Confidence	Used for
Barr, Harman, McMinn, Shahbaz, Yoo, *The Oracle Problem in Software Testing: A Survey* (IEEE TSE, 2015)	**A**	The oracle problem; differential testing as a mitigation.
Segura et al., *Metamorphic Testing: A Review of Challenges and Opportunities*, ACM CSUR 51(1) — https://dl.acm.org/doi/10.1145/3143561	**A**	Metamorphic relations; oracle-problem alleviation.
Cadar, Dunbar, Engler, *KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs*, OSDI 2008 — https://www.usenix.org/legacyurl/klee-unassisted-and-automatic-generation-high-coverage-tests-complex-systems-programs-0	**A**	Symbolic execution reference implementation.
Boundary Value Test Input Generation Using Prompt Engineering with LLMs — https://arxiv.org/html/2501.14465v1	**A**	LLM-driven boundary value generation; fault detection and coverage.
Twitter Diffy — https://github.com/twitter-archive/diffy · https://github.com/opendiffy/diffy	**A**	Multicast proxy; response comparison; "if two implementations return similar responses across a sufficiently large and diverse set of requests, they can be treated as equivalent"; used at Twitter, Airbnb, Baidu, ByteDance.
Microsoft, *Shadow Testing* (Code-with Engineering Playbook) — https://microsoft.github.io/code-with-engineering-playbook/automated-testing/shadow-testing/	**A**	Shadow testing definition and practice.

COBOL technical

Source	Confidence	Used for
GnuCOBOL project — https://gnucobol.sourceforge.io/ · https://sourceforge.net/projects/gnucobol/files/gnucobol/3.2/	**A**	Licensing (GPL/LGPL); 19 dialects; `-std=ibm-strict`; Enterprise COBOL 6.3 reserved words.
GnuCOBOL bug #997 (coverage on Ubuntu 24.04) — https://sourceforge.net/p/gnucobol/bugs/997/	**A**	Known coverage-build friction.
NIST CCVS85 test suite — https://github.com/Zaneham/nist-cobol85-test-suite · https://github.com/z390development/nistcobol85	**A**	512 programs, 8,800+ test cases, public domain. GnuCOBOL passes 9,700/9,748.
IBM, *Mapping between COBOL and Java data types* — https://www.ibm.com/docs/en/cobol-zos/6.4.0?topic=ci-mapping-between-cobol-java-data-types-non-oo-coboljava-interoperability	**A**	Precision loss on COMP-1/COMP-2 ↔ float/double; BigDecimal as legal partner for zoned and packed decimal.
IBM, *PACKED-DECIMAL (COMP-3)* — https://www.ibm.com/docs/en/SS6SG3_6.3.0/perf/packed_decimal.html	**A**	Packed decimal representation and exactness.
ProLeap COBOL parser — https://github.com/uwol/proleap-cobol-parser	**A**	ANTLR4 grammar; AST + ASG with data/control flow; `COPY` / `REPLACE` preprocessing; `EXEC SQL` / `EXEC CICS` extracted as text; passes NIST; MIT; applied to banking and insurance COBOL.
GCBLUnit — https://github.com/OlegKunitsyn/gcblunit	**A**	GnuCOBOL unit testing with JUnit-format reporting.
gcov (GCC) — https://en.wikipedia.org/wiki/Gcov	**B**	`-fprofile-arcs -ftest-coverage`; per-statement execution counts.
`aws-samples/aws-mainframe-modernization-carddemo` — https://github.com/aws-samples/aws-mainframe-modernization-carddemo	**A**	Apache 2.0 COBOL credit-card application; deliberately varied coding styles; `CBACT04C.cbl` interest calculation; AWS's own tooling benchmark.

Vendor landscape

Source	Confidence	Used for
IBM, *About watsonx Code Assistant for Z* — https://www.ibm.com/docs/en/watsonx/watsonx-code-assistant-4z/2.x?topic=welcome-overview-watsonx-code-assistant-z	**A**	Validation Assistant; semantic equivalence via automated test generation.
IBM Research blog — https://research.ibm.com/blog/watsonx-code-assistant-for-z-is-the-rosetta-stone-for-mainframes	**A**	Same inputs through both, divergence flagged, developer assisted to fix.
AWS, *Refactoring applications automatically with AWS Blu Age* — https://docs.aws.amazon.com/m2/latest/userguide/refactoring-m2.html	**A**	Automated refactoring; Blu Age Compare; reference datasets from mainframe/iSeries.
AWS, *Mainframe Modernization Application Testing (Preview)* — https://aws.amazon.com//about-aws/whats-new/2023/11/aws-mainframe-modernization-application-testing-preview	**A**	Test capture, automated replay, comparison, regression at scale.
AWS, *AWS Transform for mainframe is now generally available* (May 2025) — https://aws.amazon.com/about-aws/whats-new/2025/05/aws-transform-mainframe-generally-available	**A**	First agentic AI mainframe modernisation service; ADP 80% rule-extraction time reduction, >90% manual effort reduction.
AWS, *Transform for mainframe now supports application reimagining* (Dec 2025) — https://aws.amazon.com/about-aws/whats-new/2025/12/transform-mainframe-application-reimagining	**A**	Business logic extraction, activity analysis, code decomposition.
Mechanical Orchard / Thoughtworks launch of Imogen — https://www.thoughtworks.com/en-us/about-us/news/2025/mechanical-orchard-launch-imogen-1st-partner-thoughtworks	**A**	Generate-validate test loop; behaviour and data flows; equivalence proof.
Mechanical Orchard platform — https://www.mechanical-orchard.com/platform	**A**	Behavioural specification from real data flows; audit-level parity before cutover.
HyperFRAME Research, *The Behavior-First Paradigm* (22 May 2026) — https://hyperframeresearch.com/2026/05/22/the-behavior-first-paradigm-moving-mainframe-modernization-past-llm-wishful-thinking/	**B**	Category naming; "compounding logical discrepancies"; vendor positioning; 14% fully modernised data architecture; 78%/37% AI strategic-importance vs structured evaluation.
HyperFRAME Research, Imogen Summer 2026 update (13 Jul 2026) — https://hyperframeresearch.com/2026/07/13/mechanical-orchards-imogen-update-tests-whether-ai-mainframe-modernization-can-move-from-translation-to-proof/	**B**	AWS Transform business-rule extraction integration; Leidos partnership.
Cognizant, *AI-led mainframe modernization* — https://www.cognizant.com/us/en/services/cloud-solutions/mainframe-modernization	**A** (about its own offering)	Accelerators extracting business rules, generating specifications, rewriting to cloud-native Java; Flowsource, Neuro, Skygrade, ZDLC.
Cognizant, *How generative AI accelerates mainframe modernization* — https://www.cognizant.com/us/en/insights/insights-blog/generative-ai-for-mainframe-modernization	**A**	Legacy understanding, documentation gap-filling, rule extraction, COBOL→Java.
TCS MasterCraft TransformPlus — https://www.tcs.com/what-we-do/products-platforms/tcs-mastercraft/solution/tcs-mastercraft-legacy-modernization-stay-relevant-modern	**A**	Analyze-Strategize-Execute; automated rule extraction; COBOL→layered cloud-ready Java; v5.0 ML enhancements.
Swimm, *5 COBOL Business Rules Extraction Tools* — https://swimm.io/learn/cobol/5-cobol-business-rules-extraction-tools-to-know-in-2026	**C**	IBM ADDI capabilities; the "high recall, low precision"

Source	Confidence	Used for
		characterisation of automated extraction.

Domain — interest calculation

Source	Confidence	Used for
Thomson Reuters Practical Law, *Day Count Convention* — https://uk.practicallaw.thomsonreuters.com/w-026-0314	**A**	30/360 and Actual/365 definitions.
Fintelligents — https://fintelligents.com/day-count-convention/	**C**	India uses 30/360 for G-Secs.

Banking migration practice

Source	Confidence	Used for
Shinetech, *Why Dual-Running Needs Reconciliation in Legacy Migration* — https://www.shinetechsoftware.com/insights/why-dual-running-needs-reconciliation-in-legacy-migration/	**C**	Dual-running without reconciliation creates false confidence.
10x Banking, *Core banking migration strategies* — https://www.10xbanking.com/insights/core-banking-migration-strategies-choosing-the-right-path-to-a-4th-generation-platform	**C**	Parallel run, dual-write, shadow accounting, blue-green.

Gaps — what could not be established

- **Judging criteria, rounds, or prizes for Digital Nurture Hackathon 2026.** Nothing in the organiser notices and nothing findable. All positioning advice in `05` assumes a technically literate Cognizant panel, which is an inference.
- **Whether GnuCOBOL and IBM Enterprise COBOL agree on specific `COMP-3` edge cases.** Would need empirical testing against a real Enterprise COBOL installation, which the team does not have. Handled as a stated limitation rather than a resolved question.
- **A published tool or paper reporting verification coverage per extracted business rule.** Searched across the extraction and validation literature and the vendor documentation; none found. This is the basis for the originality claim in `03 §3a`, and it is an absence-of-evidence finding — a more exhaustive search could overturn it.